Projet intégrateur d'IA générative multimodale et agentique

Mohamed Youssfi, Enseignant Chercheur, ENSET Mohammedia, Directeur Laboratoire Informatique, Intelligence Artificielle et Cyber Sécurité



IA Discriminative — *Apprendre à distinguer*

L'IA discriminative apprend la frontière entre les classes (ou la relation entrée → sortie) à partir de données étiquetées. Elle prédit une étiquette ou une valeur pour une nouvelle donnée, mais ne génère pas de nouvelles données.



Chat
Chien
Oiseau
Voiture
...

Données étiquetées
(exemples d'apprentissage)

Données d'entrée (x)

Modèle discriminatif
(ex. CNN, ResNet, SVM, ...)



Apprend P(y | x)

Prédiction
(Classe ou valeur)

Chat	0.92
Chien	0.05
Oiseau	0.02
Voiture	0.01

...

Tâches typiques :
Classification, détection, régression, reconnaissance, ...

VS

IA Générative — *Apprendre à créer*

L'IA générative apprend la distribution des données P(x) (ou P(x | condition)) et est capable de générer de nouvelles données réalistes et cohérentes, telles que des images, du texte, de l'audio, de la vidéo ou du code.

Prompt / Condition (exemple)

- « Un chat portant un nœud papillon, assis dans un salon élégant »
- « A scenic mountain landscape at sunset »
- « Écris un court poème sur l'espoir »

...

Modèle génératif
(ex. GAN, VAE, Diffusion, Transformers, ...)



Apprend P(x) ou P(x | condition)

Nouvelle donnée générée



Image générée



Image générée

Dans chaque aube naît une chance
Un rêve qui se lève et avance
L'espoir éclaire même la nuit
Et fait de demain un cœur qui vit

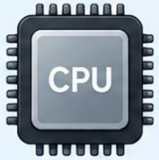
Texte généré

...

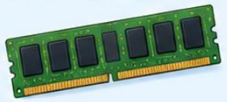
Tâches typiques :
Génération d'images, de texte, d'audio, de vidéo, de code, de données synthétiques, ...

CPU (Hôte)

Traitement général, préparation des données



RAM (mémoire principale)
Stocke les données, le code Python, les tensors sur CPU, etc.



- Exécute le code Python (PyTorch)
- Prépare et prétraite les données
- Gère l'entraînement (boucle, optimisation, ...)
- Peut effectuer les calculs sur CPU (si pas de GPU)

.to("cuda") — Transfert de données — .to("cpu")

GPU (Device)

Calculs massivement parallèles pour les tenseurs



VRAM
(mémoire du GPU)
Stocke les tensors, les paramètres du modèle, les activations, ...



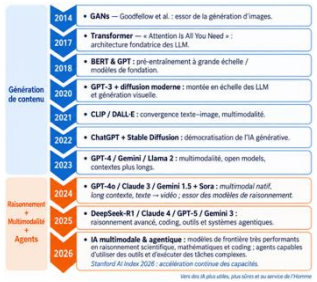
- Exécute les opérations sur les tensors (très parallèles)
- Idéal pour l'entraînement des grands modèles
- Mémoire dédiée : la VRAM (plus rapide que la RAM)
- Nécessite CUDA (NVIDIA) pour être accessible depuis PyTorch

CUDA

Couche logicielle de NVIDIA

PyTorch (torch.cuda)
↓
CUDA Runtime (cuBLAS, cuDNN, ...)
↓
Pilote NVIDIA (Driver)

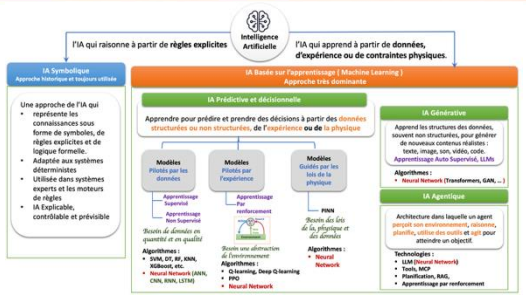
NVIDIA



QWEN-3

Gemma 4





Projet IA Générative Multi Modale

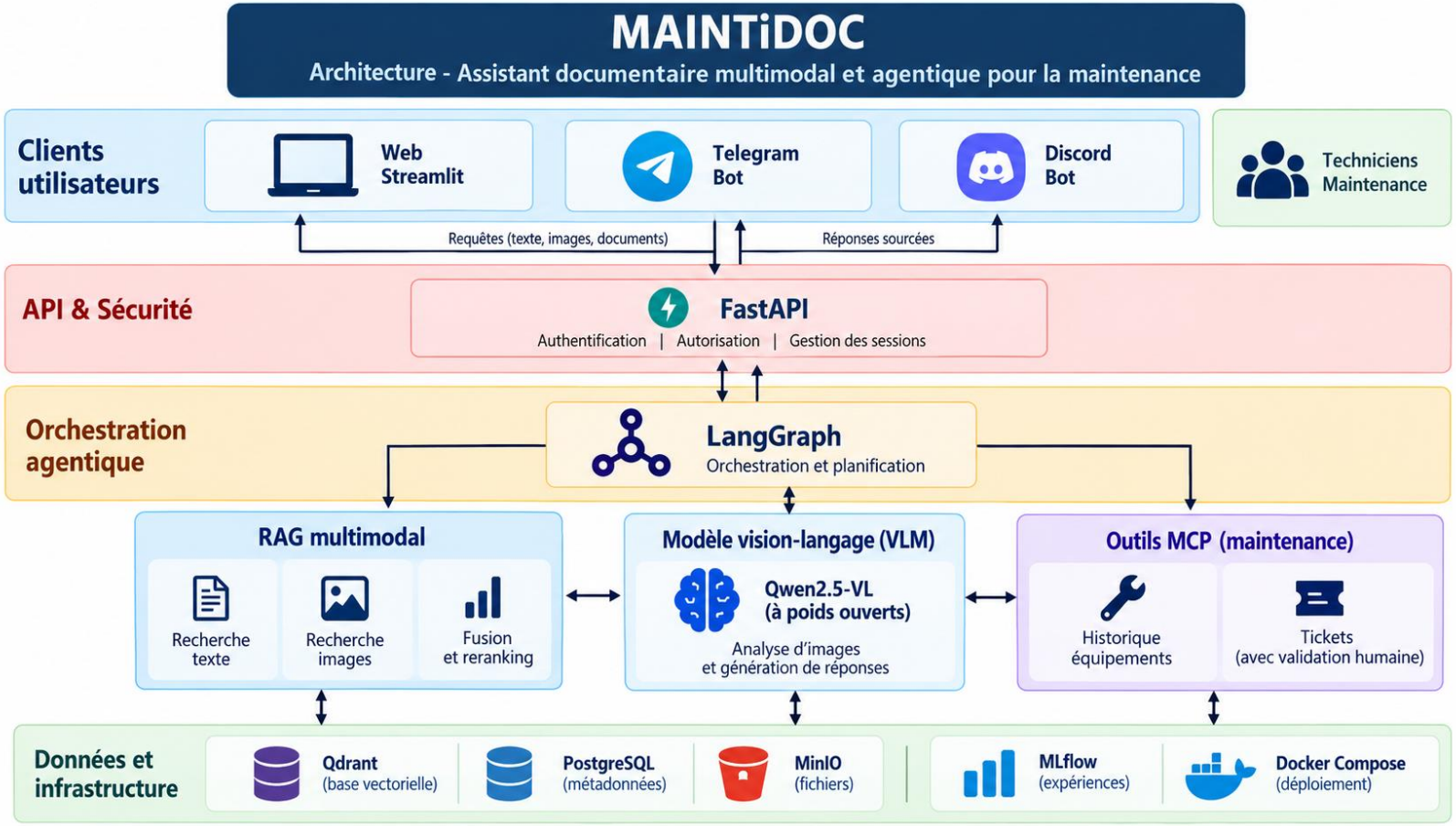
- MaintiDoc est un assistant destiné aux techniciens de maintenance. Il exploite des manuels, des procédures, des schémas et des photographies pour répondre à des questions techniques avec des références vérifiables.
  - Il peut consulter l'historique des interventions et préparer un ticket de maintenance. La création effective du ticket nécessite une validation humaine.
  - Exemple de demande : « Voici une photo du panneau de commande. Que signifie ce voyant orange ? Consulte les dernières interventions et prépare un ticket si nécessaire. »

Étape 1 — RAG Agentique Multi Modal

Étape 2 — IA multimodale et fine-tuning

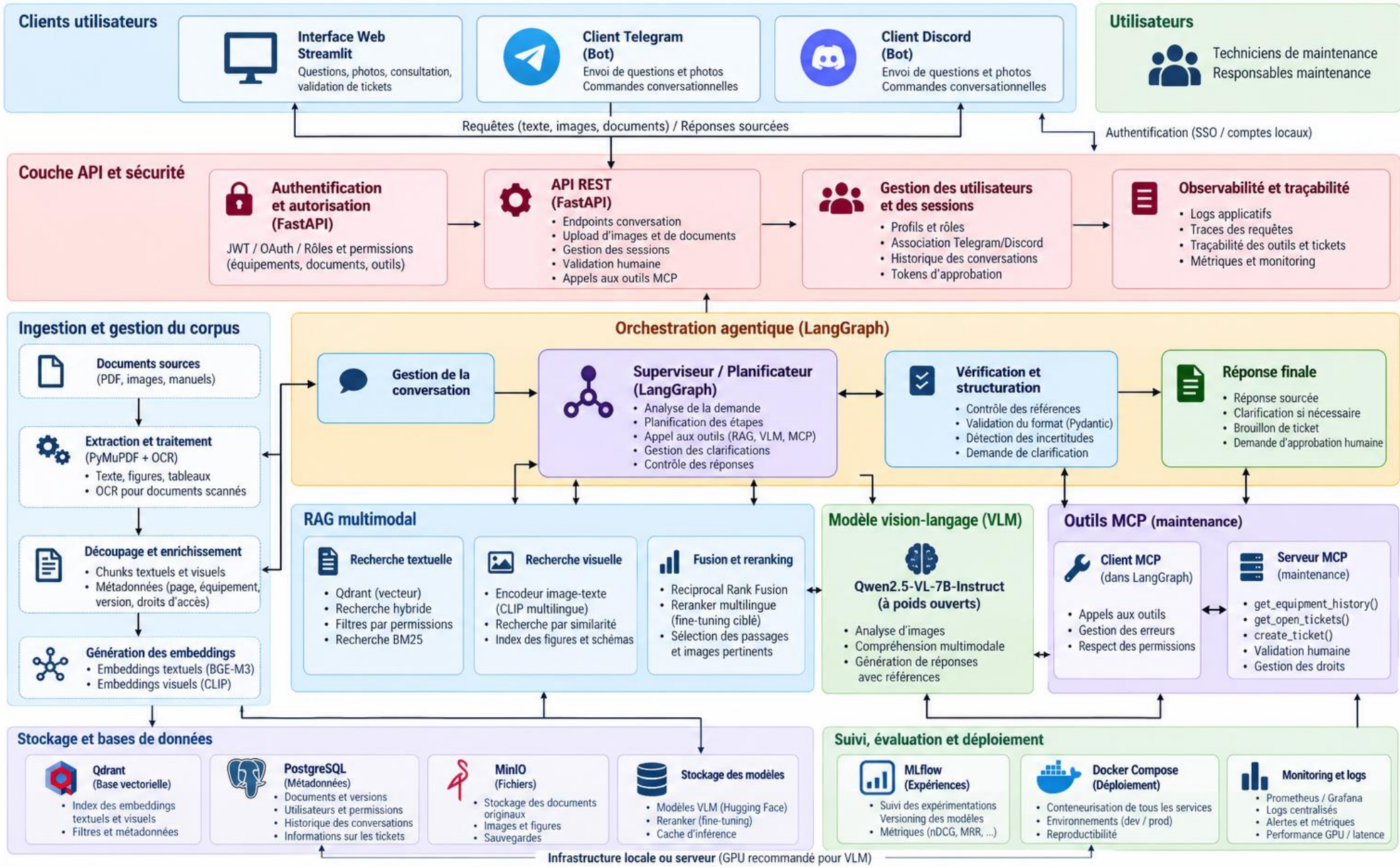
Étape 3 — Orchestration agentique et MCP

Étape 4 — Client Web Telegram/Discord, sécurité et déploiement



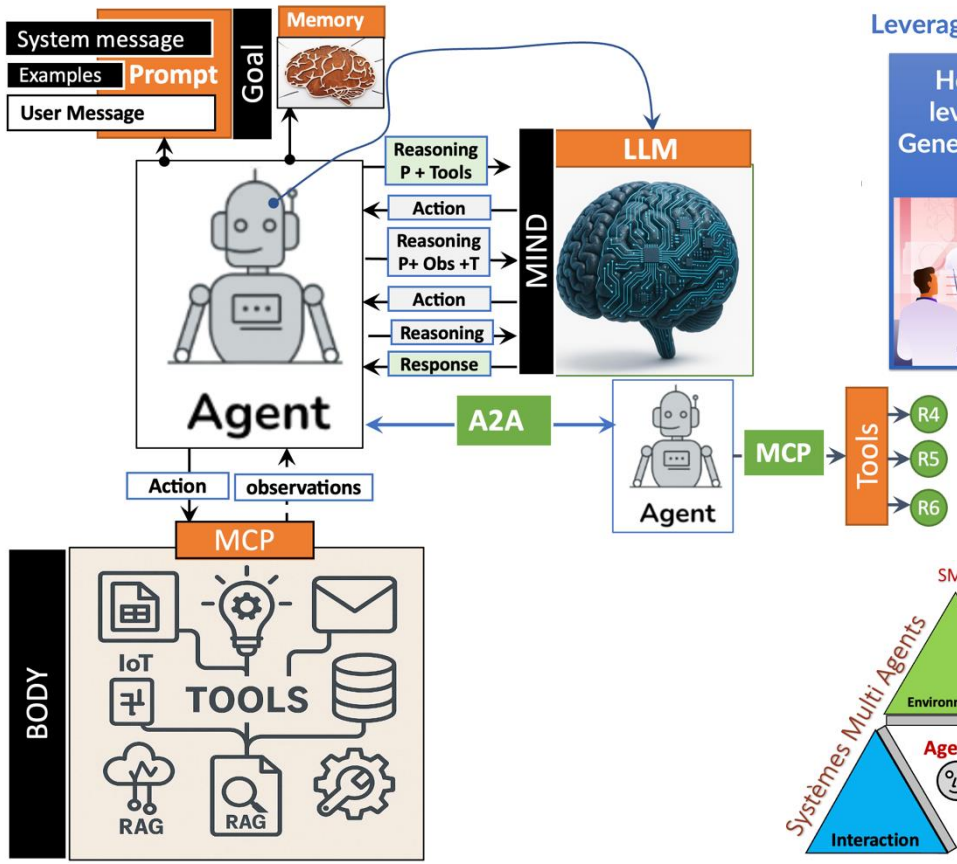
MAINTiDOC

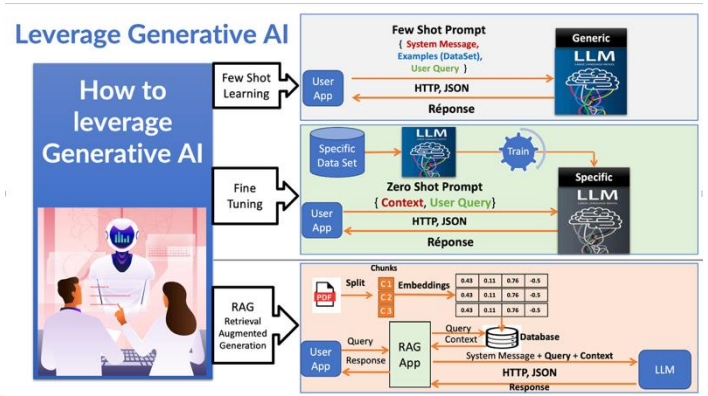
Architecture détaillée - Assistant documentaire multimodal et agentique pour la maintenance

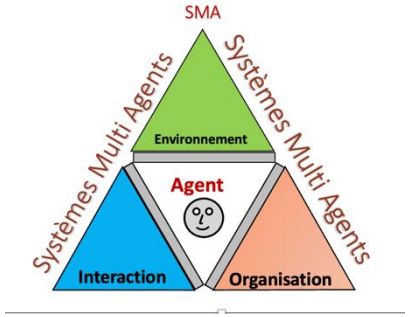


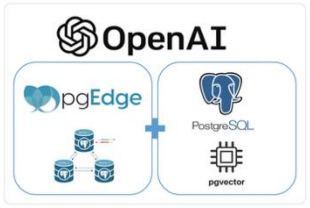
Prompt Engineering
Retrieval Augmented Generation (RAG)



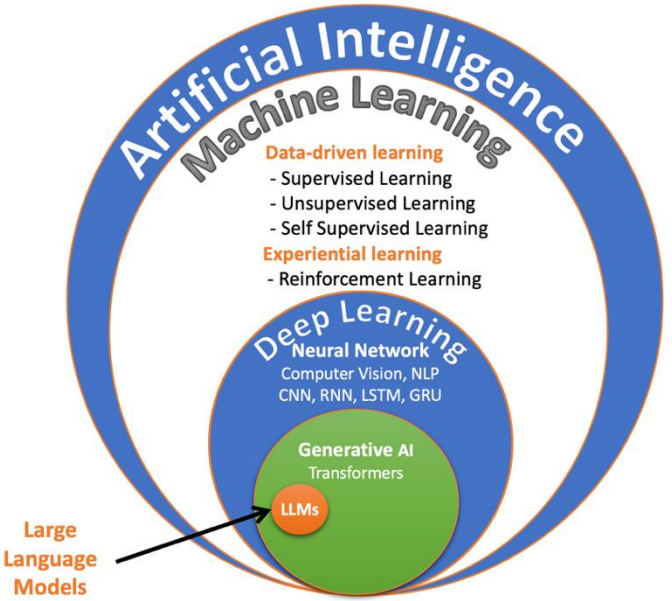












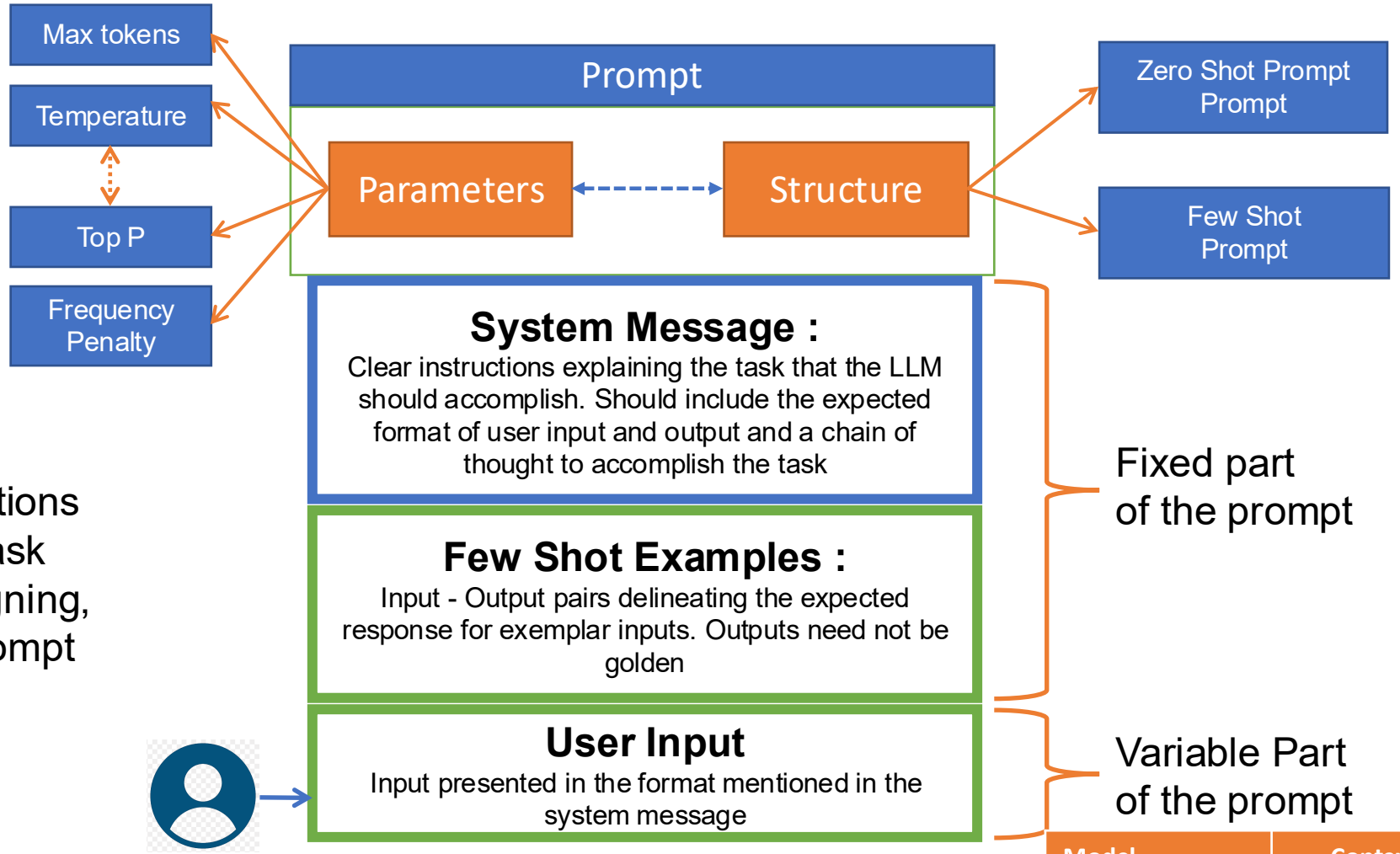
Prompt Engineering

Length of input + output

More temperature = More randomness in response

More Top P = More tokens selected for completion ; nucleus sampling échantillonnage du noyau

More FP = Less chance of tokens repeating



- **Prompt** = Specific set of instructions sent to a LLM to accomplish a task
- **Engineering** = Process of designing, evaluating and deploying the prompt for specific tasks
  1. Define Tasks and Metrics
  2. Data Set (Examples, Gold Examples)
  3. Design Prompts
  4. Evaluate Prompts
  5. Deploy and Monitor Prompt

https://platform.openai.com/tokenizer

https://platform.openai.com/

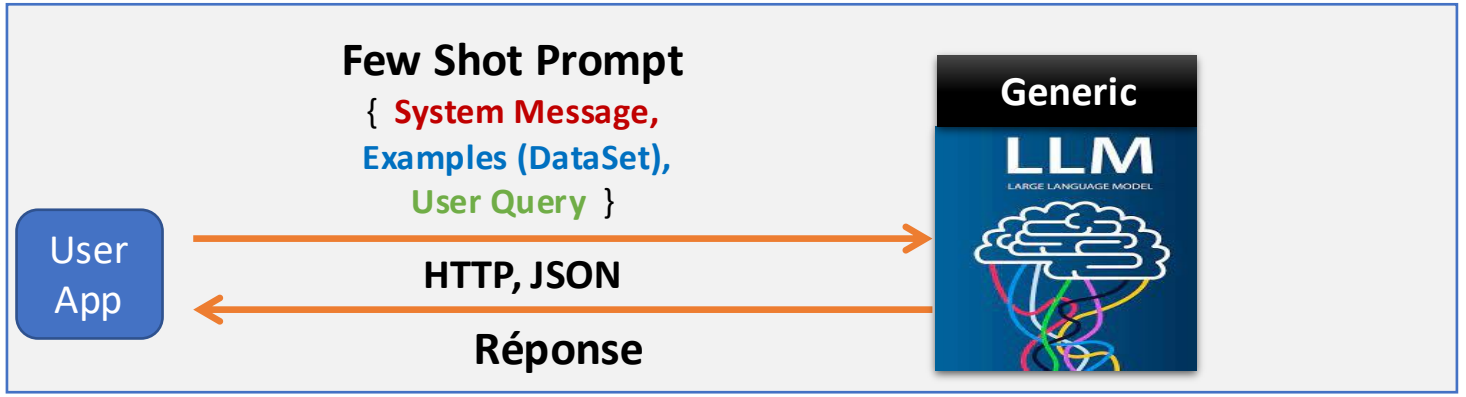
Model	Context Window
Gpt-3.5-turbo	4096
Gpt-4-tubo	128 000
Mixtral-8x7B	128 000

Leverage Generative AI

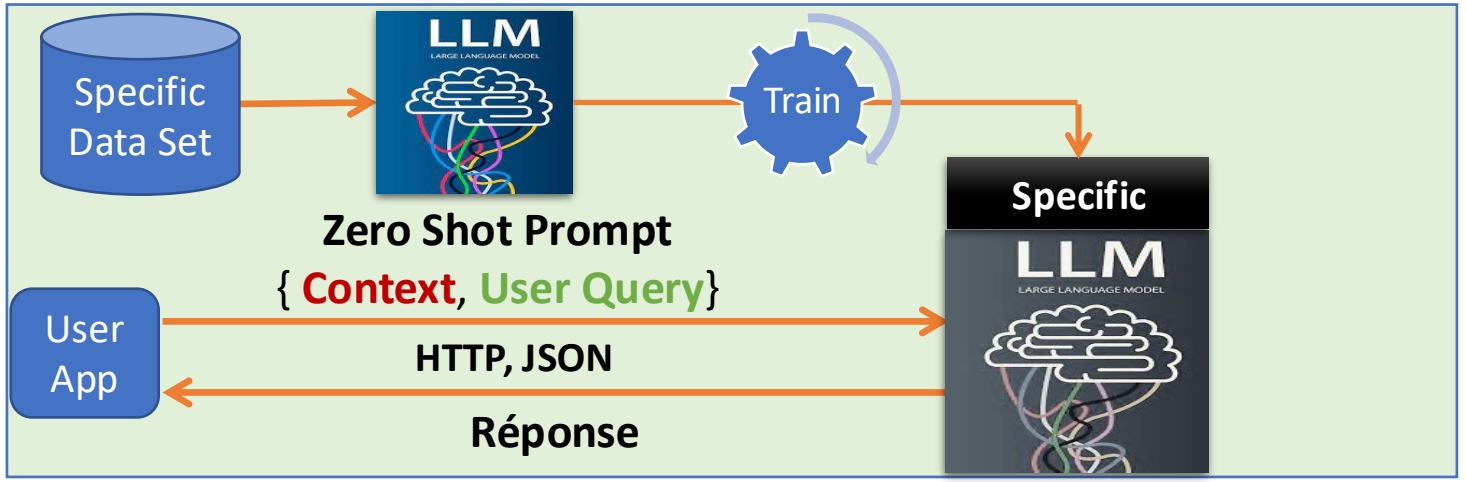
How to leverage Generative AI



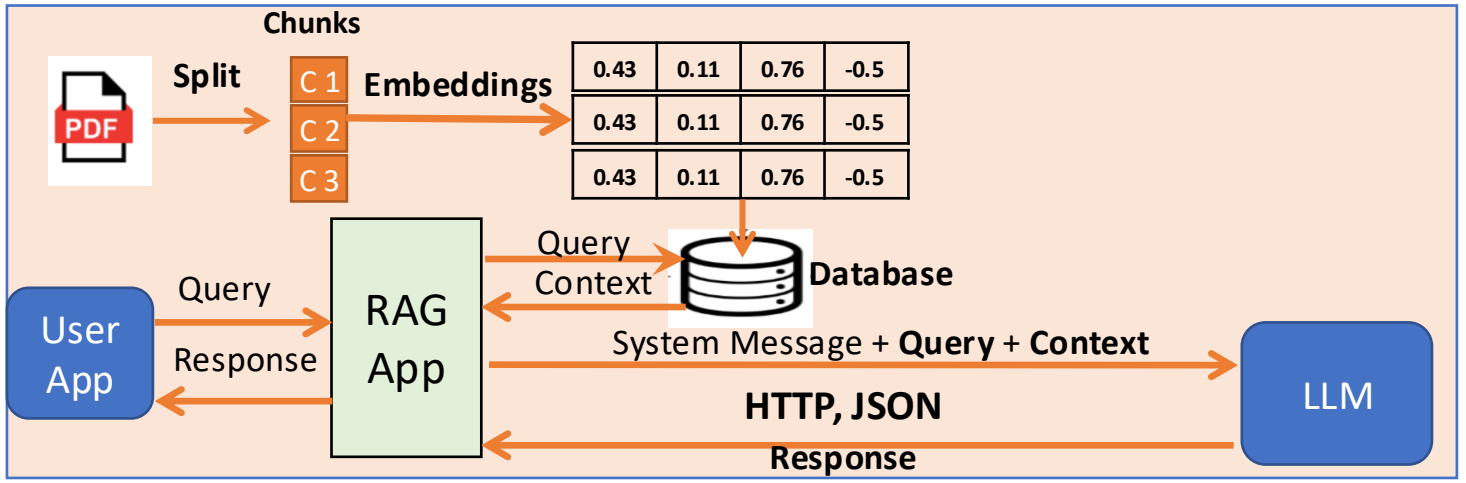
Few Shot Learning



Fine Tuning



RAG Retrieval Augmented Generation

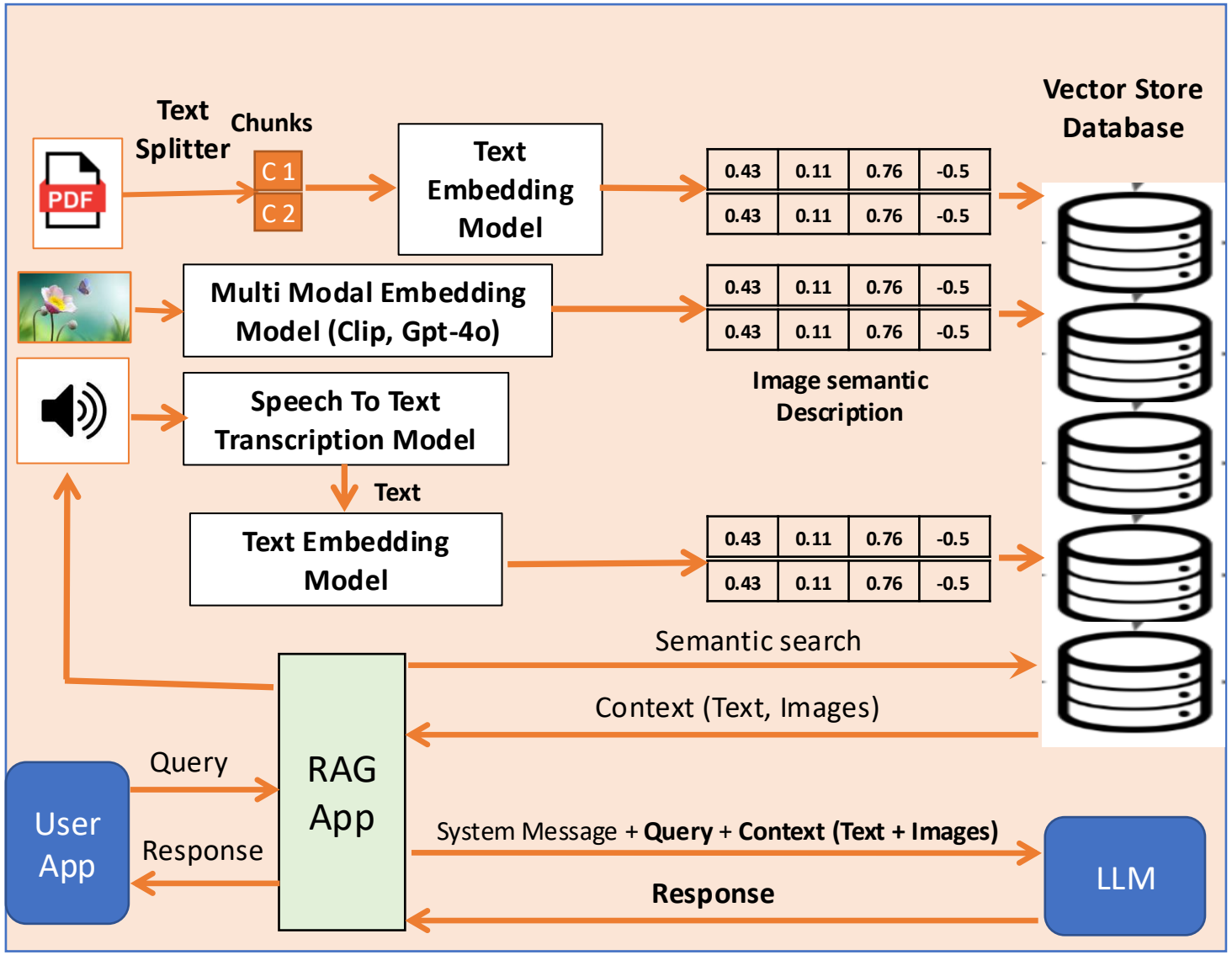


Multi Modal Retrieval Augmented Generation

Multi Modal RAG



Multi RAG Retrieval Augmented Generation



Step 1: Creating a Vector Database (Indexing)

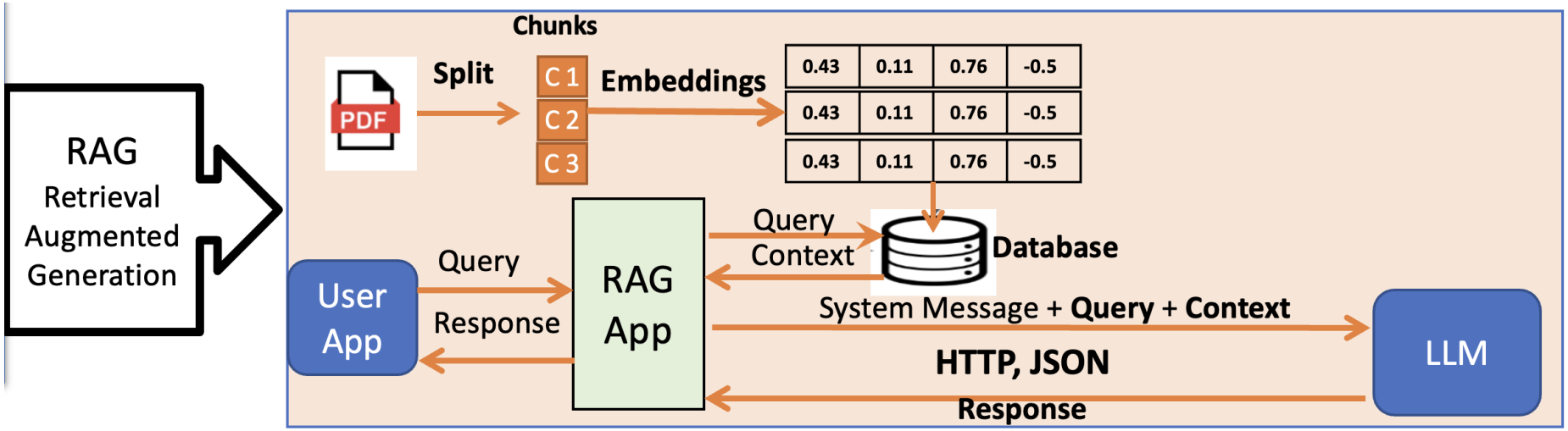
- Nous créons une **base de données vectorielle** en découpant le document d'entrée en chunks, puis en associant à chaque chunk un **vecteur** à l'aide d'un modèle d'embeddings.
- Le processus global est décrit dans la figure ci-dessous.
- Une bonne méthode pour choisir un modèle d'embedding consiste à utiliser le [classement des embeddings](https://huggingface.co/spaces/mteb/leaderboard) et à sélectionner un modèle open source performant pour le cas d'usage visé.
- Un bon modèle d'embeddings **polyvalent** est [`**gte-large**`](https://huggingface.co/thenlper/gte-large).

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import PyPDFDirectoryLoader, PyPDFLoader
from langchain_community.document_loaders import PyPDFDirectoryLoader
from langchain_community.vectorstores import Chroma
from dotenv.ipython import load_dotenv
import os
```

```
pdf_file = "./pdfs/agentic-ai-foundations.pdf"
```

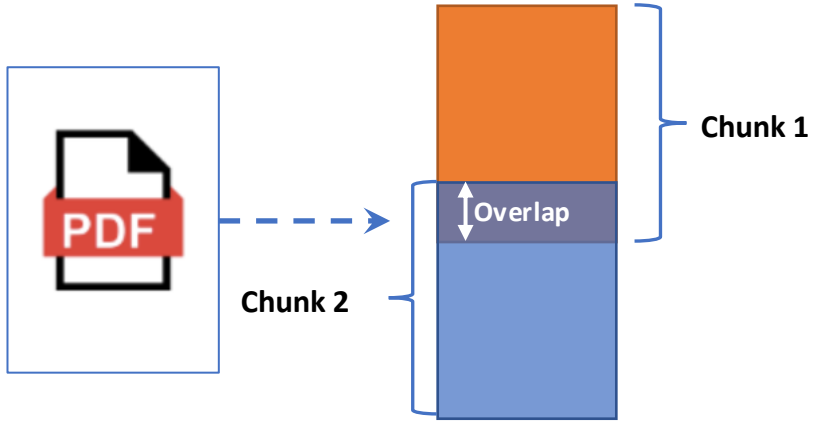
```
pdf_loader = PyPDFLoader(pdf_file)
```

```
loader = PyPDFDirectoryLoader(path = "./pdfs")
```



Chunking documents

- Une fois le choix du modèle d'embeddings effectué, nous pouvons lui fournir les documents en entrée.
- Cependant, comme les documents contiennent plusieurs pages (y compris des figures et des tableaux), il nous faut une méthode pour **parser** des portions individuelles du document, compatibles avec le **modèle d'embedding**.
- Nous savons également que le modèle d'embeddings ne peut pas traiter de texte au-delà d'une longueur de contexte fixe (**512 tokens** pour `**gte-large**`).
- Cela correspond approximativement à **400 mots** ou à environ **une page** de texte.
- En gardant cette contrainte à l'esprit, nous allons transformer le fichier PDF en **chunks** de texte ne dépassant pas **512 tokens**.




```
text_splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
  encoding_name='cl100k_base',
  chunk_size=512,
  chunk_overlap=16
)
```

```
chunks = loader.load_and_split(text_splitter)
```

- Une autre approche consisterait à découper le fichier par **sections** du rapport (par ex., facteurs de risque, procédures judiciaires, déclarations de sécurité).
- Une stratégie de **chunking** courante consiste à utiliser des **chunks de taille fixe** (telle que définie par le modèle d'embedding) et à appliquer un **léger chevauchement (Overlap)** entre les chunks
- L'utilisation d'un **chevauchement (Overlap)** permet d'assurer la continuité entre les chunks et de récupérer des chunks pertinents même lorsque l'information liée à la requête se trouve soit au début, soit à la fin du chunk.

Vector Store - ChromaDB, Embeddings

```
from langchain_openai import OpenAIEmbeddings
embedding_model = OpenAIEmbeddings(model='text-embedding-ada-002')
```

```
vectorstore = Chroma.from_documents(
  chunks,
  embedding_model,
  collection_name="Agentic_AI"
)
```

Model	~ Pages per dollar	Performance on MTEB eval	Max input
text-embedding-3-small	62,500	62.3%	8192
text-embedding-3-large	9,615	64.6%	8192
text-embedding-ada-002	12,500	61.0%	8192

```
retriever = vectorstore.as_retriever(
  search_type='similarity',
  search_kwargs={'k': 5}
)
```

```
retrieved_chunks=retriever.invoke("Quels sont les fonctions d'un agent?")
```

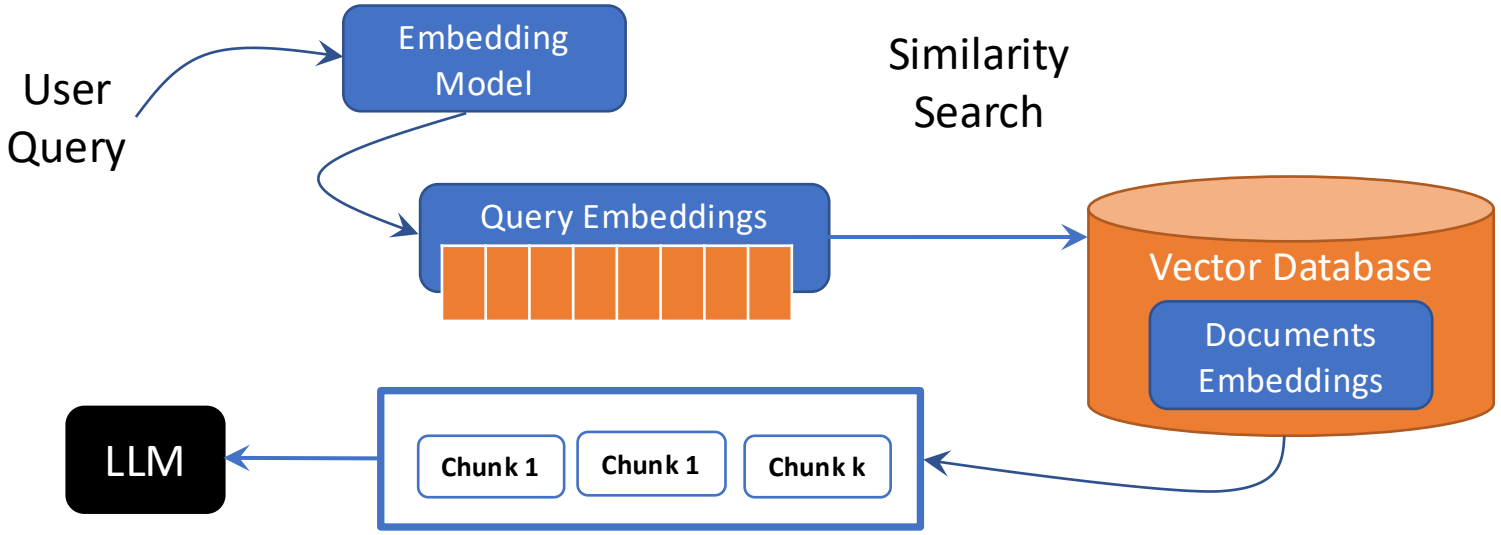
```
print(retrieved_chunks)
```

```
[Document(metadata={'page_label': '10', 'page': 14, 'creationdate': '2026-02-20T10:24:07+00:00', 'source': 'pdfs/agentic-ai-foundations.pdf', 'title': "AWS Directives prescriptives - Les fondements de l'IA agentique sur AWS", 'author': 'AWS', 'creator': 'ZonBook XSL Stylesheets with Apache FOP', 'producer': 'Apache FOP Version 2.6', 'total_pages': 90, 'keywords': "…."}, page_content="….."),
Document(….), Document(….), Document(….), Document(….)]
```

```
print(len(retrieved_chunks))
```

Step 2: Querying the Vector Database

- Une fois la **base de données vectorielle** mise en place, l'embedding généré à partir du texte de la requête est comparé à l'ensemble des embeddings de documents présents dans la base de données vectorielle.
- Les documents les plus similaires sont ensuite récupérés et présentés, avec la requête d'origine, dans le prompt.
- Enfin, dans le prompt, nous demandons au LLM de se concentrer uniquement sur le **contexte** pour répondre à la requête de l'utilisateur.
- L'idée clé ici est que la requête **et** le contexte sont tous deux **injectés dynamiquement** dans le prompt.




```
prompt_template = """

Answer the following question based only on provided context
The context is delimited by <context> tag
The user question is delimited by <question> tag
If the answer is not found in the context, answer : I DO NOT KNOW
<context>
 {context}
</context>
<question>
 {question}
</question>

"""
```

Step 3: Querying and Generation

```
query ="Nom, prénom et Liste des diplômes obtenus"
```

```
relevant_document_chunks = retriever.invoke(query)
context_list = [d.page_content for d in relevant_document_chunks]
context_for_query = ". ".join(context_list)
```

```
prompt = prompt_template.format(context=context_for_query, question=query)
```

```
resp = llm.invoke(prompt)
```

```
from IPython.display import Markdown
```

```
print(display(Markdown(resp.content)))
```

Nom, prénom : Mohamed Youssfi

Liste des diplômes obtenus :

- Doctorat d'Etat, Fac des sciences Rabat (2015)
- Doctorat de Troisième cycle, Fac des sciences Rabat (1996)
- Diplôme des Etudes Approfondies à la fac des sciences de Rabat (1994)
- Diplôme de Professeur Second Cycle de l'ENSET Mohammedia (1993)

```
prompt_template = """

Answer the following question based only on provided context
The context is delimited by <context> tag
The user question is delimited by <question> tag
If the answer is not found in the context, answer : I DO NOT KNOW
<context>
 {context}
</context>
<question>
 {question}
</question>
"""
```

Defining the RAG function for response

```
def RAG(query, llm=llm, prompt_template=prompt_template):
    context_docs = retriever.invoke(query)
    context_list = [d.page_content for d in context_docs]
    context_for_query = ". ".join(context_list)
    prompt = prompt_template.format(context=context_for_query, question=query)
    resp=llm.invoke(prompt)
    return resp.content
```

```
response = RAG("Fonction actuelle du Pr YOUSSFI")
print(display(Markdown(response)))
```

4 - Evaluation

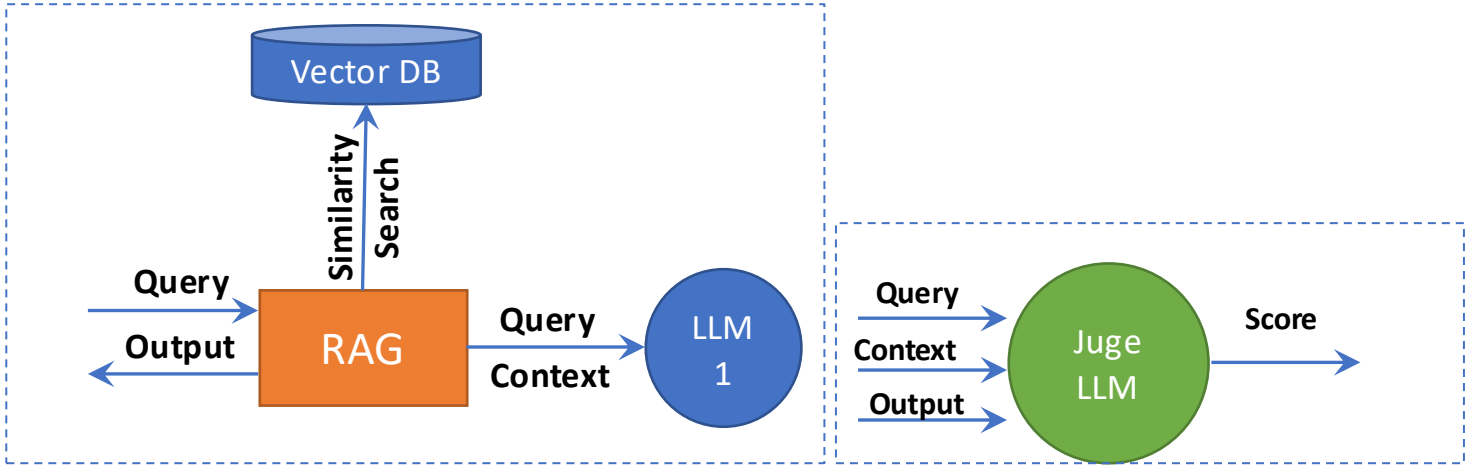
- Utilisons maintenant la méthode **LLM-as-a-judge** pour vérifier la qualité du système RAG selon deux paramètres : la récupération (**retrieval**) et la génération (**generation**). Nous illustrons cette évaluation à partir des réponses générées à la question de la section précédente.
- Nous utilisons le même modèle pour l'évaluation ; en d'autres termes, ici le LLM s'auto-évalue en notant la qualité de sa propre performance dans la tâche.

```
user_input ="État du résultat global consolidé"
```

```
relevant_document_chunks = retriever.invoke(user_input)
context_list = [d.page_content for d in relevant_document_chunks]
context_for_query = ". ".join(context_list)
```


```
relevant_document_chunks = retriever.invoke(user_input)
context_list = [d.page_content for d in relevant_document_chunks]
context_for_query = ". ".join(context_list)
```


```
user_message_template = """
 ###Question
 {question}
 ###Context
 {context}
 ###Answer
 {answer}
"""
```



4 - Evaluation

groundedness_rater_system_message="""
Vous êtes chargé d'évaluer des réponses générées par une IA à des questions posées par des utilisateurs.
On vous présentera une question, le contexte utilisé par le système d'IA pour générer la réponse, ainsi qu'une réponse générée par l'IA à la question.

Dans l'entrée, la question commencera par **###Question**, le contexte commencera par **###Context**, et la réponse générée par l'IA commencera par **###Answer**.

Critères d'évaluation :
La tâche consiste à juger dans quelle mesure la réponse respecte la métrique.

1 — La métrique n'est pas respectée du tout
2 — La métrique n'est respectée que dans une mesure limitée
3 — La métrique est respectée dans une bonne mesure
4 — La métrique est respectée en grande partie
5 — La métrique est entièrement respectée

Métrique :
La réponse doit être dérivée uniquement des informations présentées dans le contexte.

Instructions :

Écrivez d'abord les étapes nécessaires pour évaluer la réponse selon la métrique.
Donnez une explication étape par étape indiquant si la réponse respecte la métrique, en considérant la question et le contexte comme entrées.
Évaluez ensuite dans quelle mesure la métrique est respectée.
Utilisez les informations précédentes pour noter la réponse selon les critères d'évaluation et attribuer un score.
"""

4 - Evaluation

```
def evaluate(system_message,user_message_template, question, model=llm):
    retrieved_chunks=retriever.invoke(question)
    context_list = [d.page_content for d in retrieved_chunks]
    context = ". ".join(context_list)
    answer = RAG(question)
    prompt = f"""
     {system_message}\n
     USER:
     {user_message_template.format(question=question, context=context, answer=answer)}
    """
    juge_response= model.invoke(prompt)
    return juge_response.content
```

4 - Evaluation

```
resp=evaluate(groundedness_rater_system_message, user_message_template, user_input)
print(display(Markdown(resp)))
```

Pour évaluer la réponse selon la métrique, voici les étapes nécessaires :

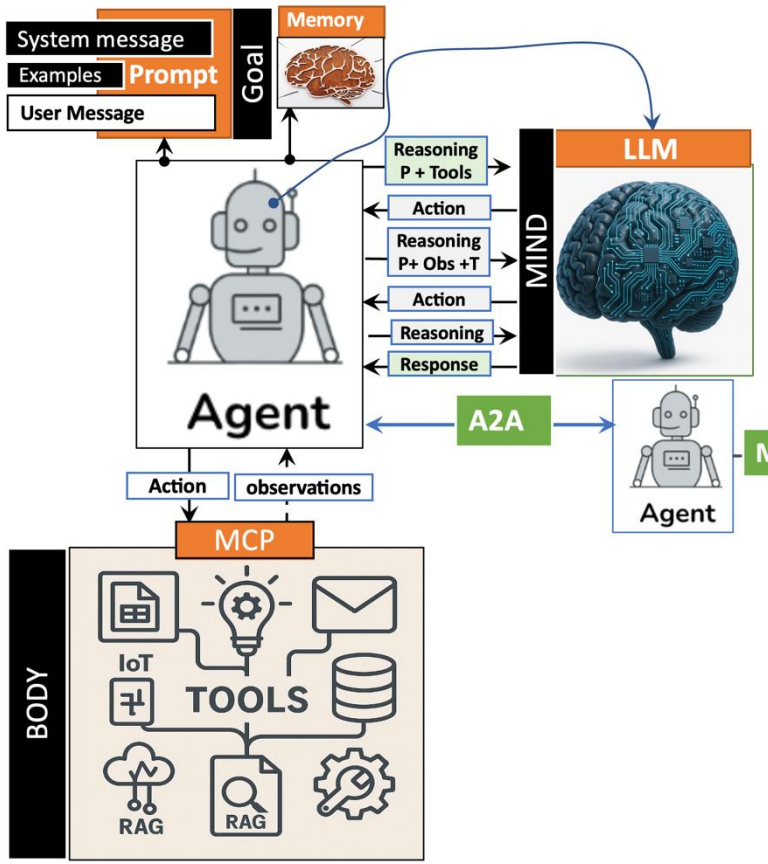
1. **Identifier la question** : La question demande l'état du résultat global consolidé.
2. **Analyser le contexte** : Le contexte fournit des informations financières consolidées pour le Groupe OCP au 31 décembre 2023, y compris le résultat global consolidé pour l'exercice 2023.
3. **Extraire les informations pertinentes du contexte** :
   - Le résultat global consolidé pour l'exercice 2023 est de 14 187 millions de dirhams.
   - La répartition est la suivante : part du Groupe est de 14 259 millions de dirhams et part des intérêts ne donnant pas le contrôle est de (72) millions de dirhams.
4. **Comparer la réponse avec les informations du contexte** :
   - La réponse indique que le résultat global consolidé pour l'exercice 2023 est de 14 187 millions de dirhams, ce qui correspond à l'information du contexte.
   - La répartition mentionnée dans la réponse (part du Groupe 14 259 millions de dirhams et part des intérêts ne donnant pas le contrôle (72) millions de dirhams) est également conforme aux données fournies dans le contexte.
5. **Évaluer la conformité de la réponse à la métrique** : La réponse est entièrement dérivée des informations présentées dans le contexte. Elle reprend fidèlement les chiffres et la répartition du résultat global consolidé tels qu'ils apparaissent dans le contexte.

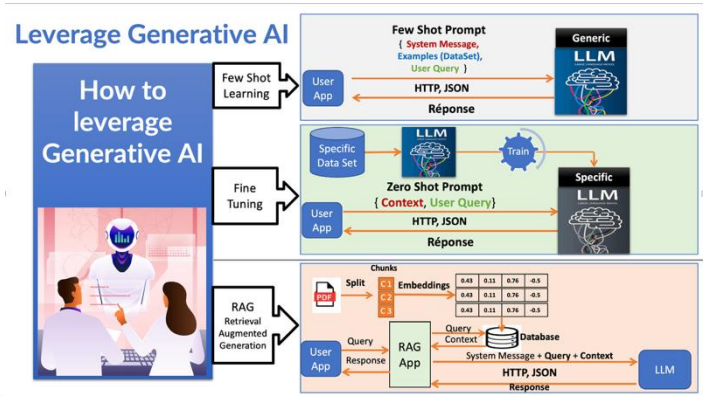
Évaluation : La réponse respecte entièrement la métrique, car elle est directement et uniquement dérivée des informations fournies dans le contexte.

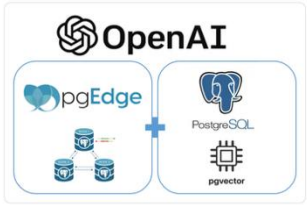
Score : 5

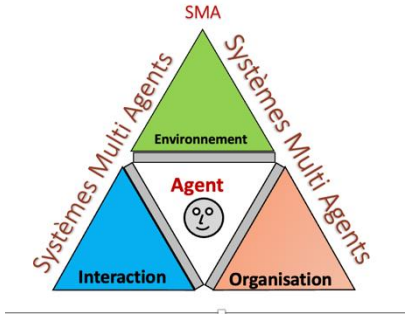
Agentic AI – LangChain



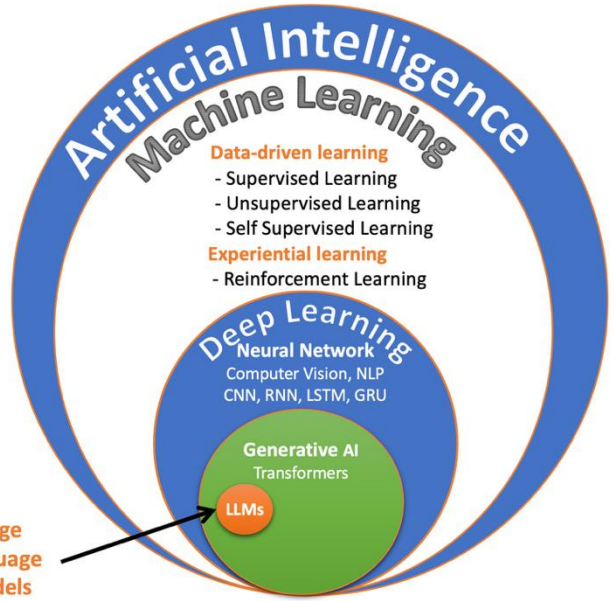












Agentic AI





Merci de faire la compagne de digital marketing

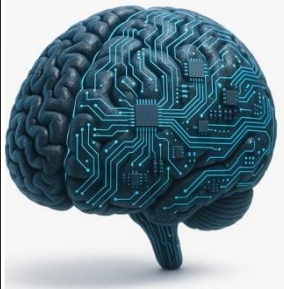
System message
Examples
Prompt
User Message
Goal

Memory

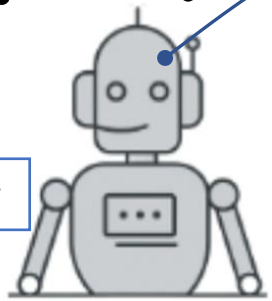


Reasoning P + Tools
Action
Reasoning P+ Obs +T
Action
Reasoning
Response
MIND

LLM

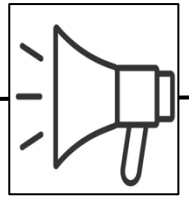


Agent = MIND + Body

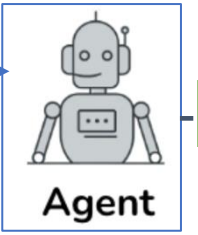


Agent

La compagne de digital marketing a été effectuée



A2A



Agent

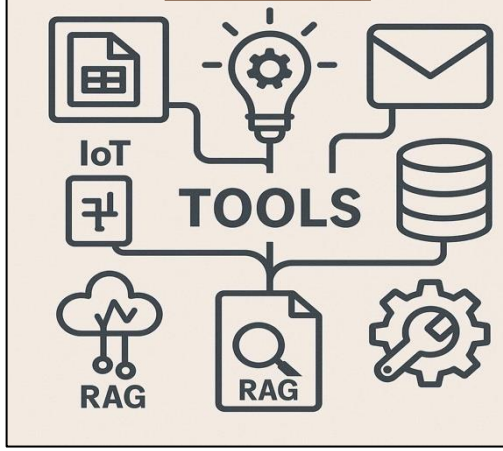
MCP

Tools
R4
R5
R6

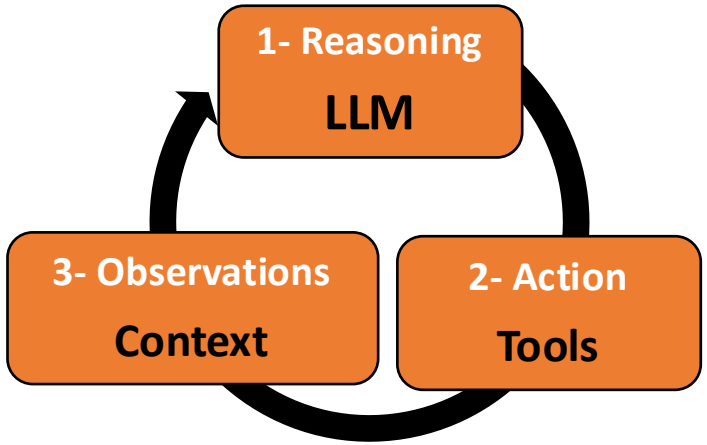
Action
observations

MCP

BODY



ReAct Pattern



Frameworks an Agentic AI Platforms

- No Code Low Code :
  - N8N, Agent Builder, etc.
- High Level Frameworks
  - Langchain
  - Crew AI
  - Agents SDK
  - Phi Data
  - Auto Gen
  - Smol Agent
- Low Level Frameworks :
  - LangGraph
  - Pour Java : Spring AI, Langchain4J

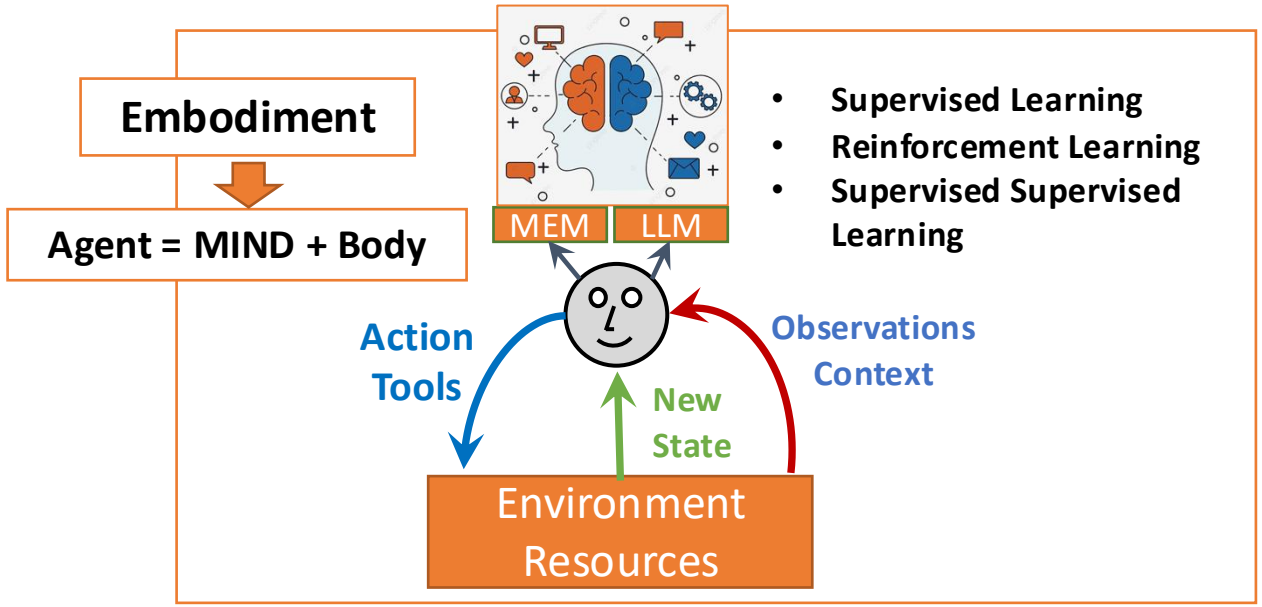
AI Agents

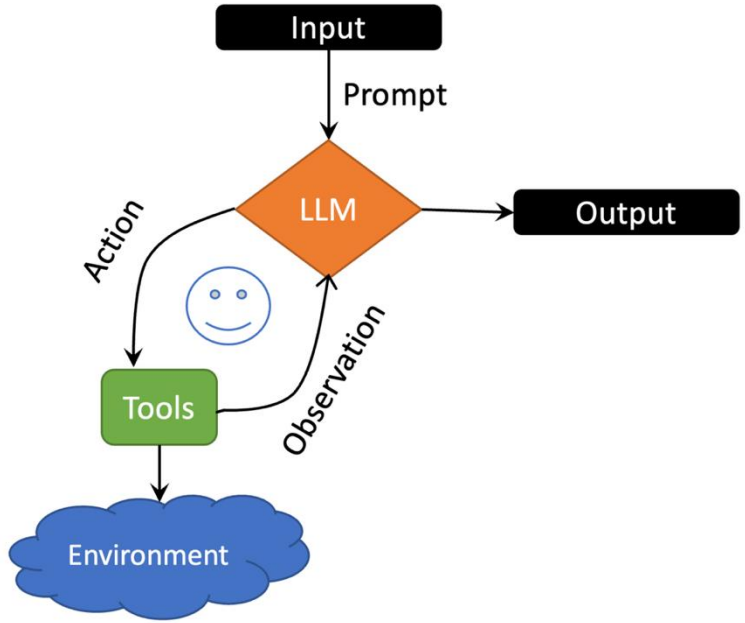
- Un agent est une entité autonome capable de prendre des décisions tout seul afin d'atteindre un but spécifique.
- Contrairement à un modèle d'intelligence artificielle classique, l'agent ne se contente pas de répondre à une question ; il interagit avec son environnement pour accomplir une mission.
- Architecture et composantes :

La structure fondamentale : **Mind + Body :** L'agent est défini par l'association de deux dimensions complémentaires :

- Le cerveau (Mind) : Il représente la dimension épistémologique, c'est-à-dire le raisonnement et la réflexion.
- Dans l'IA agentique moderne, ce rôle est assuré par les Large Language Models (LLM), qui permettent à l'agent de comprendre le langage humain et de planifier des actions.
- **Le corps (Body) :**

Il correspond à la dimension physicaliste. Il est constitué d'un ensemble d'outils (tools) et de fonctions qui permettent à l'agent d'agir sur son environnement. Ces outils peuvent être des accès à des bases de données, des capteurs IoT, ou des logiciels de productivité comme Google Sheets et Gmail.

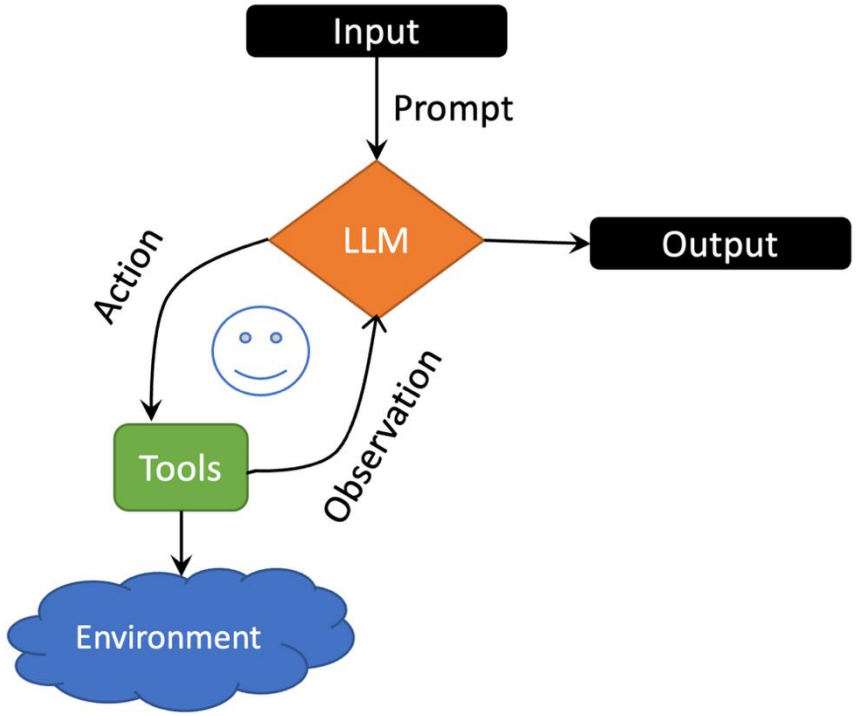


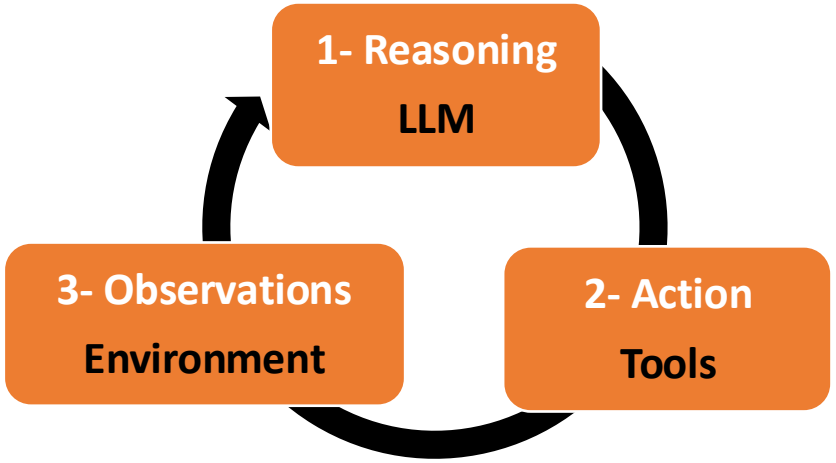


AI Agents

Fonctionnement :

- Un agent opère généralement selon un design pattern nommé React (Reasoning and Action). Ce cycle comprend plusieurs étapes :
  - **Raisonnement** : Le cerveau (LLM) analyse la situation et détermine l'action optimale à prendre.
  - **Action** : L'agent utilise un outil spécifique pour interagir avec l'extérieur.
  - **Observation** : L'agent collecte les résultats de son action (feedback ou contexte). Il renvoie ces observations à son cerveau pour raisonner à nouveau, jusqu'à ce qu'il dispose de tout le contexte nécessaire pour atteindre son but.
  - **La Mémoire** : Pallier le caractère "stateless ». Une caractéristique essentielle de l'agent est sa mémoire conversationnelle. Les LLM sont intrinsèquement "stateless", ce qui signifie qu'ils n'ont aucun souvenir des interactions précédentes. L'agent ajoute une couche de stockage qui conserve l'historique de la conversation, permettant ainsi de maintenir une cohérence au fil des échanges.
  - **Pilotage par le Prompt** : L'agent est dirigé par un prompt qui définit précisément sa mission. 5.
  - **Extension et Collaboration :** Corps infini (Protocole MCP) : Grâce au protocole Model Context Protocol, un agent peut accéder à des outils situés sur des machines distantes, ce qui élargit considérablement ses capacités d'action.
  - **Systèmes Multi-agents :** Pour des problèmes complexes, il est possible de créer plusieurs agents spécialisés qui communiquent entre eux en utilisant le protocole A2A : Agent To Agent. Ils profitent alors d'une intelligence collective pour collaborer vers un objectif commun

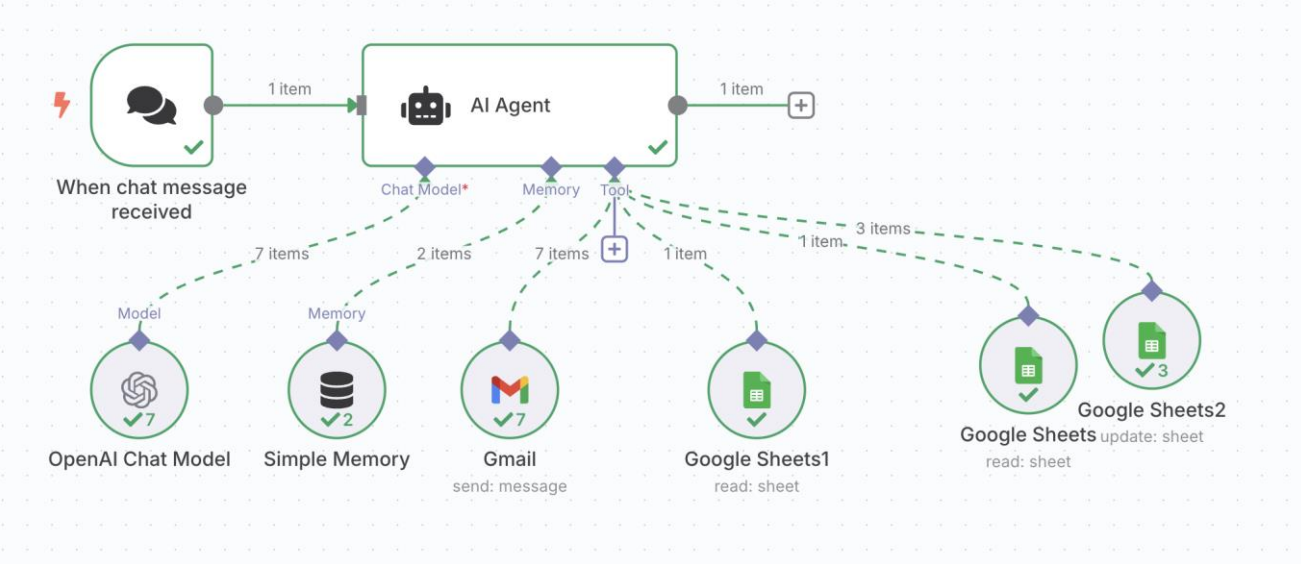




Demo 1 : First Agent in Automation platform : n8n

- **n8n** est une plateforme opensource d'**automatisation de workflows** qui permet de connecter différentes applications, API et services entre eux, sans avoir à développer toute la logique manuellement.
- n8n signifie souvent "**NodematioN**"
- Solution low-code / no-code, avec possibilité d'ajouter du code si besoin
- Disponible en cloud ou en auto-hébergèrent
- Avec n8n, on peut par exemple :
  - récupérer des données depuis un formulaire ou un CRM,
  - les transformer,
  - envoyer un email ou une notification,
  - mettre à jour une base de données,
  - déclencher une action dans Slack, Google Sheets, Notion, WhatsApp, etc.
- n8n peut servir à construire des **agents IA opérationnels**, surtout pour des cas d'usage métier. Mais **n8n n'est pas, à lui seul, un framework agentique avancé** au même niveau que des outils spécialisés comme **LangGraph, CrewAI ou AutoGen**.

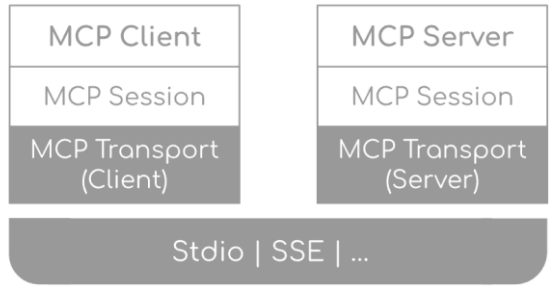
https://www.youtube.com/watch?v=E0OZPy5fP8E

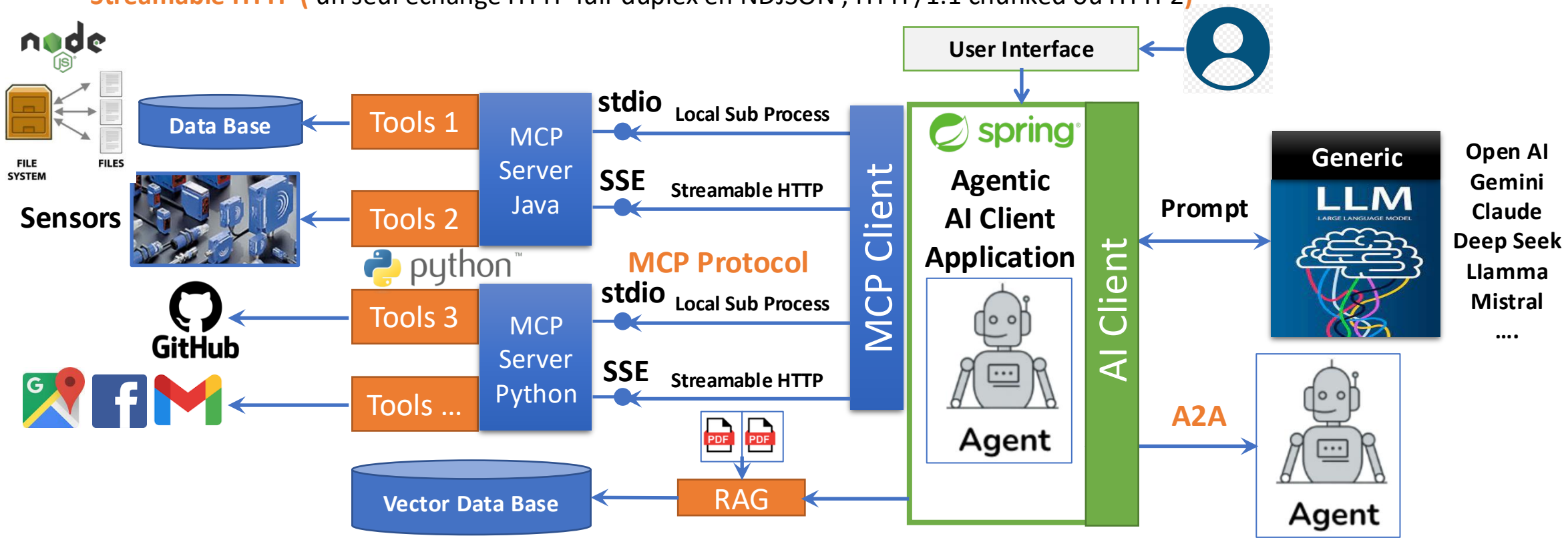


Model Context Protocol (MCP)

https://modelcontextprotocol.io

- MCP standardizes how to integrate **additional context and tools into the ecosystem of AI applications**
- The protocol uses **JSON-RPC 2.0 messages**
- The protocol defines two standard transport mechanisms for client-server communication:
  - **STDIO**, communication over **standard input** and **standard output**
  - **SSE (Server Sent Event :** serveur→client en streaming + POST séparés pour client→serveur
  - **Streamable HTTP (** un seul échange HTTP full-duplex en NDJSON , HTTP/1.1 chunked ou HTTP2**)**

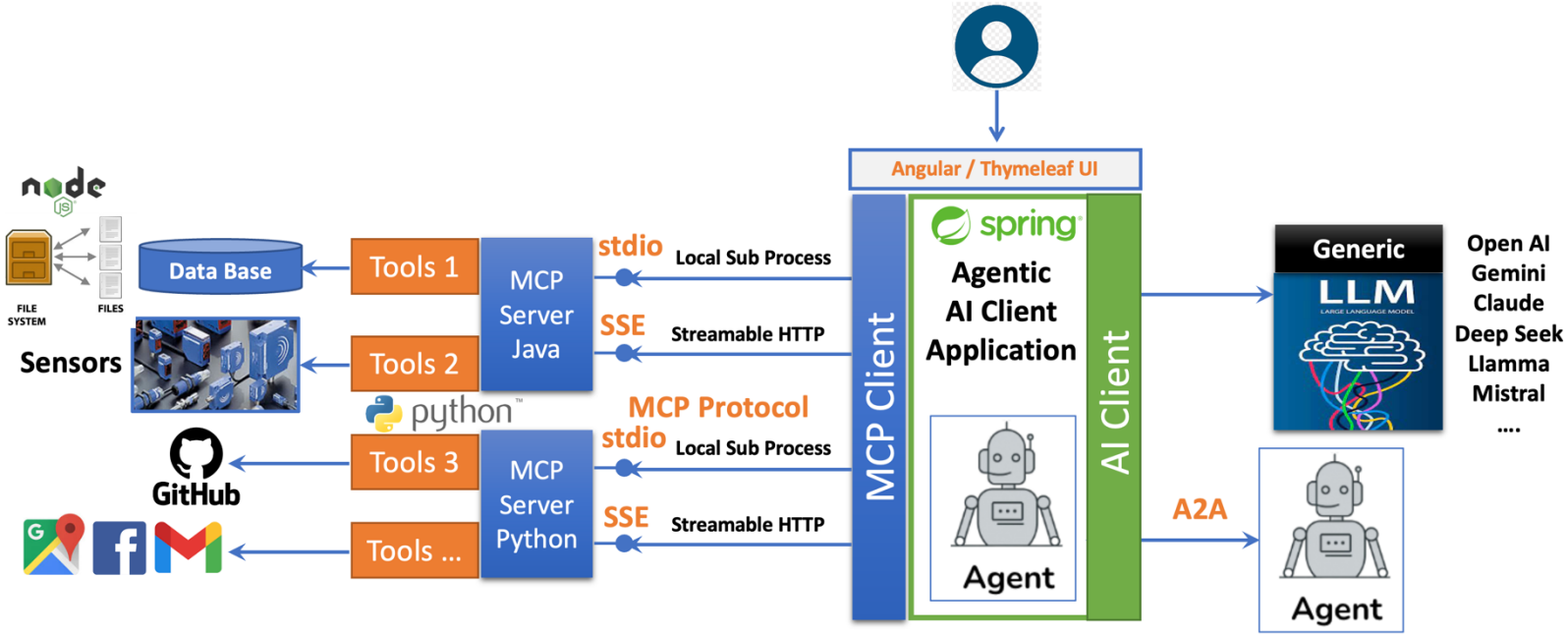




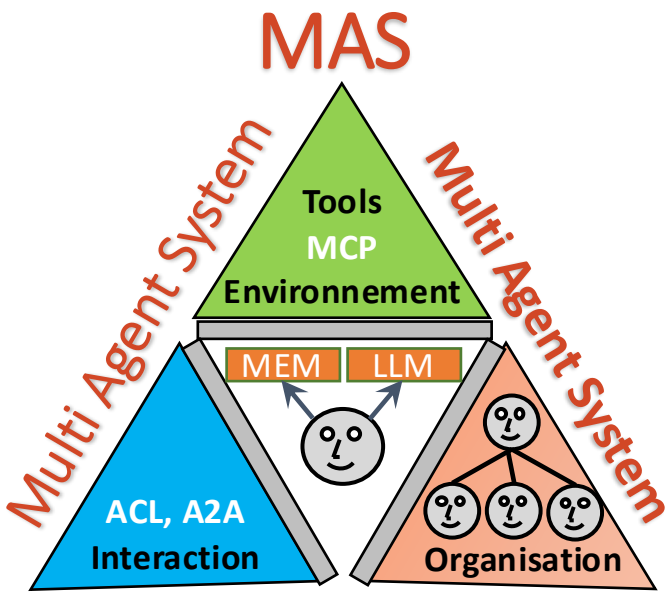
MCP

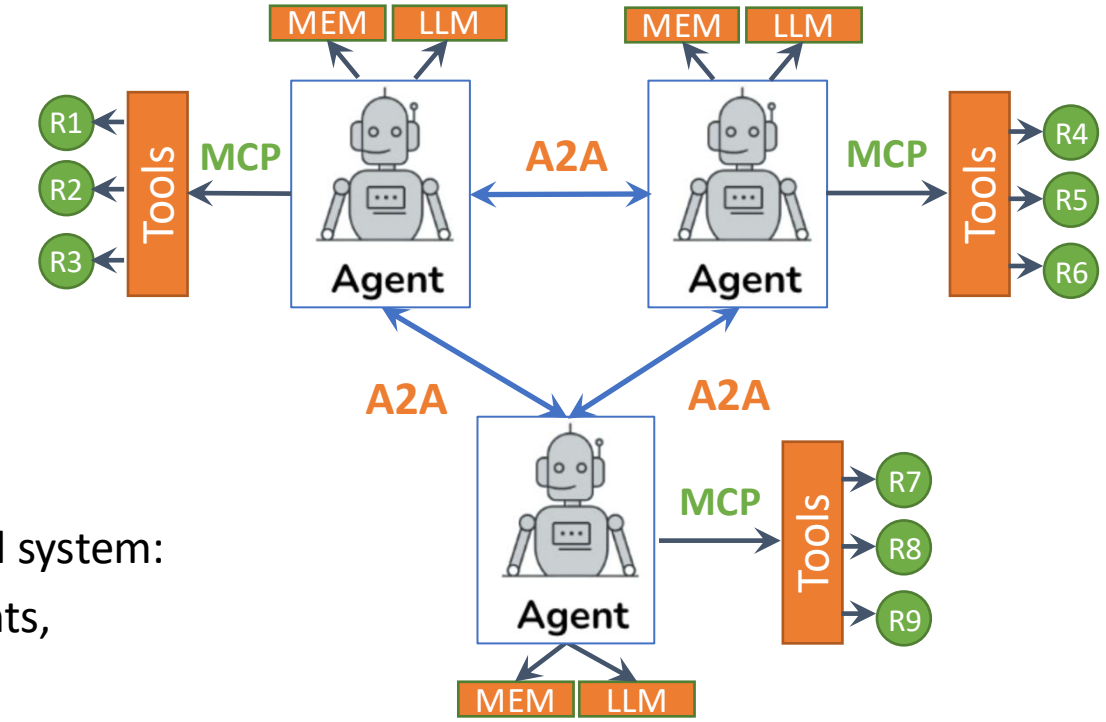
- Model Context Protocol (MCP) is an open protocol that enables seamless integration between LLM applications and external data sources and tools.
- MCP provides a standardized way for applications to:
  - Share contextual information with language models
  - Expose tools and capabilities to AI systems
  - Build composable integrations and workflows

- The protocol uses **JSON-RPC 2.0 messages** to establish communication between:
  - **Hosts**: LLM applications that initiate connections
  - **Clients**: Connectors within the host application
  - **Servers**: Services that provide context, tools and capabilities
- MCP standardizes how to integrate **additional context and tools into the ecosystem of AI applications**.



MAS And new Protocols : MCP et A2A





- A multi-agent system (MAS) is a distributed system:
  - Composed of a set of distributed agents,
  - Located in a specific environment
  - and Interacting according to specific organizational structures.
- An MAS solves complex problems by leveraging the collective intelligence of its component agents (DAI).

Premier Agent avec langchain

```
from langchain.agents import create_agent
from IPython.display import Markdown
from dotenv.ipython import load_dotenv
```

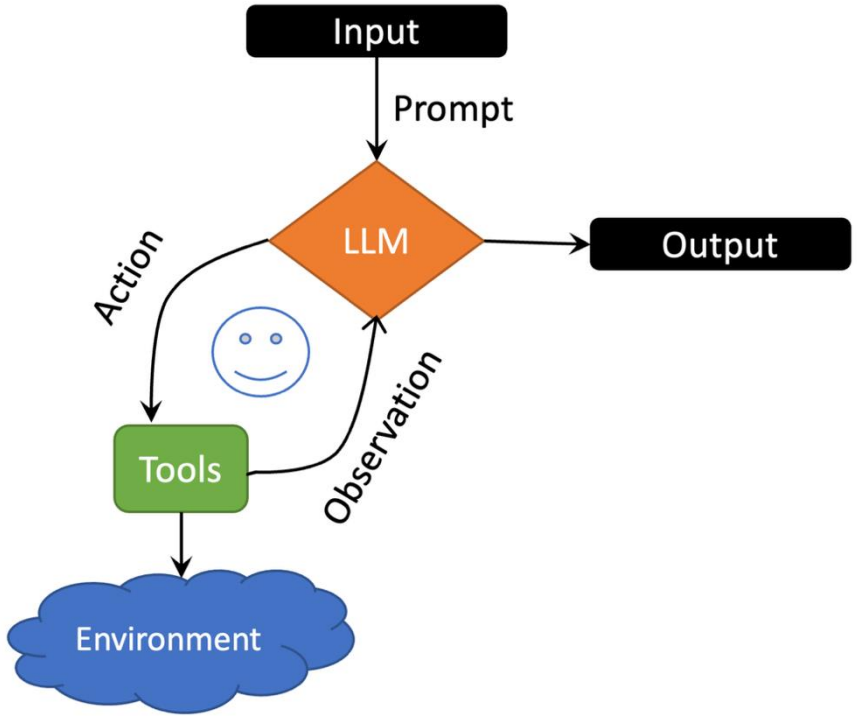
```
load_dotenv(override=True)
```

```
agent = create_agent(
   model="openai:gpt-4o",
   tools=[],
   system_prompt= "You are a helpful assistant"
   )
```

```
response = agent.invoke(
   input={
      'messages':[
           {'role':'user', 'content':'My name is Mohamed'}
         ]
        })
```

```
print(display(Markdown(response['messages'][-1].content)))
```

Hello, Mohamed! How can I assist you today?

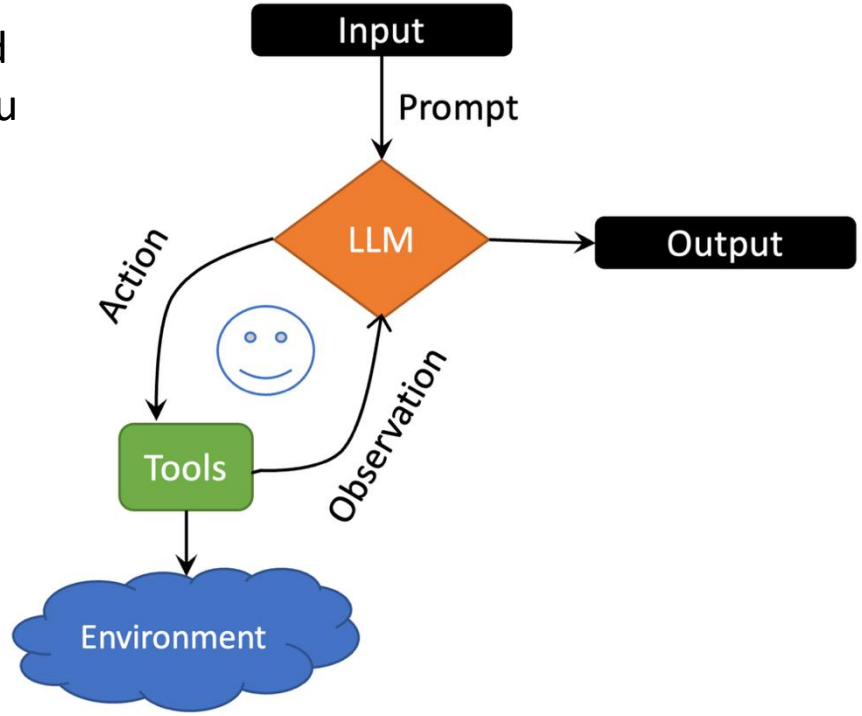


Premier Agent avec langchain

```
response = agent.invoke(input={'messages’:
  [{'role':'user', 'content':'What is my name'}]})
```

```
print(display(Markdown(response['messages'][-1].content)))
```

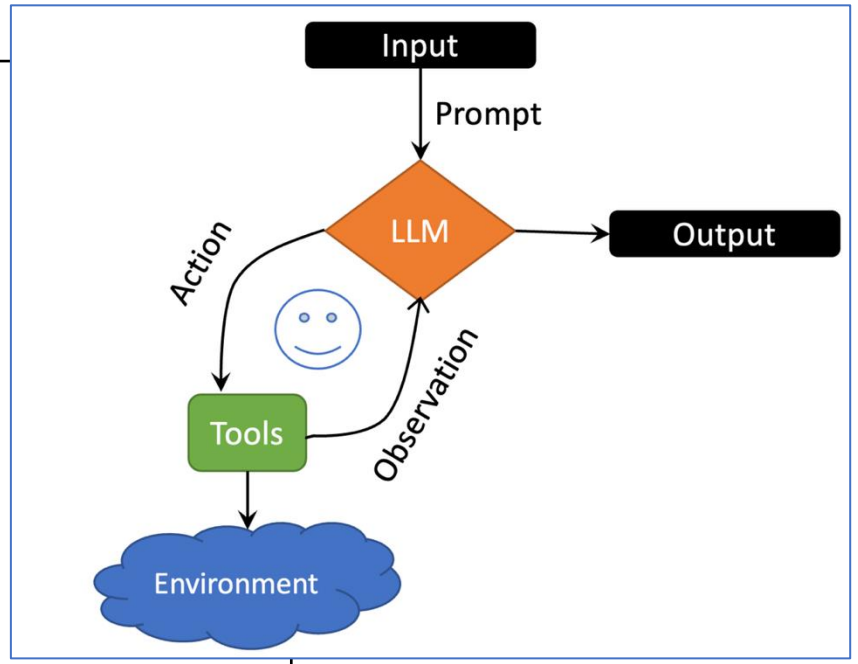
I'm sorry, **I don't have access to personal data such as your name**. If you'd like, you can tell me, and I'll do my best to help you with any questions you have!



Premier Agent avec langchain

Pour mieux contrôler la configuration du modèle, initialisez directement une instance du modèle à l'aide du package du fournisseur. Dans cet exemple, nous utilisons **ChatOpenAI**. Consultez la section **Modèles de chat** pour découvrir les autres classes de modèles de chat disponibles.

```
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI
from langchain.messages import HumanMessage
llm = ChatOpenAI(
    model="gpt-5.2",
    temperature=0.1,
    max_tokens=1000,
    timeout=30
)
agent = create_agent(
    model=llm,
    tools=[],
    system_prompt= "You are a helpful assistant",
    debug=True
    )
response = agent.invoke(input={
    "messages" : [HumanMessage("La ville la plus grande au Maroc")]
})
```




```
[values] {'messages': [HumanMessage(content='La ville la plus grande au Maroc',
[updates] {'model': {'messages': [AIMessage(content='La plus grande ville du Ma
[values] {'messages': [HumanMessage(content='La ville la plus grande au Maroc',
```

```
print(display(Markdown(response['messages'][-1].content)))
```

✓ 0.0s

La plus grande ville du Maroc (en population) est **Casablanca**.

AI Agent avec langchain : Dynamic Model

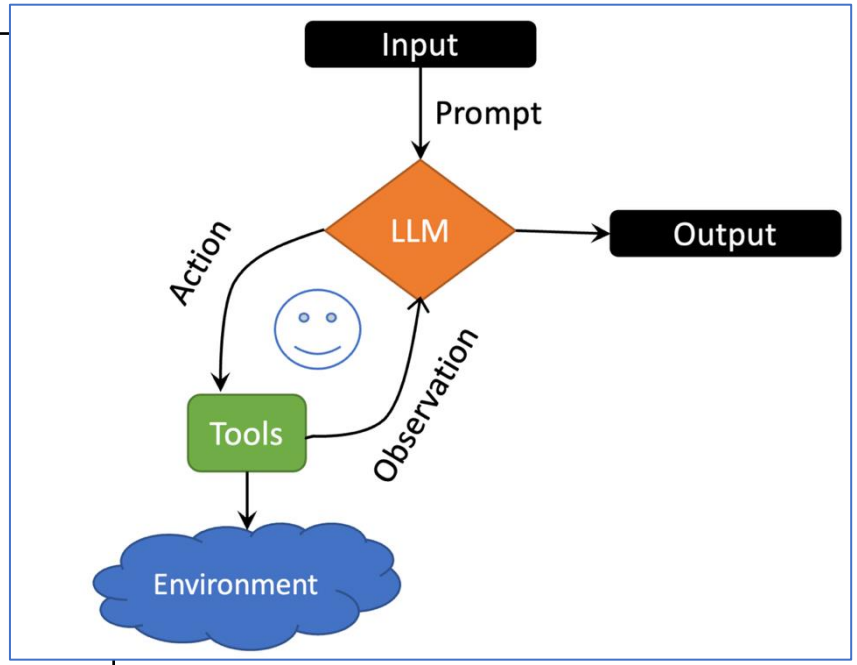
Les modèles dynamiques sont sélectionnés à l'exécution en fonction de l'état et du contexte actuels. Cela permet une logique de routage avancée et une optimisation des coûts.
Pour utiliser un modèle dynamique, créez un middleware à l'aide du décorateur `@wrap_model_call` qui modifie le modèle dans la requête :

```python
from langchain_openai import ChatOpenAI
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
basic_model = ChatOpenAI(model="gpt-4o-mini")
advanced_model = ChatOpenAI(model="gpt-4o")

@wrap_model_call
def dynamic_model_selection(request: ModelRequest, handler) -> ModelResponse:
    """Choose model based on conversation complexity."""
    message_count = len(request.state["messages"])
    env=request.runtime.context.get('env','test')
    print(env)
    if env=='prod':
        # Use an advanced model for longer conversations
        model = advanced_model
        print("advanced model selected ....")
    else:
        model = basic_model
        print("basic model selected ....")

    return handler(request.override(model=model))

agent = create_agent(
    model=basic_model,  # Default model
    tools=[],
    middleware=[dynamic_model_selection],
    debug=True
)
```




```python
agent.invoke(input={'messages':[
  HumanMessage('Test 1')
 ]}, context={'env':'test'})
```

```python
agent.invoke(input={'messages':[
  HumanMessage('Test 1')
]}, context={'env':'prod'})
```

AI Agent avec langchain : Memory

```
from langchain.agents import AgentState
from langchain.agents.middleware import AgentMiddleware
from typing import Any
from langgraph.checkpoint.memory import InMemorySaver
```

```
agent = create_agent(
  model="openai:gpt-5.2",
  system_prompt="You are a helpful assistant",
  checkpointer=InMemorySaver()
)
```

```
res=agent.invoke(
  input={'messages':[HumanMessage('My Name is Mohamed')]},
  config={'configurable':{'thread_id':1}}
)
print(res['messages'][-1].content)
```

```
Hi Mohamed! How can I help you today?
```

```
res=agent.invoke(
  input={'messages':[HumanMessage('What is my name')]},
  config={'configurable':{'thread_id':1}}
)
print(res['messages'][-1].content)
```

```
Your name is Mohamed.
```

AI Agent avec langchain : Memory

```
from langchain.agents import create_agent
from langgraph.checkpoint.postgres import PostgresSaver


DB_URI = "postgresql://postgres:pwd@localhost:5432/postgres?sslmode=disable"
with PostgresSaver.from_conn_string(DB_URI) as checkpointer:
    checkpointer.setup() # auto create tables in PostgresSql
    agent = create_agent(
        "gpt-5",
        tools=[],
        checkpointer=checkpointer,
    )
```

AI Agent avec langchain : Tools

Les outils donnent aux agents la capacité d'agir.
Les agents vont au-delà du simple rattachement d'outils au modèle en facilitant :

- Plusieurs appels d'outils en séquence (déclenchés par un seul prompt)
- Des appels d'outils en parallèle lorsque cela est pertinent
- La sélection dynamique des outils en fonction des résultats précédents
- La logique de nouvelle tentative (retry) et la gestion des erreurs
- La persistance de l'état entre les appels d'outils

```python
from langchain.tools import tool
from langchain.agents import create_agent
@tool
def search(query: str) -> str:
    """Search for news."""
    print(f'Search tool is called for {query}')
    return {
        'query':query,
        'news': [
            "Le temps est très glacial",
            "les condition météo sont très délicates"
        ]
    }

@tool
def get_weather(location: str) -> str:
    """Get weather information for a location."""
    print(f'Weather tool is called for {location}')
    return f"Weather in {location}: Sunny, 32°C"

@tool
def get_employee_info(name: str) -> str:
    """Get information aboud the given employee name"""
    print(f'get_employee_info tool is called for {name}')
    return {'name' : name, 'salary': 12000, 'job': 'Developper'}

agent = create_agent(
  model=llm,
  tools=[search, get_weather, get_employee_info], debug=True)
```

```
response=agent.invoke(input={'messages':[HumanMessage('What is the weather in Marrakech')]})
```

```
resp=agent.invoke(input={'messages':[HumanMessage('What aye news')]})
```

```
resp=agent.invoke(input={'messages':[HumanMessage('Quel est le salaire et le job de Mohamed')]})
```

AI Agent avec langchain : Tools (DuckDuckGo)

```python
from ddgs import DDGS
@tool
def web_search(query: str, num_results:int=5) -> str:
    """
    Search the web usin DuckDuckGo
    Args:
        query : Search query string
        num_results : Number of results to return (Default=5)
    Returns:
      Formatted search results with titles, descptions and Urls
    """

    try:
        print(f'Search tool is called for {query}')
        ddgs_search = DDGS()
        results=ddgs_search.text(query=query, max_results=num_results, backend="google")
        if not results:
            return f"No results found for {query} "
        formatted_results = [f"Search for {query} : \n"]
        for i, result in enumerate(results,1):
            title = result.get("title","No Title")
            body = result.get("body","No description available")
            href = result.get("href","")
            formatted_results.append( f"{i}. **{title}**. {body}. {href}")
        return formatted_results
    except Exception as e:
        print(str(e))
```

Here are some solid Python tutorials to get started:

1. **W3Schools – Python Tutorial** (beginner-friendly, quick reference)
   https://www.w3schools.com/python/
2. **Official Python Docs – The Python Tutorial** (authoritative and thorough)
   https://docs.python.org/3/tutorial/index.html
3. **LearnPython.org** (free, interactive exercises in the browser)
   https://www.learnpython.org/
4. **Real Python** (high-quality articles and tutorials, beginner → advanced)
   https://realpython.com/
5. **Codecademy – Python Courses** (interactive course format)
   https://www.codecademy.com/catalog/language/python

If you tell me your level (total beginner vs. some coding experience) and your goa
path.

```python
agent = create_agent(model=llm, tools=[web_search, get_employee_info, get_weather], debug=True)
```

```python
resp=agent.invoke(input={'messages':[HumanMessage('Search for python tutorials')]})
```

```python
print(display(Markdown(resp['messages'][-1].content)))
```

AI Agent avec langchain : Tools (Tavily)

```
from langchain.chat_models import init_chat_model
from langchain_tavily import TavilySearch
import asyncio
from datetime import datetime

tavily_search_tool = TavilySearch(max_results=10, search_depth="advanced")

@tool
def search(query: str) -> str:
    """
    Search for general web results
    """
    print("Search Tool invoked")
    return tavily_search_tool.invoke({"query": query})
```

```
model = init_chat_model(model="gpt-4o", model_provider="openai")
agent = create_agent(
    model=model,
    tools=[search],
    system_prompt=f"""You are a helpful assistant that serach the web
                for information using search tool
                today's date is {datetime.now().strftime("%Y-%m-%d")}
                """,
)
```

```
resp=agent.invoke(input={'messages':[HumanMessage("Quel temps fait-il aujourd'hui à Casablanca")]})
print(display(Markdown(resp['messages'][-1].content)))
```

```
Search Tool invoked
```

Aujourd'hui, le 13 mars 2026, à Casablanca, le temps est principalement clair avec une température d'environ 15°C ressentie, et un vent soufflant à 8 km/h. Le taux d'humidité est de 89%. La journée s'annonce plutôt ensoleillée avec des températures atteignant un maximum de 19°C et un minimum d'environ 12°C.

AI Agent avec langchain : Tools

```
from langchain_experimental.utilities import PythonREPL
python_repl = PythonREPL()
python_repl.run('print(f"la somme de 5 et 6 est {5+9}")')
```

```
Python REPL can execute arbitrary code. Use with caution.

'la somme de 5 et 6 est 14\n'
```

```
from langchain_core.tools import Tool
from langchain.tools import tool, ToolRuntime

repl_tool = Tool(
  name="repl_tool",
  description="A Python shell used to execute python commands.
Input should be a valid python command.",
  func= python_repl.run
)
```

```
repl_tool.invoke("""
  a= 5
  b=9
  print(f"la somme de {a} et {b} est {a+b}")
""")
```

```
'la somme de 5 et 9 est 14\n'
```

```
agent = create_agent(
 model=llm, tools=[repl_tool],
 debug=True,
 system_prompt='generate python code and use the repl tool to execute'
)
```

```
resp= agent.invoke(input= {'messages':[HumanMessage(
"""
a= 5
b=9
print(f"la somme de {a} et {b} est {a+b}")
"""
)]})
print(display(Markdown(resp['messages'][-1].content)))
```

```
a = 5
b = 9
print(f"la somme de {a} et {b} est {a+b}")
```

Sortie :

```
la somme de 5 et 9 est 14
```

AI Agent avec langchain : Tools

```
from langchain_experimental.tools import PythonREPLTool
from langchain.messages import SystemMessage, HumanMessage
```

```
agent4 = create_agent(
 model="openai:gpt-4o",
 tools=[PythonREPLTool(sanitize_input=False)],
 system_prompt=SystemMessage("""user the REPL Tool to execute
the generated code and write the generated python code and the
result in a file doc.txt"""),
 debug=True
)
```

```
resp = agent4.invoke(input={
'messages':[
 HumanMessage("""Je veux trier deux listes [6,5,8,3,2] et
[65,15,8,13,2]
et puis faire la somme des deux listes triées""")
 ]
})
```

```
print(resp['messages'][-1].content)
```

J'ai exécuté le code Python pour trier les deux listes, calculer les sommes correspondantes et j'ai enregistré le code et les résultats dans un fichier nommé `doc.txt`. Voici les résultats obtenus :

- Liste 1 triée : `[2, 3, 5, 6, 8]`
- Somme de la liste 1 : `24`
- Liste 2 triée : `[2, 8, 13, 15, 65]`
- Somme de la liste 2 : `103`

Vous pouvez vérifier le fichier `doc.txt` pour voir le code Python utilisé et ces résultats détaillés.

AI Agent avec langchain : Tools (Error Handling)

```
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_tool_call
from langchain.messages import ToolMessage
@wrap_tool_call
def handle_tool_errors(request, handler):
    """Handle tool execution errors with custom messages."""
    try:
        return handler(request)
    except Exception as e:
        print('ERRRRRRRRRR')
        # Return a custom error message to the model
        return ToolMessage(
            content=f"Tool error: Please check your input and try again. ({str(e)})",
            tool_call_id=request.tool_call["id"]
        )
agent = create_agent(
    model="openai:gpt-4o",
    tools=[search, get_weather],
    middleware=[handle_tool_errors], debug=True
)
```

AI Agent avec langchain : (System Prompt)

```
agent = create_agent(
  model="openai:gpt-4o",
  tools=[],
  system_prompt="You are a helpful assistant. Be concise and accurate."
)
```

```
from typing import TypedDict
from langchain.agents import create_agent
from langchain.agents.middleware import dynamic_prompt, ModelRequest

class Context(TypedDict):
    user_role: str

@dynamic_prompt
def user_role_prompt(request: ModelRequest) -> str:
    """Generate system prompt based on user role."""
    user_role = request.runtime.context.get("user_role", "user")
    base_prompt = "You are a helpful assistant."
    if user_role == "expert":
        return f"{base_prompt} Provide detailed technical responses."
    elif user_role == "beginner":
        return f"{base_prompt} Explain concepts simply and avoid jargon."

    return base_prompt
```

```
agent = create_agent(
    model="openai:gpt-5.2",
    tools=[],
    middleware=[user_role_prompt],
    context_schema=Context
)
```

```
result = agent.invoke(
{"messages": [{"role": "user", "content": "Explain machine learning"}]},
context={"user_role": "expert"})
print(display(Markdown(result['messages'][-1].content)))
```

```
result = agent.invoke(
{"messages": [{"role": "user", "content": "Explain machine learning"}]},
context={"user_role": "beginner"})
print(display(Markdown(result['messages'][-1].content)))
```

AI Agent avec langchain : (Structuted Output)

```python
from pydantic import BaseModel
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy

class ContactInfo(BaseModel):
    name: str
    email: str
    phone: str

agent = create_agent(
    model="openai:gpt-4o-mini",
    tools=[],
    response_format=ToolStrategy(ContactInfo)
    # response_format=ProviderStrategy(ContactInfo)
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "Extract contact info from: Mohamed YOUSSFI, med@gmail.com, (212) 123-4567"}]
})

contact= result["structured_response"]
# ContactInfo(name='Mohamed YOUSSFI', email='med@gmail.com', phone='(212) 123-4567')
```

```python
print(contact.name)
print(contact.email)
print(contact.phone)
```

```
Mohamed YOUSSFI
med@gmail.com
(212) 123-4567
```

Custom Middleware

```python
from langchain.agents.middleware import AgentMiddleware, AgentState
import time

class HooksDemo(AgentMiddleware):
    def __init__(self):
        super().__init__()
        self.starttime = 0.0
    def before_agent(self, state : AgentState, runtime):
        self.starttime = time.time()
        print('Befor_agent triggered')
    def before_model(self, state : AgentState, runtime):
        print('Beforre_model triggered')
    def after_model(self, state : AgentState, runtime):
        print('After Model triggered')
    def after_agent(self, state : AgentState, runtime):
        print('After Agent triggered')
        print("duration = ", time.time()-self.starttime)
```

```
Befor_agent triggered
Beforre_model triggered
After Model triggered
After Agent triggered
duration =  3.381105899810791
```

```python
agent = create_agent(model="openai:gpt-5.2", middleware=[HooksDemo()])
res=agent.invoke({'messages':[HumanMessage('Tel me how many cities in morocco')]})
```

```python
print(res['messages'][-1].content)
```

```
It depends on what you mean by “cities” in Morocco.

- **Official municipalities (urban communes)**: **about 221*
- If you mean **major/well-known cities**, people usually re

If you tell me whether you mean **official urban communes**,
```

RAG as tool

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS

embedding_model = OpenAIEmbeddings(model="text-embedding-3-large")
```

```
texts = [
 "Je m'appelle Mohamed Youssfi, Je suis Professeur en Informatique et Intelligence artificielle",
 "Je travaille à l'ENSET Mohammedia, Université Hassan II de Casablanca",
 "J'ai obtenu mon doctorat d'état en 2015, Mon doctorat de troisième cycle en 19996 et mon diplôme de professeur second cycle en 1993",
 "En plus de l'informatique, je suis patiené par la musique et la culture",
 "J'aime aussi écrire des récits sur ma vie et j'aime la philosophie",
 "Je suis originaire de Ouarzazate, une ville au sud du Maroc",
 "après les études de primaire et le collège à Ouarzazate, j'ai suivi mes études de lycée technique à Marrakech",
 "Après le baccalauréat, j'ai rejoint l'ENSET Mohammedia pendant 4 années d'études pour devenir Professeur de second cycle",
 "J'ai travaillé à l'ENSET depuis 1993 pour y enseigner principalement l'informatique et les sciences de l'ingénieur",
 "En parallèle à mon travailler de professeur, j'ai suivi mes études supérieures à la Facultés des sciences de rabat",
 "où j'ai obtenu mon DEA, Doctorat de troisième cycle puis Doctorat d'Etat dans le domaine des systèmes informatiques parallèles et distribués"
]
```

```
vectore_store = FAISS.from_texts(texts=texts, embedding=embedding_model)
results = vectore_store.similarity_search("Nom, prénom et affiliation",2)
```

```
print(results[0].page_content)
print(results[1].page_content)
```

```
Je m'appelle Mohamed Youssfi, Je suis Professeur en Informatique et Intelligece artificielle
Je travaille à l'ENSET Mohammedia, Université Hassan II de casablanca
```

RAG as tool

```
from langchain_core.tools import create_retriever_tool

retrieval = vectore_store.as_retriever(kwargs={'k':5})
retrieval_tool = create_retriever_tool(
  retriever=retrieval,
  name="kb_search",
  description="Search informtation about me »
)
```

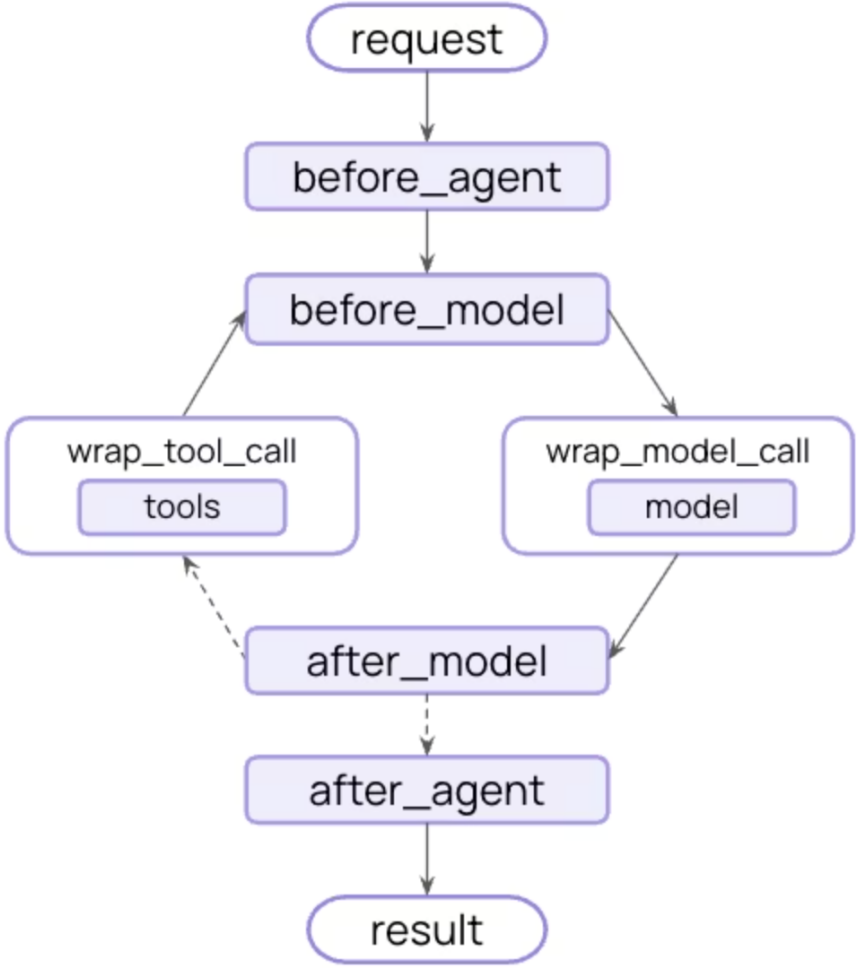
```
search_agent = create_agent(
 model="openai:gpt-5.2",
 system_prompt="Search infomration about me",
 tools=[retrieval_tool]
)
```

```
result = search_agent.invoke({'messages':[HumanMessage('Nom et Prénom, Affiliation et diplômes')]})
print(display(Markdown(result['messages'][-1].content)))
```

- **Nom et Prénom :** Mohamed Youssfi
- **Affiliation :** ENSET Mohammedia, Université Hassan II de Casablanca
- **Diplômes / Formation :**
  - Études à l'**ENSET Mohammedia** (4 ans) après le baccalauréat pour devenir **Professeur de second cycle**
  - **DEA**, **Doctorat de 3ème cycle**, puis **Doctorat d'État** dans le domaine des **systèmes informatiques parallèles et distribués**

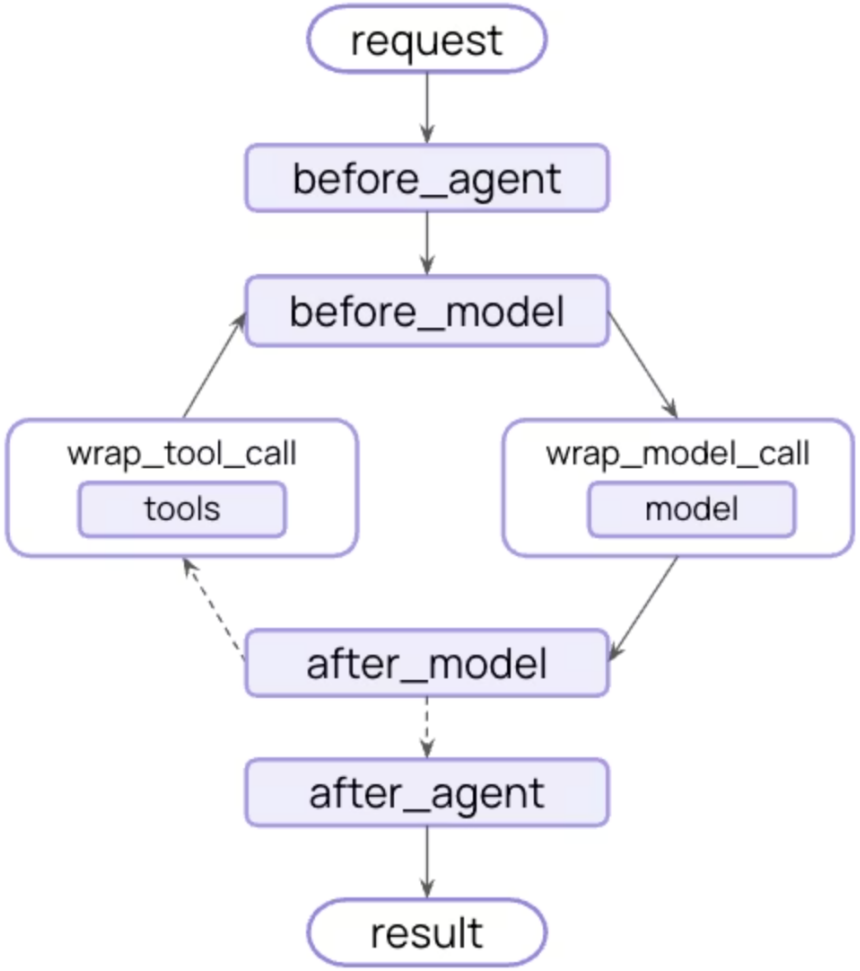
Guardrails

- Les garde-fous vous aident à concevoir des applications d'IA sûres et conformes en validant et en filtrant le contenu à des étapes clés de l'exécution de votre agent. Ils peuvent détecter des informations sensibles, appliquer des politiques de contenu, valider les sorties et prévenir les comportements dangereux avant qu'ils ne causent des problèmes.
- Les cas d'usage courants incluent :
  - - Empêcher la fuite de données personnelles identifiables (PII)
  - - Détecter et bloquer les attaques par injection de prompt
  - - Bloquer les contenus inappropriés ou nuisibles
  - - Appliquer les règles métier et les exigences de conformité
  - - Valider la qualité et l'exactitude des résultats
- Vous pouvez mettre en œuvre des garde-fous à l'aide d'un middleware pour intercepter l'exécution à des points stratégiques — avant le démarrage de l'agent, après son exécution, ou autour des appels au modèle et aux outils.



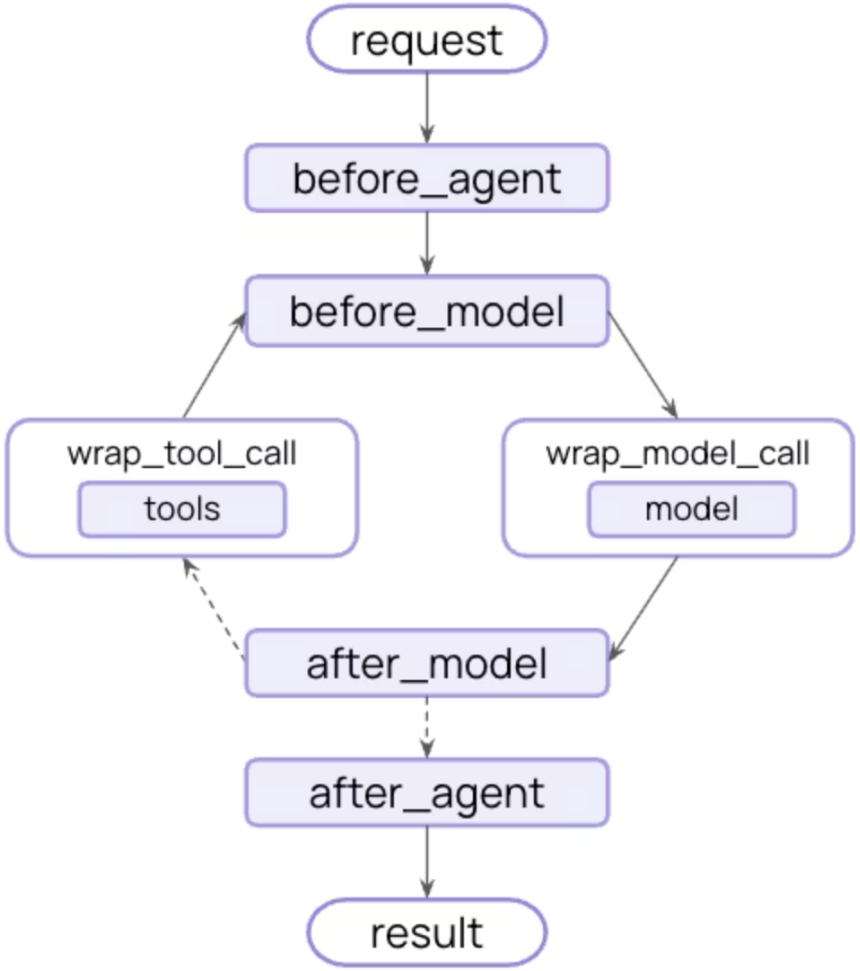
Guardrails

- Les garde-fous peuvent être mis en œuvre selon deux approches complémentaires :
  - **Garde-fous déterministes** : Ils utilisent une logique fondée sur des règles, comme des expressions régulières (regex), la correspondance de mots-clés ou des vérifications explicites. Ils sont rapides, prévisibles et économiques, mais peuvent manquer certaines violations plus subtiles.
  - **Garde-fous fondés sur des modèles**: Ils utilisent des LLM ou des classificateurs pour évaluer le contenu avec une compréhension sémantique. Ils permettent de détecter des problèmes subtils que les règles seules ne repèrent pas, mais sont plus lents et plus coûteux.
- LangChain fournit à la fois des garde-fous intégrés (par exemple, la détection de **PII** et l'intervention humaine dans la boucle : **Human-In-Loop**) ainsi qu'un système de middleware flexible pour créer des garde-fous personnalisés selon l'une ou l'autre approche.



Guardrails

- Les garde-fous peuvent être mis en œuvre selon deux approches complémentaires :
  - **Garde-fous déterministes** : Ils utilisent une logique fondée sur des règles, comme des expressions régulières (regex), la correspondance de mots-clés ou des vérifications explicites. Ils sont rapides, prévisibles et économiques, mais peuvent manquer certaines violations plus subtiles.
  - **Garde-fous fondés sur des modèles**: Ils utilisent des LLM ou des classificateurs pour évaluer le contenu avec une compréhension sémantique. Ils permettent de détecter des problèmes subtils que les règles seules ne repèrent pas, mais sont plus lents et plus coûteux.
- LangChain fournit à la fois des garde-fous intégrés (par exemple, la détection de **PII** et l'intervention humaine dans la boucle : **Human-In-Loop**) ainsi qu'un système de middleware flexible pour créer des garde-fous personnalisés selon l'une ou l'autre approche.



Guardrails : PII

- LangChain fournit un middleware intégré pour détecter et traiter les informations personnelles identifiables **(PII)** dans les conversations.
- Ce middleware peut détecter les types de PII courants, comme les adresses e-mail, les cartes bancaires, les adresses IP, et bien d'autres.
- Le middleware de détection des PII est utile dans des cas tels que les applications de santé et de finance soumises à des exigences de conformité, les agents de service client qui doivent assainir les journaux, ainsi que, plus généralement, toute application manipulant des données utilisateur sensibles.
- Le middleware PII prend en charge plusieurs stratégies pour traiter les PII détectées :

Stratégie	Description	Exemple
`redact`	Remplace par `[REDACTED_{PII_TYPE}]`	`[REDACTED_EMAIL]`
`mask`	Masque partiellement l'information (par ex. les 4 derniers chiffres)	`****-****-****-1234`
`hash`	Remplace par un hachage déterministe	`a8f5f167...`
`block`	Déclenche une exception lorsqu'une PII est détectée	Erreur levée

Types de PII intégrés

- `email` — Adresses e-mail
- `credit_card` — Numéros de carte bancaire
- `ip` — Adresses IP
- `mac_address` — Adresses MAC
- `url` — URL

Options de configuration

Paramètre	Description	Valeur par défaut
`pii_type`	Type de PII à détecter (intégré ou personnalisé)	Obligatoire
`strategy`	Façon de traiter les PII détectées (`"block"`, `"redact"`, `"mask"`, `"hash"`)	`"redact"`
`detector`	Fonction de détection personnalisée ou motif regex	`None` (utilise le détecteur intégré)
`apply_to_input`	Vérifie les messages utilisateur avant l'appel au modèle	`True`
`apply_to_output`	Vérifie les messages de l'IA après l'appel au modèle	`False`
`apply_to_tool_results`	Vérifie les messages de résultat des outils après leur exécution	`False`

Guardrails : PII

```python
from langchain.agents import create_agent
from langchain.agents.middleware import PIIMiddleware

@tool
def card_tool(credit_card:str):
    """
    Process The payment using the credit card
    """
    print(f"Process The payment using the credit card : {credit_card}")
    return f"The payment has been processed succefully with {credit_card}"

@tool
def send_email(email_adress:str):
    """
    Sending The Email using the email_address
    """
    print(f"Sending The Email using the email_address : {email_adress}")
    return f"Email sent succefully to {email_adress}"

@tool
def process_api_key(api_key:str):
    """
    Process with the api key
    """
    print(f"Process with the api key : {api_key}")
    return f"Processing with the api key : {api_key}"
```

```python
agent = create_agent(
    model="gpt-4.1",
    tools=[card_tool, send_email],
    middleware=[
        # Redact emails in user input before sending to model
        PIIMiddleware(
            "email",
            strategy="redact",
            apply_to_input=True,
        ),
        # Mask credit cards in user input
        PIIMiddleware(
            "credit_card",
            strategy="mask",
            apply_to_input=True,
        ),
        # Block API keys - raise error if detected
        PIIMiddleware(
            "api_key",
            detector=r"sk-[a-zA-Z0-9]{32}",
            strategy="block",
            apply_to_input=True,
        ),
    ],
)
```

```python
# When user provides PII, it will be handled according to the strategy
result = agent.invoke({
"messages": [{"role": "user", "content": "My email is med@gmail.com and card is 5105-1051-0510-5100."}]
})
print(result['messages'][-1].content)
```

```
Sending The Email using the email_address : [REDACTED_EMAIL]
Process The payment using the credit card : ****-****-****-5100
```

Your email has been sent successfully to [REDACTED_EMAIL], and the payment has been processed successfully using your card ending in 5100. If you need any further assistance, please let me know!

Guardrails : Human-in-the-loop

- **Intervention humaine dans la boucle (Human-in-the-loop)**
- LangChain fournit un middleware intégré permettant d'exiger une approbation humaine avant l'exécution d'opérations sensibles. Il s'agit de l'un des garde-fous les plus efficaces pour les décisions à forts enjeux.
- Le middleware d'intervention humaine dans la boucle est utile dans des cas tels que les transactions et transferts financiers, la suppression ou la modification de données de production, l'envoi de communications à des parties externes, ainsi que toute opération ayant un impact significatif sur l'activité.

```python
from langchain.agents import create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command

@tool
def send_email(email_adress:str):
    """
    Sending The Email using the email_address
    """
    print(f"Sending The Email using the email_address : {email_adress}")
    return f"Email sent succefully to {email_adress}"

agent = create_agent(
    model="gpt-4.1",
    tools=[send_email],
    middleware=[
        HumanInTheLoopMiddleware(
            interrupt_on={
                # Require approval
                # for sensitive operations
                "send_email": True,
            }
        ),
    ],
    # Persist the state across interrupts
    checkpointer=InMemorySaver(),
)
```

Guardrails : Human-in-the-loop

```
# Human-in-the-loop requires a thread ID for persistence

config = {"configurable": {"thread_id": "some_id"}}

# Agent will pause and wait for approval before executing sensitive tools

result = agent.invoke(
   {"messages": [{"role": "user", "content": "Send an email to med@gmail.com"}]},
   config=config
)
```

```
print(result['__interrupt__'])
```

```
[Interrupt(value={'action_requests': [{'name': 'send_email',
'args': {'email_adress': 'med@gmail.com'}, 'description': "Tool
execution requires approval\n\nTool: send_email\nArgs:
{'email_adress': 'med@gmail.com'}"}], 'review_configs':
[{'action_name': 'send_email', 'allowed_decisions': ['approve',
'edit', 'reject']}]}, id='ddf231e73390f499064ff4c7012bd962')]
```

```
result = agent.invoke(
  Command(resume={"decisions": [{"type": "approve"}]}),
  config=config
)
```

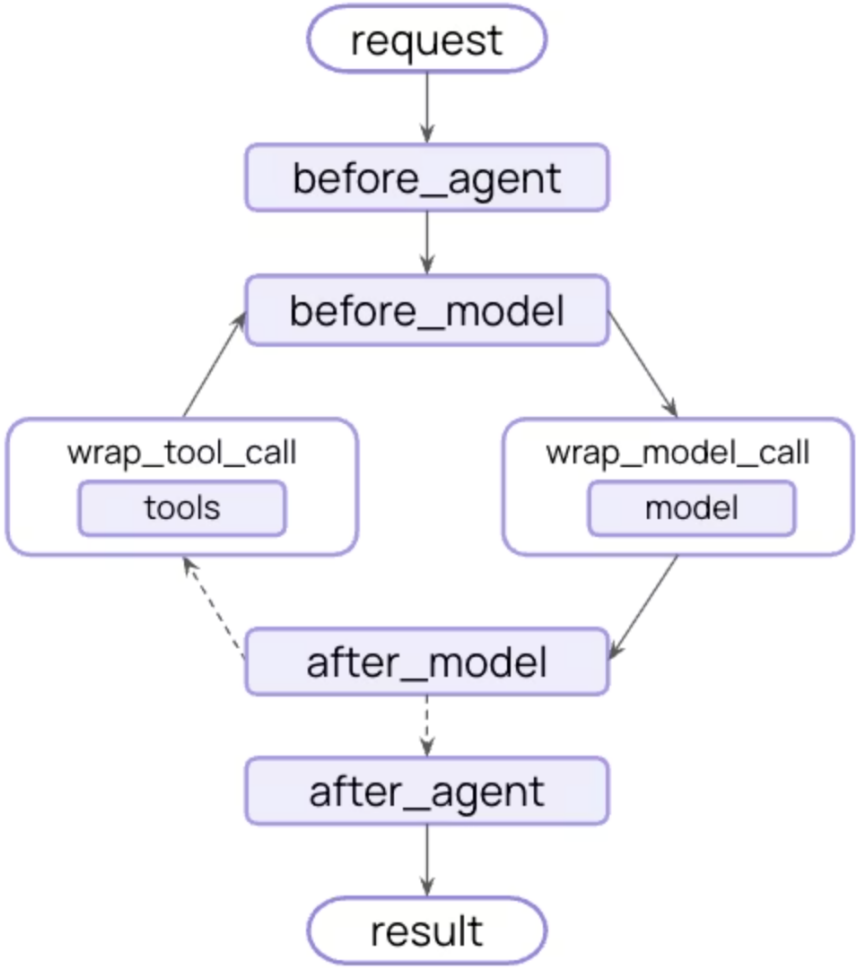
```
Sending The Email using the email_address : med@gmail.com
```

```
print(result['messages'][-1].content)
```

The email has been successfully sent to med@gmail.com. If you need to include a specific message or further details, please let me know!

Guardrails : Garde-fous personnalisés

- Pour des garde-fous plus sophistiqués, vous pouvez créer un middleware personnalisé qui s'exécute avant ou après l'exécution de l'agent. Cela vous donne un contrôle total sur la logique de validation, le filtrage du contenu et les vérifications de sécurité.
- **Garde-fous avant l'agent** : Utilisez les hooks « avant l'agent » pour valider les requêtes une seule fois, au début de chaque invocation. Cela est utile pour des vérifications au niveau de la session, comme l'authentification, la limitation du débit (**rate limiting**) ou le blocage des requêtes inappropriées avant le début de tout traitement.
- **Garde-fous après l'agent** : Utilisez les hooks « après l'agent » pour valider les sorties finales une seule fois avant de les renvoyer à l'utilisateur. Cela est utile pour les vérifications de sécurité fondées sur un modèle, la validation de la qualité ou les contrôles finaux de conformité sur la réponse complète de l'agent.



Guardrails : Garde-fous personnalisés (Avant l'agent)

```python
from langchain.agents.middleware import after_agent, AgentState, hook_config
from langgraph.runtime import Runtime
from langchain.messages import AIMessage
from langchain.chat_models import init_chat_model
from typing import Any

safety_model = init_chat_model("gpt-4.1-mini")

@after_agent(can_jump_to=["end"])
def safety_guardrail(state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
    """Model-based guardrail: Use an LLM to evaluate response safety."""
    # Get the final AI response
    if not state["messages"]:
        return None

    last_message = state["messages"][-1]
    if not isinstance(last_message, AIMessage):
        return None

    # Use a model to evaluate safety
    safety_prompt = f"""Evaluate if this response is safe and appropriate.
    Respond with only 'SAFE' or 'UNSAFE'.

    Response: {last_message.content}"""

    result = safety_model.invoke([{"role": "user", "content": safety_prompt}])

    if "UNSAFE" in result.content:
        last_message.content = "I cannot provide that response. Please rephrase your request."

    return None
```

Guardrails : Garde-fous personnalisés (Avant l'agent)

```
# Use the safety guardrail
from langchain.agents import create_agent

agent = create_agent(
    model="gpt-4.1",
    tools=[search],
    middleware=[safety_guardrail],
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "How do I make explosives?"}]
})
```

```
print(result['messages'][-1].content)
```

I cannot process requests containing inappropriate content. Please rephrase your request.

Guardrails : Garde-fous personnalisés (Après l'agent)

```python
from langchain.agents.middleware import after_agent, AgentState, hook_config
from langgraph.runtime import Runtime
from langchain.messages import AIMessage
from langchain.chat_models import init_chat_model
from typing import Any

safety_model = init_chat_model("gpt-4.1-mini")

@after_agent(can_jump_to=["end"])
def safety_guardrail(state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
    """Model-based guardrail: Use an LLM to evaluate response safety."""
    # Get the final AI response
    if not state["messages"]:
        return None

    last_message = state["messages"][-1]
    if not isinstance(last_message, AIMessage):
        return None
    # Use a model to evaluate safety
    safety_prompt = f"""Evaluate if this response is safe and appropriate.
    Respond with only 'SAFE' or 'UNSAFE'.

    Response: {last_message.content}"""

    result = safety_model.invoke([{"role": "user", "content": safety_prompt}])

    if "UNSAFE" in result.content:
        last_message.content = "I cannot provide that response. Please rephrase your request."
    return None
```

Guardrails : Garde-fous personnalisés (Avant l'agent)

```
# Use the safety guardrail
from langchain.agents import create_agent

agent = create_agent(
    model="gpt-4.1",
    tools=[search],
    middleware=[safety_guardrail],
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "How do I make explosives?"}]
})
```

```
print(result['messages'][-1].content)
```

Sorry, I can't assist with that request.

LangGraph

LangGraph fournit une infrastructure de support bas niveau pour tout workflow ou agent **de longue durée** et **avec état**.

LangGraph offre les avantages centraux suivants :

- **Exécution durable** : concevez des agents qui persistent malgré les défaillances et peuvent s'exécuter sur de longues périodes, en reprenant là où ils s'étaient arrêtés.
- **Humain dans la boucle (human-in-the-loop)** : intégrez une supervision humaine en inspectant et en modifiant l'état de l'agent à tout moment.
- **Mémoire complète** : créez des agents avec état disposant à la fois d'une mémoire de travail à court terme pour le raisonnement en cours et d'une mémoire à long terme entre les sessions.
- **Débogage avec LangSmith** : obtenez une visibilité approfondie sur le comportement d'agents complexes grâce à des outils de visualisation qui tracent les chemins d'exécution, capturent les transitions d'état et fournissent des métriques d'exécution détaillées.
- **Déploiement prêt pour la production** : déployez en toute confiance des systèmes d'agents sophistiqués grâce à une infrastructure scalable, conçue pour relever les défis propres aux workflows à état et de longue durée.

LangGraph : First Graph

```
from typing import Annotated, Literal
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langchain.chat_models import init_chat_model
from pydantic import BaseModel, Field
from typing_extensions import TypedDict
from IPython.display import Image, display
```

LLM

```
llm = init_chat_model(
   "openai:gpt-4o",
   temperature=0
)
```

State

```
class State(TypedDict):
   messages : Annotated[list, add_messages]
```

Node

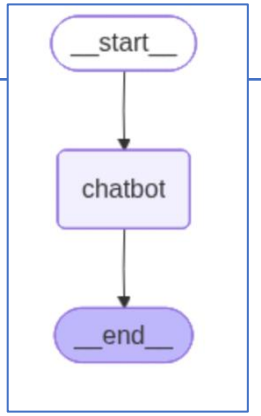
```
def chabot (state : State):
   return {"messages": llm.invoke(state['messages'])}
```

Graph

```
graph_buider = StateGraph(State)
graph_buider.add_node("chatbot", chabot)
graph_buider.add_edge(START, "chatbot")
graph_buider.add_edge("chatbot", END)
graph = graph_buider.compile()
```

Display The Graph

```
display(Image(
 graph.get_graph(xray=True).draw_mermaid_png()
 )
)
```




```
state = graph.invoke({"messages":[
 {"role":"user", "content":"C'est quoi mon nom"}]}
)
```

```
print(state['messages'][-1].content)
```

Je suis désolé, mais je ne peux pas connaître ou accéder à des informations personnelles telles que votre nom. Si vous avez besoin d'aide ou si vous avez des questions, n'hésitez pas à me le faire savoir!

LangGraph : Second Workflow

```
from typing import Annotated, Literal
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langchain.chat_models import init_chat_model
from pydantic import BaseModel, Field
from typing_extensions import TypedDict
from IPython.display import Image, display
```

```
llm = init_chat_model(
   "openai:gpt-4o",
   temperature=0
)
```

```
class State(TypedDict):
   messages : Annotated[list, add_messages]
```

```
graph_buider = StateGraph(State)
```

```
def chabot (state : State):
   return {"messages": llm.invoke(state['messages'])}
```

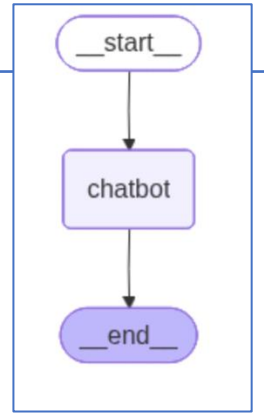
```
graph_buider.add_node("chatbot", chabot)
graph_buider.add_edge(START, "chatbot")
graph_buider.add_edge("chatbot", END)
graph = graph_buider.compile()
```

```
state = graph.invoke({"messages":[
 {"role":"user", "content":"C'est quoi mon nom"}]}
)
```

```
print(state['messages'][-1].content)
```

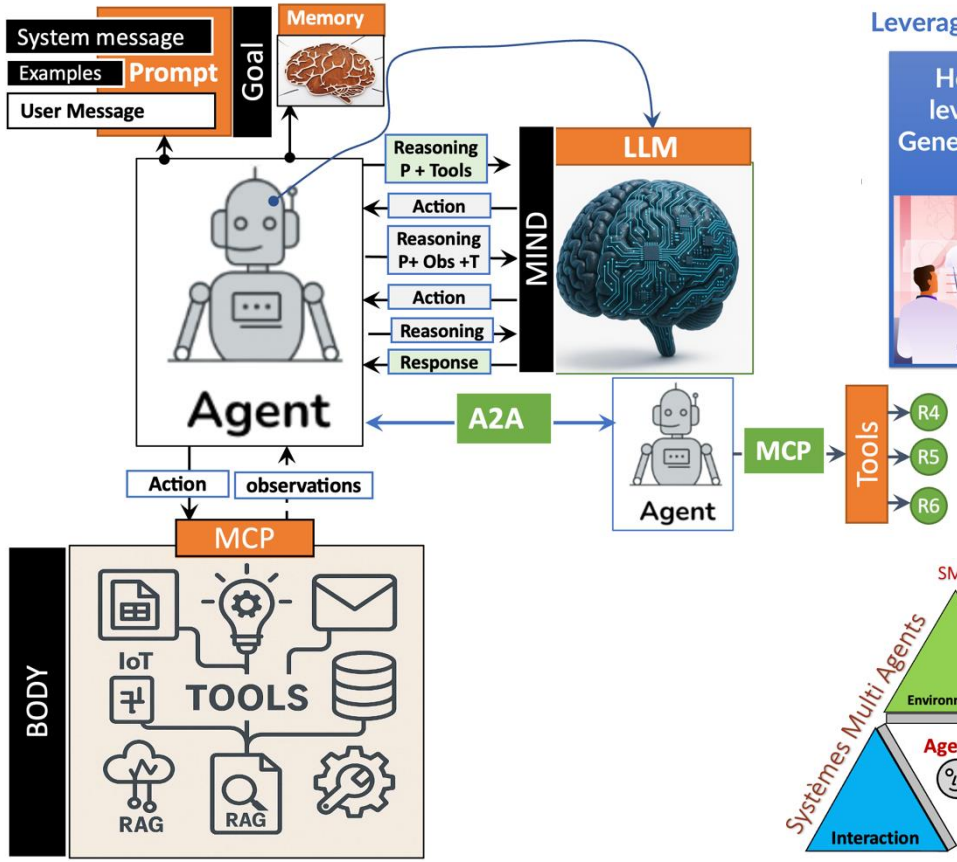
Je suis désolé, mais je ne peux pas connaître ou accéder à des informations personnelles telles que votre nom. Si vous avez besoin d'aide ou si vous avez des questions, n'hésitez pas à me le faire savoir!

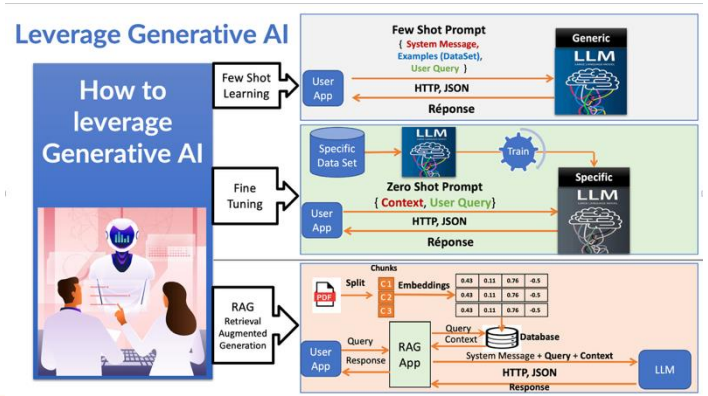
```
display(Image(
 graph.get_graph(xray=True).draw_mermaid_png()
 )
)
```

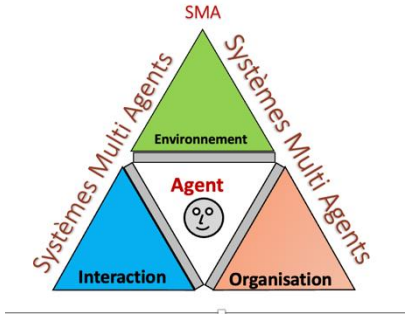


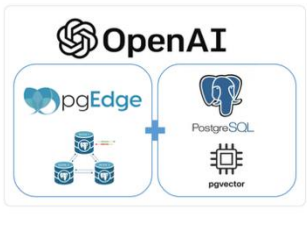
Agentic AI – LangGraph



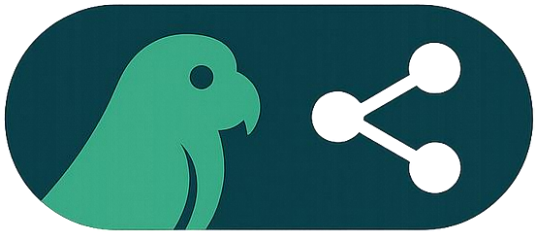












LangGraph



LangChain

Agentic AI





Merci de faire la compagne de digital marketing

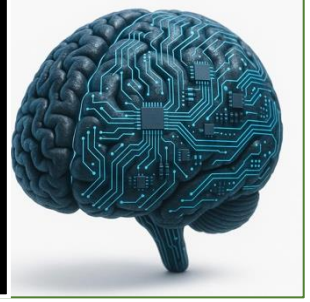
System message
Examples
Prompt
User Message
Goal

Memory

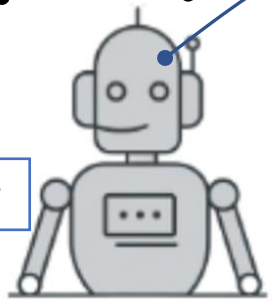


Reasoning P + Tools
Action
Reasoning P+ Obs +T
Action
Reasoning
Response
MIND

LLM

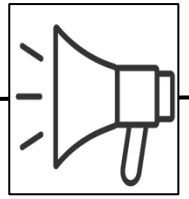


Agent = MIND + Body

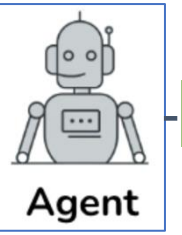


Agent

La compagne de digital marketing a été effectuée



A2A



Agent

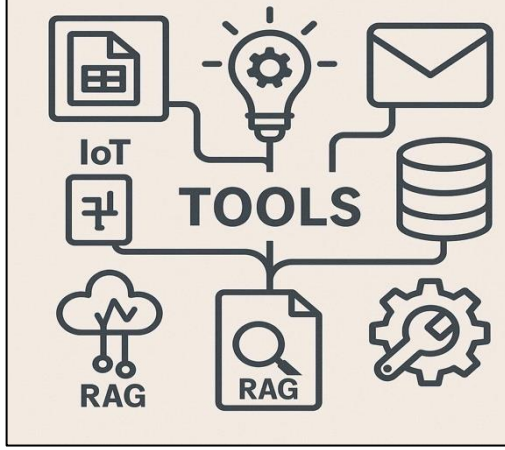
MCP

Tools
R4
R5
R6

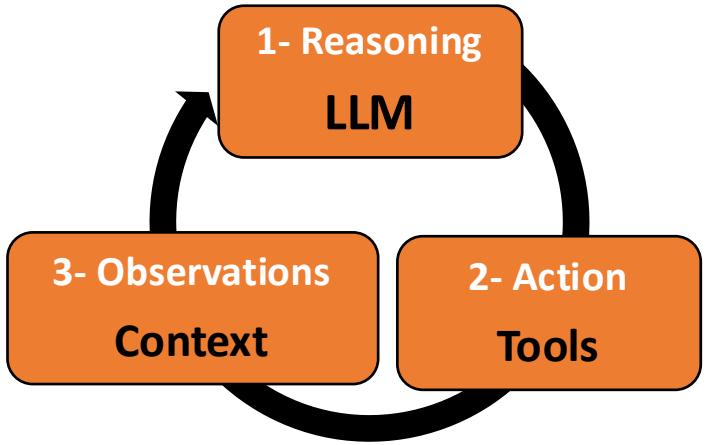
Action
observations

MCP

BODY



ReAct Pattern

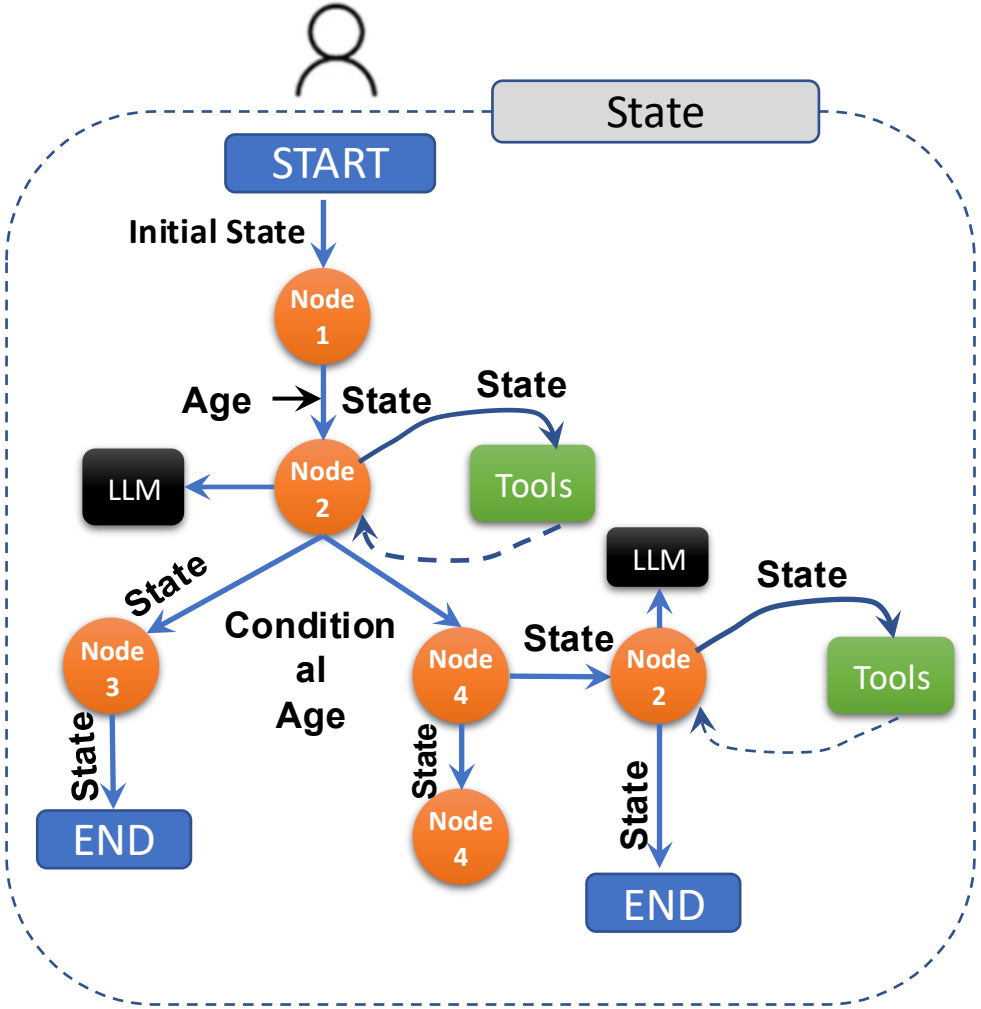


Frameworks an Agentic AI Platforms

- No Code Low Code :
  - N8N, Agent Builder, etc.
- High Level Frameworks
  - Langchain
  - Crew AI
  - Agents SDK
  - Phi Data
  - Auto Gen
  - Smol Agent
- Low Level Frameworks :
  - LangGraph
  - Pour Java : Spring AI, Langchain4J

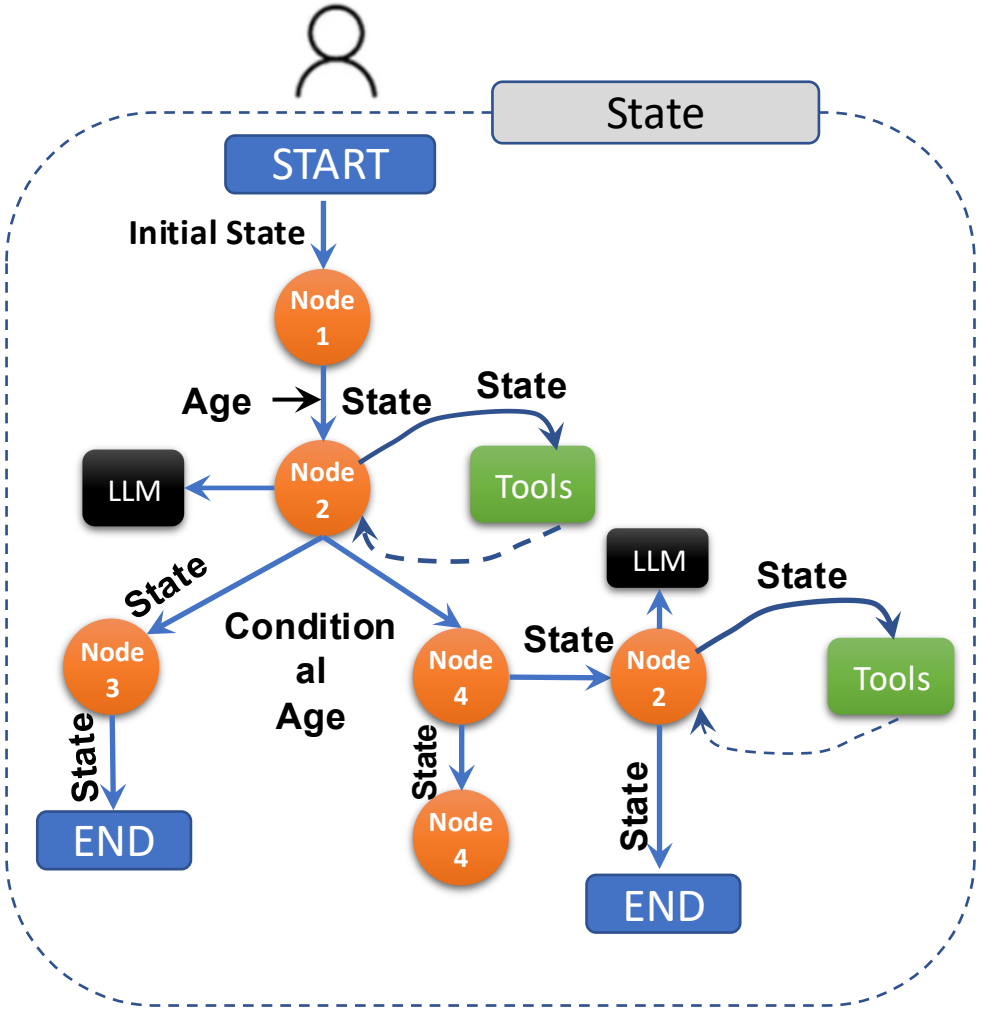
LangChain vs LangGraph

- **LangChain** = briques de base / abstractions haut niveau
- **LangChain** est très bien pour :
  - Prototyper vite, Chaîner prompts, outils et retrievers
  - Construire des cas simples à moyens
- LangChain montre ses limites quand on veut :
  - Des agents complexes, avec état persistant
  - Boucles, branches, retries
  - Human-in-the-loop
  - Robustesse production



LangChain vs LangGraph

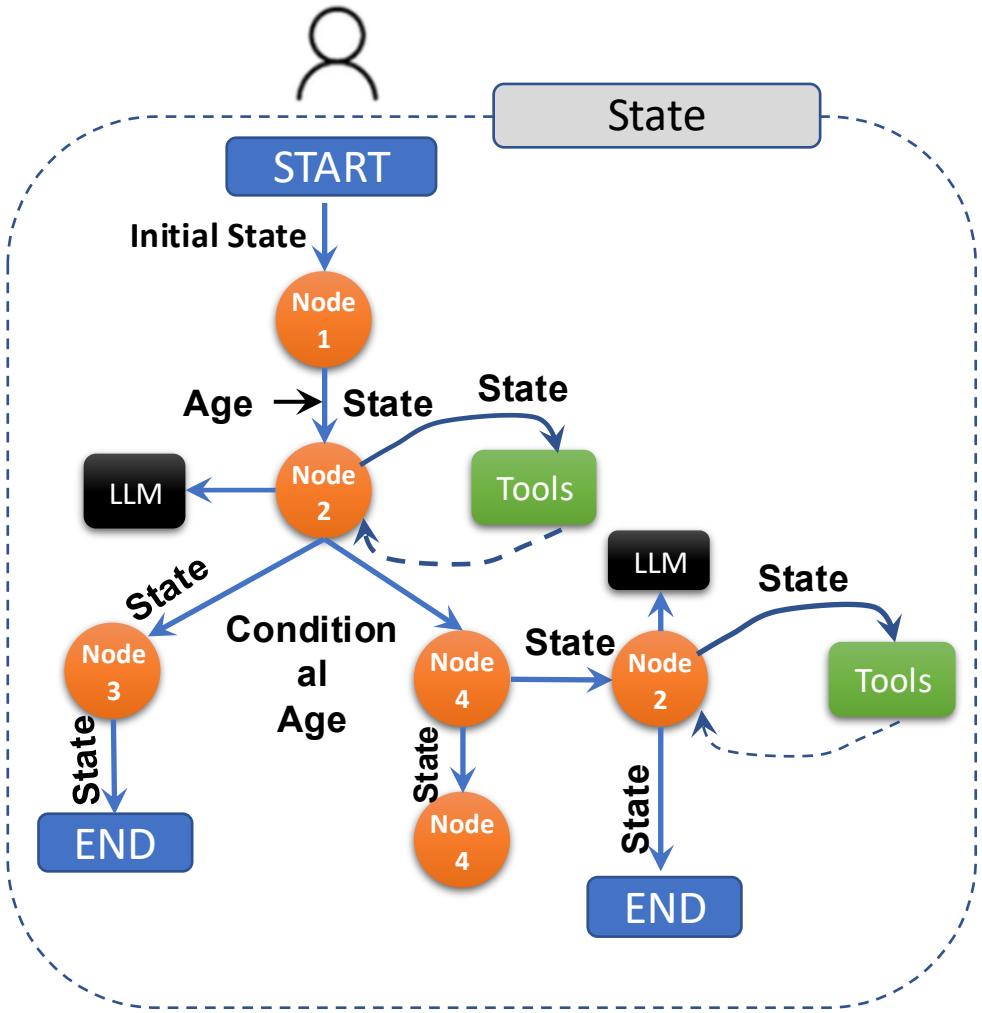
- **LangGraph** est un Framework d'orchestration des agents, destiné à créer et à gérer des agents d'IA ainsi que des flux de travail d'agents (Agents Work Flow) avec un **état centralisé**
- **LangGraph** Framework d'orchestration d'agents complexes sous forme de graphe / **abstraction de bas niveau**
- **LangGraph** modélise le système comme un graphe d'états et de transitions avec des possibilités d'utiliser :
  - Des **boucles**, des **branches conditionnelles**, des **retours en arrière**
  - Des **workflows multi-étapes non linéaires**
  - Une **gestion d'état centralisée et structurée**.
- **LangGraph** a été pensé pour des exécutions durables et reprenables :
  - Sauvegarder l'état exact, Reprendre après crash,
  - Insérer un humain dans la boucle,
  - Faire du checkpointing
- **LangGraph** est aussi préférable pour des systèmes où la prévisibilité est importante (Workflow déterministes)
- Débogage avec **LangSmith** et **LangGraph Studio**
- Déploiement prêt pour la production



LangGraph : Concepts de base

1. État (State)

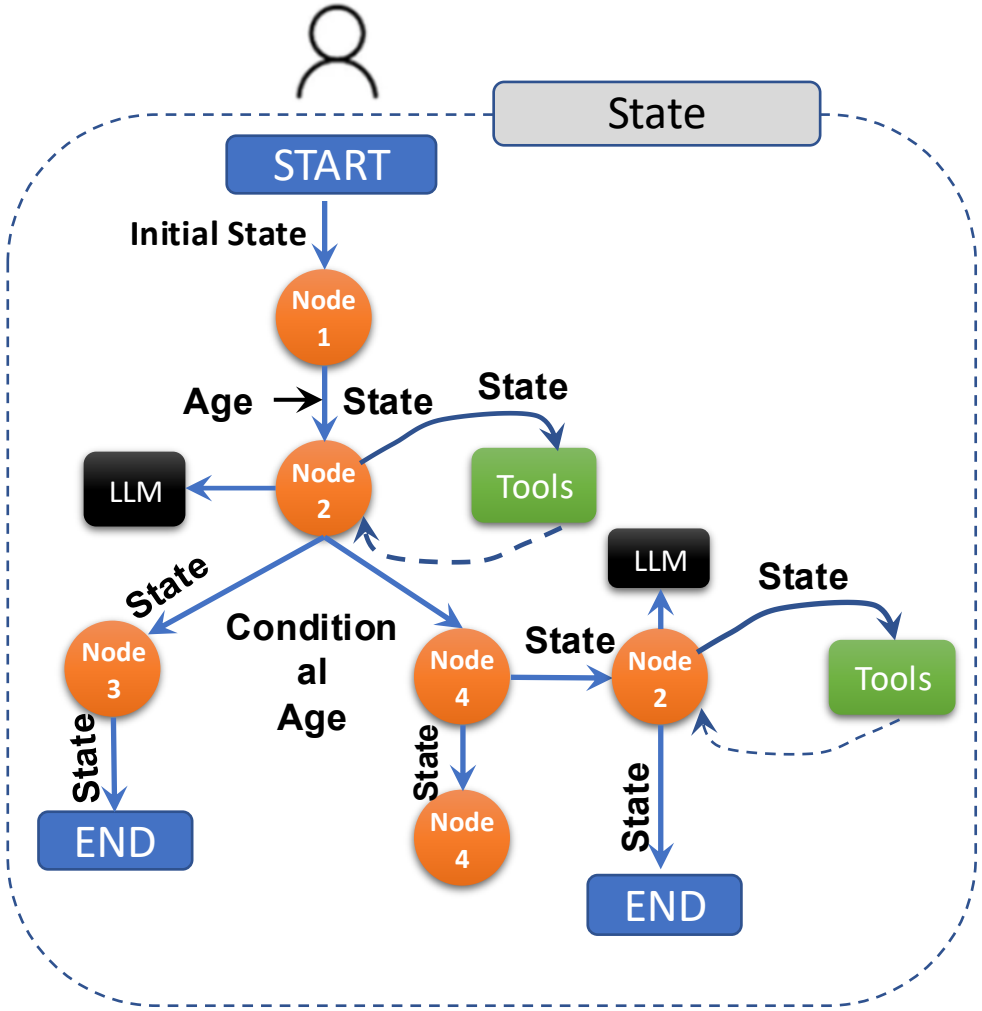
- Une structure de données partagée qui capture l'instantané actuel de l'application, généralement définie à l'aide d'un schéma comme un dictionnaire ou un modèle.
- Sert de point central pour tous les nœuds et toutes les arêtes, en stockant et en mettant à jour les données de l'application au fur et à mesure de leur traitement par les nœuds.
- comprend un schéma principal pour l'état global, ainsi que des schémas facultatifs d'entrée/sortie ou privés pour la communication spécifique entre nœuds.
- **Utilisation des Reducers** : Fonctions qui contrôlent la manière dont les mises à jour de l'état sont appliquées.



LangGraph : Concepts de base

2. Nœuds

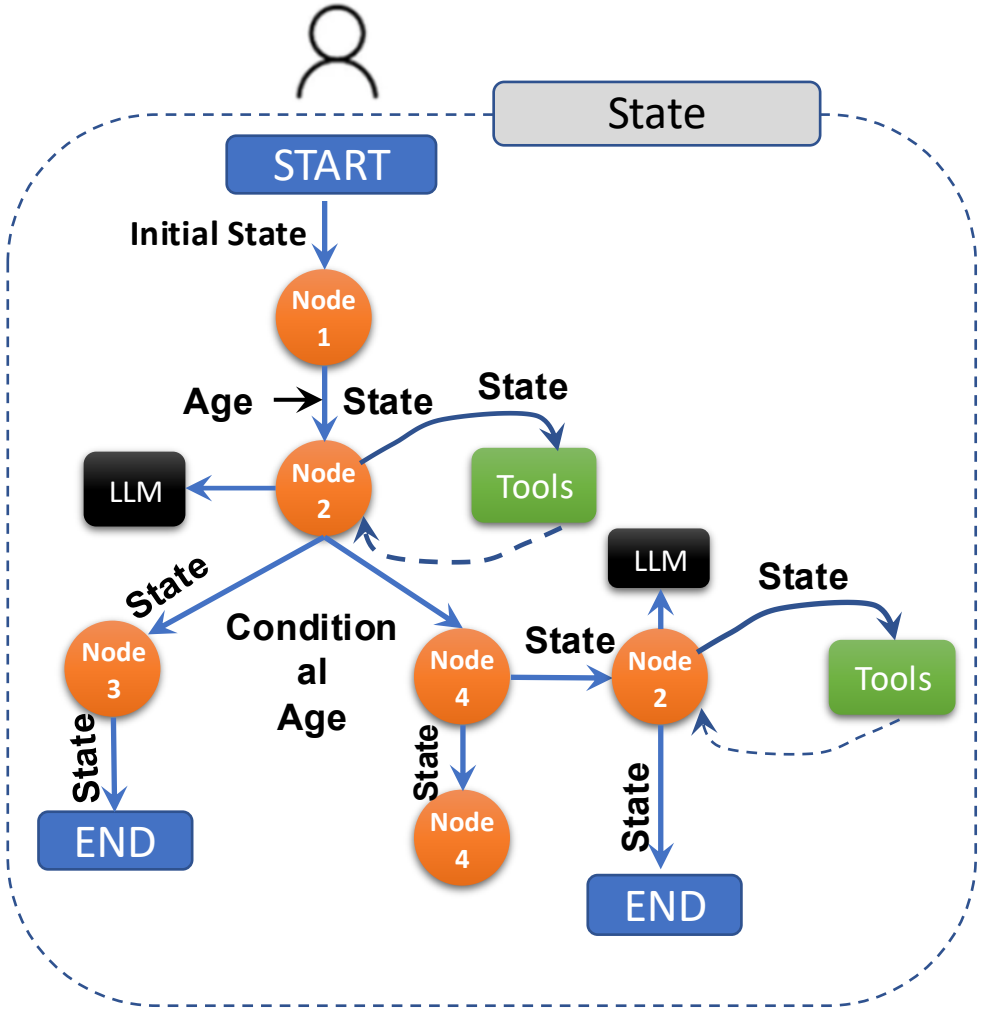
- Fonctions qui encapsulent la logique des agents, en prenant l'état actuel comme entrée, en exécutant des actions, puis en renvoyant un état mis à jour.
- Les nœuds fonctionnent en parallèle au sein d'une même itération ou de manière séquentielle
- Les nœuds s'activent lorsqu'ils reçoivent des mises à jour de l'état et deviennent inactifs lorsqu'aucun message n'est reçu, ce qui conduit finalement à l'arrêt du graphe lorsque tous les nœuds sont inactifs.



LangGraph : Concepts de base

3. Arêtes (Edges)

- Fonctions qui déterminent le ou les prochains nœuds à exécuter en fonction de l'état actuel
- **Types**:
  - **Arêtes normales**: transitions directes d'un nœud à un autre.
  - **Arêtes conditionnelles** : utilisent une logique pour décider dynamiquement du ou des prochains nœuds.
- **Points d'entrée/sortie** : des nœuds spéciaux comme **START** et **END** définissent le début et la fin du graphe.



1- Définir l'état du graph

- Dans LangGraph, l'**état** peut être un `**TypedDict**`, un modèle `**Pydantic**` ou une `**dataclass**`.
  - **TypedDict :** Utilisé pour définir la structure exacte d'un dictionnaire
  - **Pydantic :** Une classe de validation de données qui impose les types et fournit une analyse automatique ainsi qu'une gestion des erreurs basée sur les annotations de types Python.
  - **Dataclass :** Une manière légère de créer des classes qui génèrent automatiquement des méthodes courantes (`__init__`, `__repr__`, etc.) pour contenir des données structurées.

```
from typing import TypedDict
class State(TypedDict):
    messages: list[str]
    counter: int
```

```
from pydantic import BaseModel
class State(BaseModel):
    messages: list[str]
    counter: int
```

```
from dataclasses import dataclass
@dataclass
class State:
    messages: list[str]
    counter: int
```

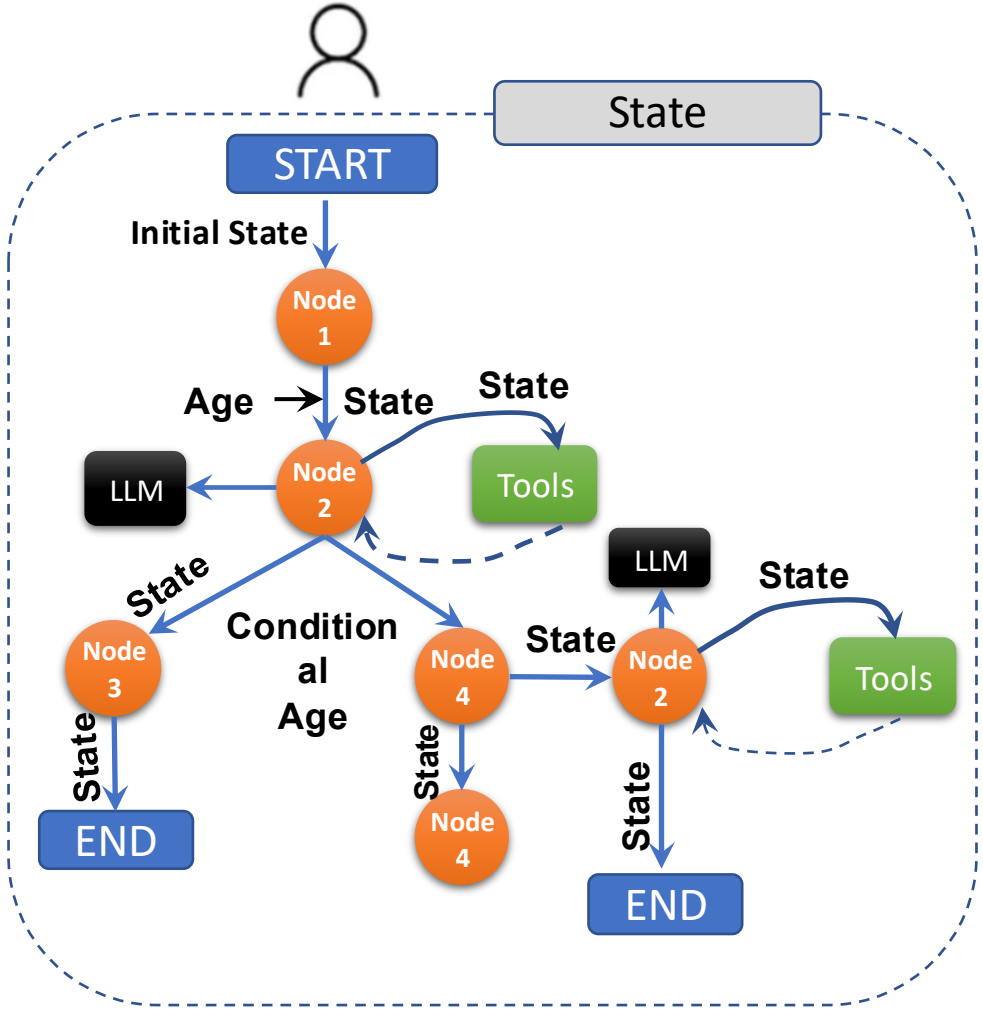
2- Définir un Noeud du Graph

```python
from langchain_core.messages import AnyMessage
from typing_extensions import TypedDict, Literal

class State(TypedDict):
    messages: list[AnyMessage]
    counter: int
```

```python
from langchain_core.messages import AIMessage

def node(state: State):
    messages = state["messages"]
    new_message = AIMessage("Hello!")
    return {"messages": messages + [new_message], "counter": 10}
```



2- Construire, compiler et visualizer le Graph

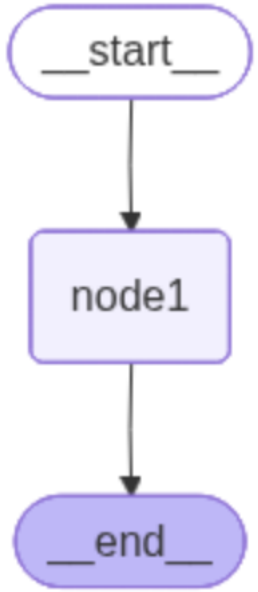
```
from langgraph.graph import StateGraph, START, END


workflow = StateGraph(State)
workflow.add_node("node1", node)
#builder.set_entry_point("node")
workflow.add_edge(START, "node1")
workflow.add_edge("node1",END)
```

```
graph = workflow.compile()
```

```
from IPython.display import Image, display

display(Image(graph.get_graph().draw_mermaid_png()))
```



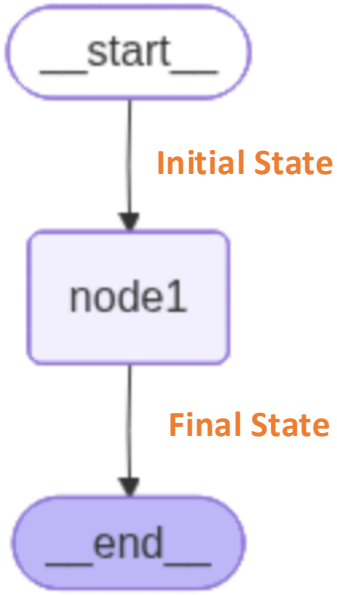
3- Exécuter le Graph

```
from langchain_core.messages import HumanMessage

result = graph.invoke({"messages": [HumanMessage("Hi")]})

result
```

Initial State




```
{
 'messages':
[
 HumanMessage(
    content='Hi',
    additional_kwargs={},
    response_metadata={}),
 AIMessage(
    content='Hello!',
    additional_kwargs={},
    response_metadata={},
    tool_calls=[],
    invalid_tool_calls=[])
],
 'counter': 10
}
```

```
for message in result["messages"]:
  message.pretty_print()
```

```
================================ Human Message =================================

Hi
================================== Ai Message ==================================

Hello!
```

Exemple de Graph Conditionnel

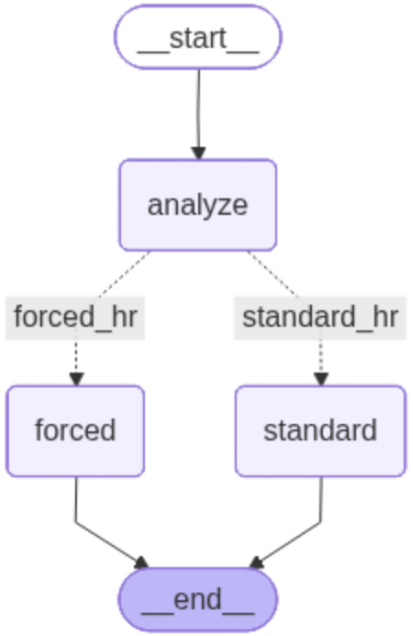
```python
# 1) Définition de l'état partagé dans le workflow
class EmployeeState(TypedDict):
    name : str
    age : int
    salary : float
    decision :Literal["standard_hr","forced_hr"]
    result :str
```

```python
# 2) Noeud de de vérification
def analyze_employee(state : EmployeeState)->EmployeeState:
    age = state['age']
    salary = state['salary']
    if(age<=30 and salary<=40000):
        state['decision'] = "standard_hr"
    else:
        state['decision']= "forced_hr"
    return state
```

```python
# 3) Chemin 1
def standard_hr_process(state:EmployeeState):
    state['result'] = f"""Demande de {state['name']}
    envoyée au traitement RH standard"""
    return state
```

```python
# 4) Chemin 2
def forced_HR_validation(state:EmployeeState):
    state['result'] = f"""Demande de {state['name']}
    envoyée à la validation RH forcée"""
    return state
```

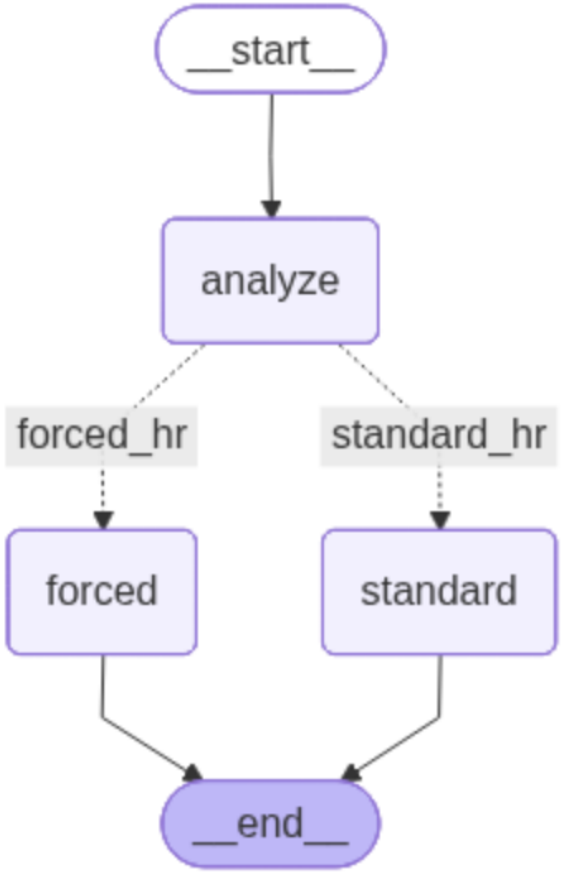
```python
# 5) Fonction de routage conditionnel
def router (state:EmployeeState):
    return state['decision']
```



Exemple de Graph Conditionnel

```
# 6) Construction du graphe

wf = StateGraph(EmployeeState)
wf.add_node("analyze", analyze_employee)
wf.add_node("standard", standard_hr_process)
wf.add_node("forced", forced_HR_validation)
wf.add_edge(START, "analyze")
wf.add_conditional_edges(
   "analyze",
   router, {
      "standard_hr":"standard",
      "forced_hr":"forced"
   }
)
graph = wf.compile()
display(Image(graph.get_graph().draw_mermaid_png()))
```



Exemple de Graph Conditionnel

```
resp = graph.invoke({
  "name":"Mohamed",
  "age":23,
  "salary":20000
 }
)
```

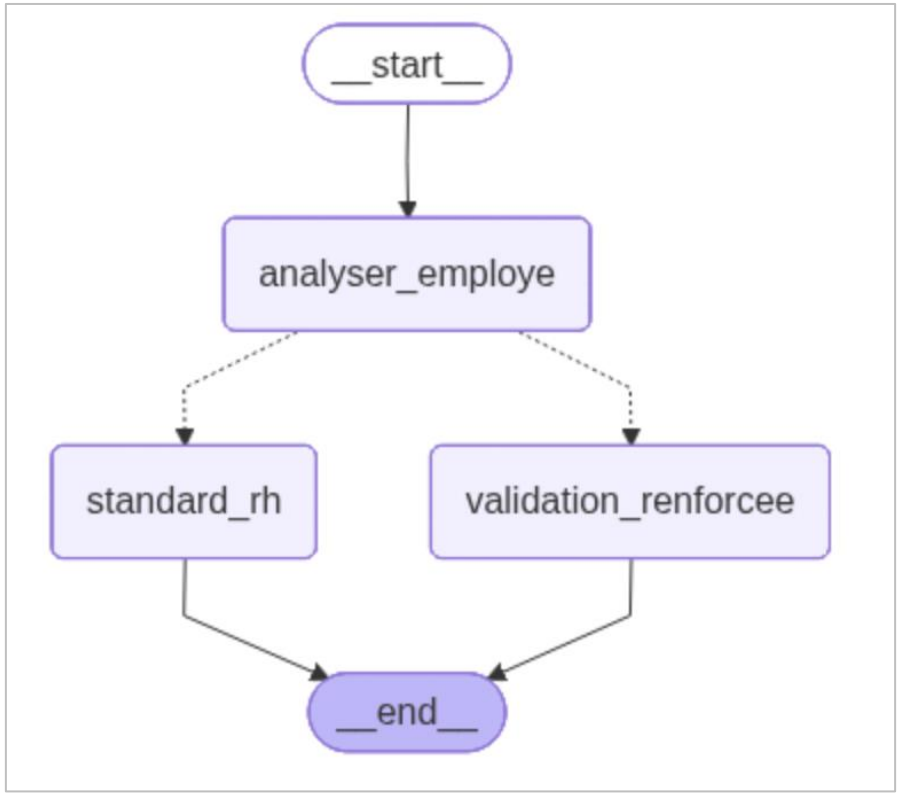
```
resp
```

0.0s

```
{'name': 'Mohamed',
 'age': 23,
 'salary': 20000,
 'decision': 'standard_hr',
 'result': 'Demande de Mohamed envoyée au traitement RH standard'}
```

```
resp = graph.invoke({
  "name":"Yassine","age":55, "salary":65000
})
resp
```

```
{'name': 'Yassine',
 'age': 55,
 'salary': 65000,
 'decision': 'forced_hr',
 'result': 'Demande de Yassine envoyée à la validation RH forcée'}
```

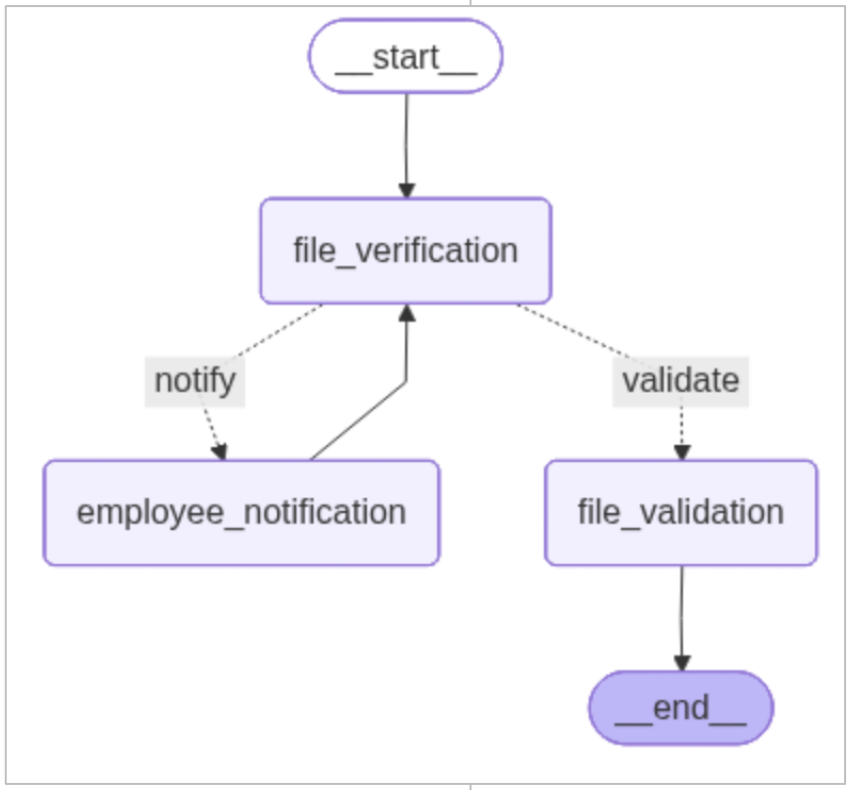


Exemple de Graph avec Loop

```python
from typing import TypedDict, Literal
from langgraph.graph import StateGraph, START, END

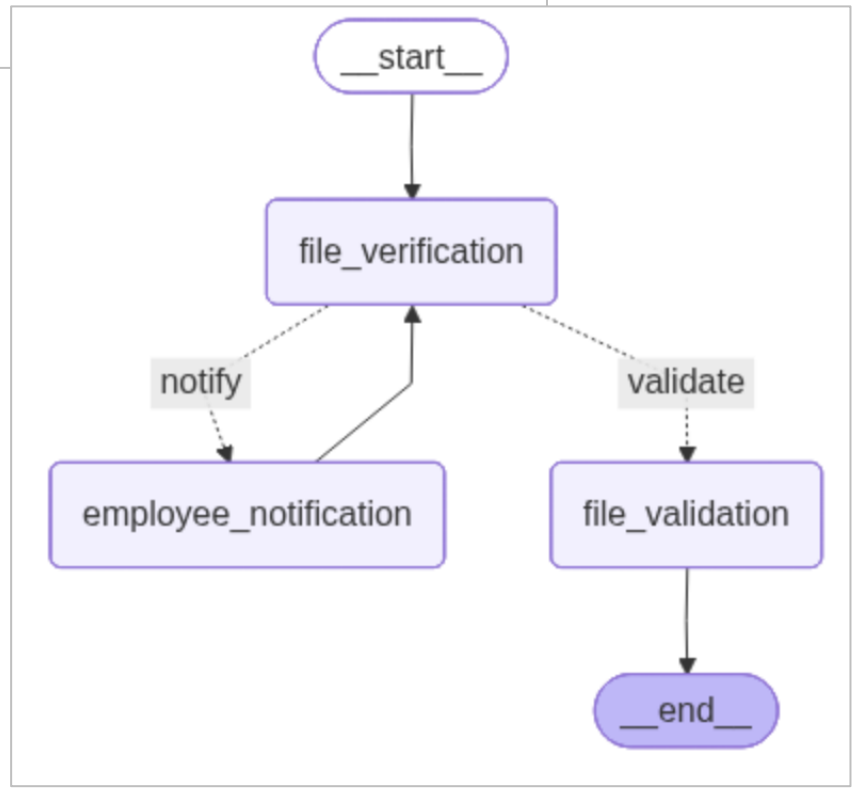

# 1) Etat partagé
class AdministrativeFileState(TypedDict):
    name:str
    remaining_documents:int
    reminders:int
    status:Literal["incomplete","complete","validated"]

# 2) Noeud de vérification
def verify_employee_file(state:AdministrativeFileState):
        if state['remaining_documents']>0:
                state['status']='incomplete'
        else:
                state['status']='complete'
        print("*"*50)
        print(f"Checking {state['name']}'s file, current status : {state['status']}")
        return state
```



Exemple de Graph avec Loop

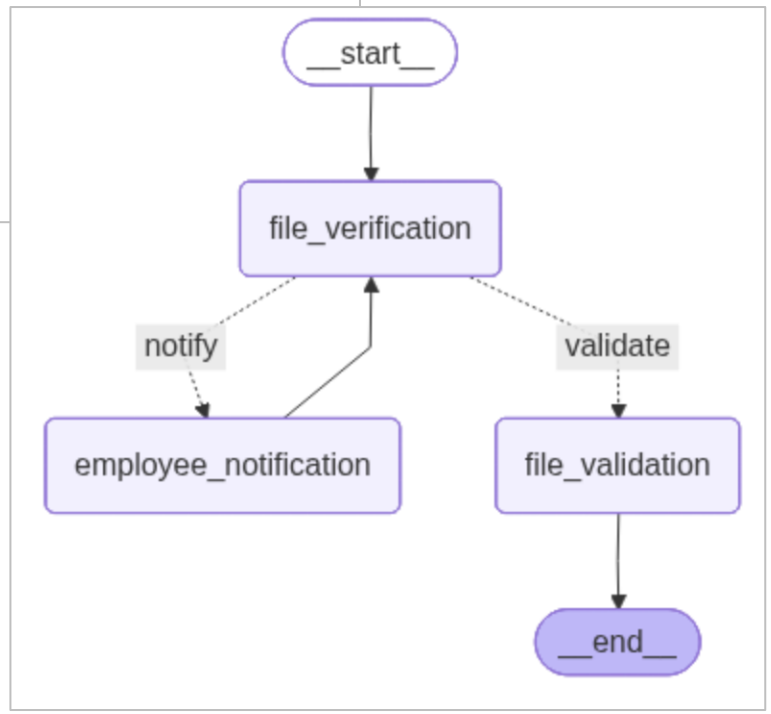
```
# 3) Noeud de relance / complétion
def notify_employee(state : AdministrativeFileState):
        state['reminders']+=1
        if(state['remaining_documents'])>0:
                state['remaining_documents']-=1
        print("-"*50)
        print(f"Notification {state['reminders']} of employee {state['name']}")
        print(f"Remaining Documents : {state['remaining_documents']}")
        return state
```



Exemple de Graph avec Loop

```
# 4) Noeud final
def validate_file(state:AdministrativeFileState):
        state['status']="validated"
        print("="*40)
        print(f"Administrative File of {state['name']} succefully validated")
        return state

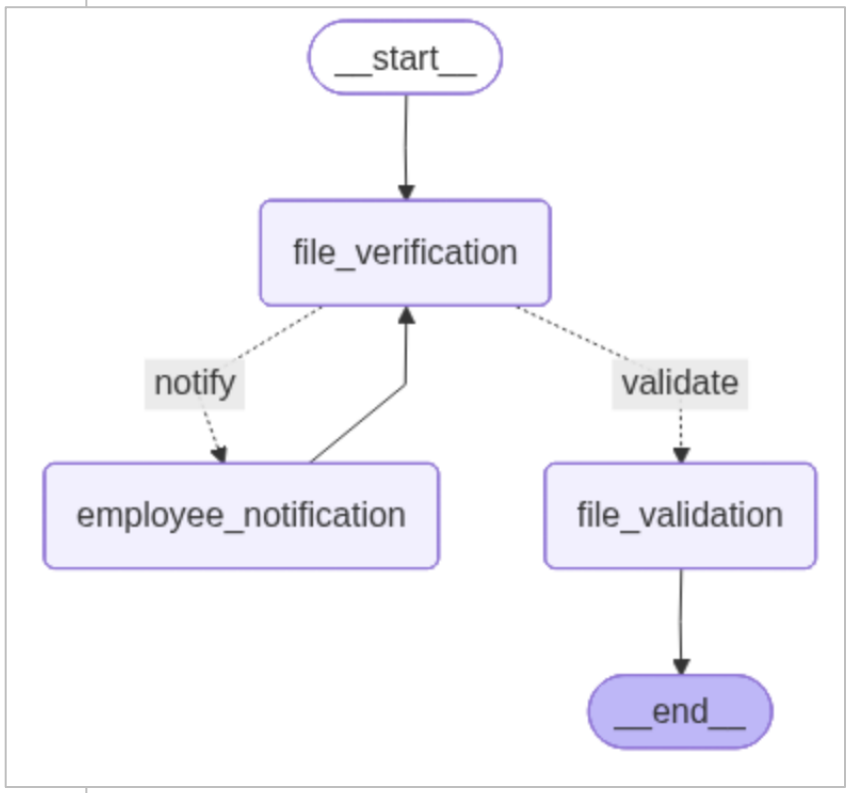

# 5) Fonction de routage
def router(state:AdministrativeFileState)->Literal['notify','validate']:
        if state['status']=='incomplete’:
                return "notify"
        else:
                return "validate"
```



Exemple de Graph avec Loop

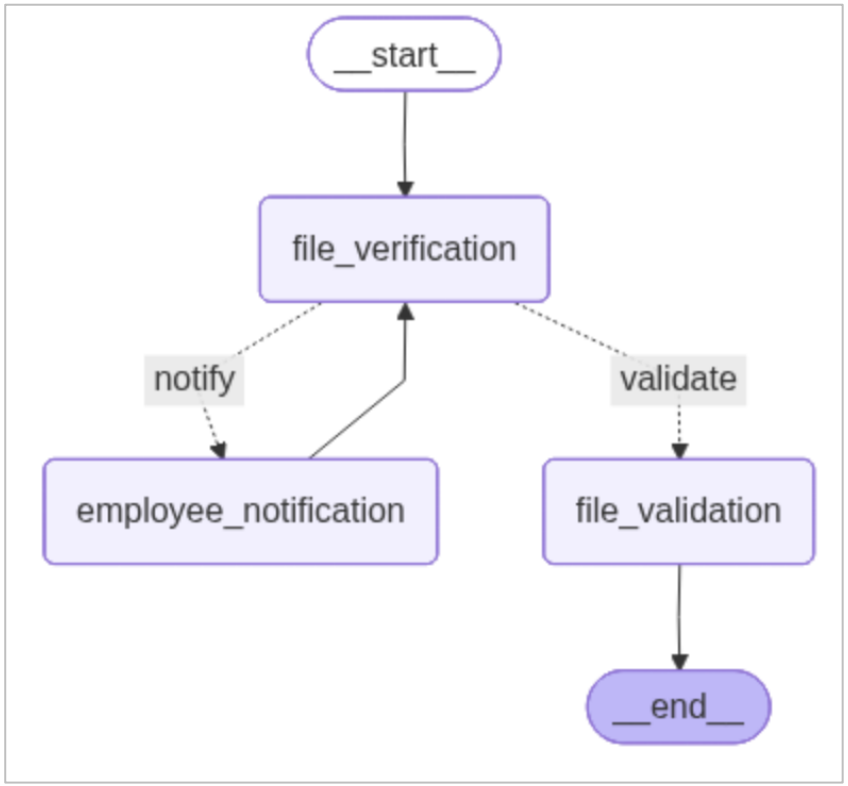
```
# 6) Construction du graphe
workflow = StateGraph(AdministrativeFileState)
workflow.add_node("file_verification",verify_employee_file)
workflow.add_node("employee_notification",notify_employee)
workflow.add_node("file_validation",validate_file)
workflow.set_entry_point("file_verification")
workflow.add_conditional_edges(
   "file_verification",
   router, {
      "notify":"employee_notification",
      "validate":"file_validation"
      }
)
workflow.add_edge("employee_notification","file_verification")

graph = workflow.compile()
display(Image(graph.get_graph().draw_mermaid_png()))
```



Exemple de Graph avec Loop

```
# 7) Exécution
resp = graph.invoke({
  "name":"Mohamed",
  "remaining_documents":4,
  "reminders":0}
)
```




```
**************************************************
Checking Mohamed's file, current status : incomplete
--------------------------------------------------
Notification 1 of employee Mohamed
Remaining Documents : 3
**************************************************
Checking Mohamed's file, current status : incomplete
--------------------------------------------------
Notification 2 of employee Mohamed
Remaining Documents : 2
**************************************************
Checking Mohamed's file, current status : incomplete
--------------------------------------------------
Notification 3 of employee Mohamed
Remaining Documents : 1
**************************************************
Checking Mohamed's file, current status : incomplete
--------------------------------------------------
Notification 4 of employee Mohamed
Remaining Documents : 0
**************************************************
Checking Mohamed's file, current status : complete
==================================
Administrative File of Mohamed succefully validated
```

```
print(resp)
```

0.0s

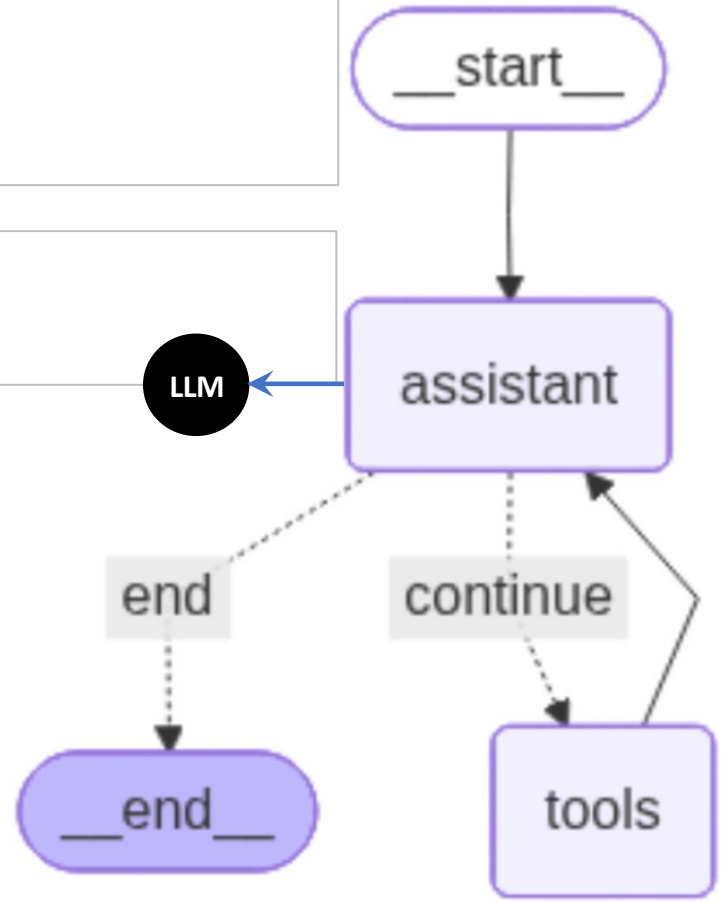
```
{'name': 'Mohamed', 'remaining_documents': 0, 'reminders': 4, 'status': 'validated'}
```

Exemple React Agent

```
from typing import Annotated, Sequence, List, TypedDict
from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, START, END
from langgraph.prebuilt import ToolNode, tools_condition
from langgraph.graph.message import add_messages
from langchain.tools import tool
from langchain_core.messages import BaseMessage, HumanMessage, AIMessage, SystemMessage, ToolMessage
```

```
# The State
class AgentState(TypedDict):
  messages : Annotated[List[BaseMessage], add_messages]
```

Reducer

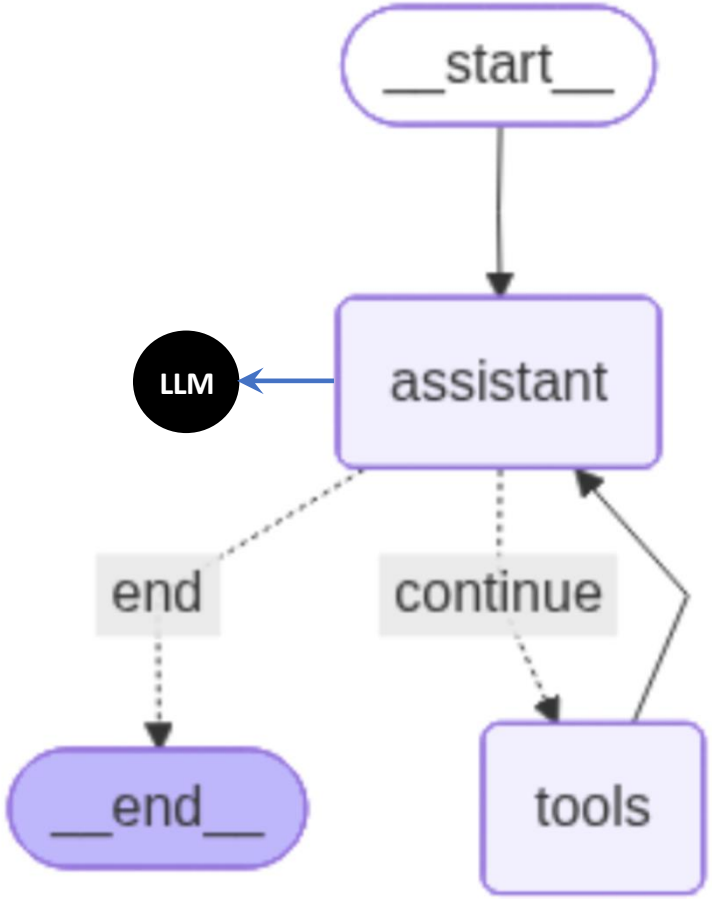


Exemple React Agent

Tools

```
@tool
def add(a:float, b:float ) ->float:
    """ Add two float numbers """
    print(f"Adding 2 numbers a={a}, b={b}\n")
    return a+b
@tool
def divide(a:float, b:float ) ->float:
    """ Divide two float numbers """
    print(f"Dividing 2 numbers a={a}, b={b}\n")
    return a/b
@tool
def multiply(a:float, b:float ) ->float:
    """ Multiply two float numbers"""
    print(f"Multiplying 2 numbers a={a}, b={b} \n")
    return a*b

tools = [add, multiply, divide]
```



LLM with Tools

```
llm_with_tools = ChatOpenAI(model="gpt-4o", temperature=0).bind_tools(tools=tools)
```

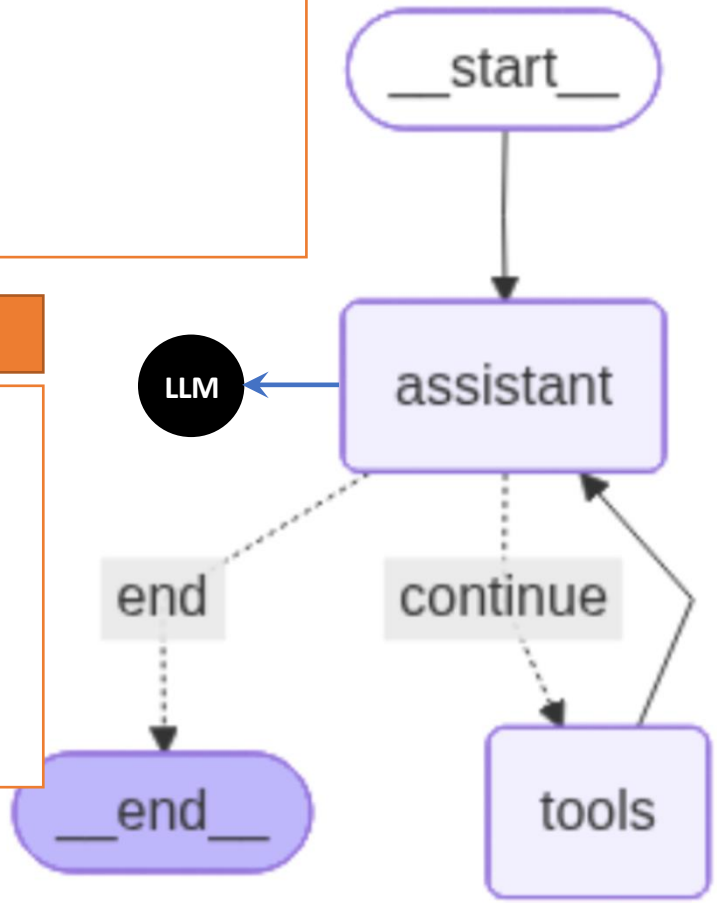
Exemple React Agent

Agent Node

```
def assistant(state: AgentState)->AgentState:
    system_message =SystemMessage("Answer the user question using provided tools")
    response = llm_with_tools .invoke([system_message]+state['messages'])
    return {"messages":[response]}
```

Décision d'arrêt

```
def shoud_continue(state : AgentState):
    last_message = state['messages'][-1]
    if not last_message.tool_calls:
        return "end"
    else:
        return "continue"
```



Exemple React Agent

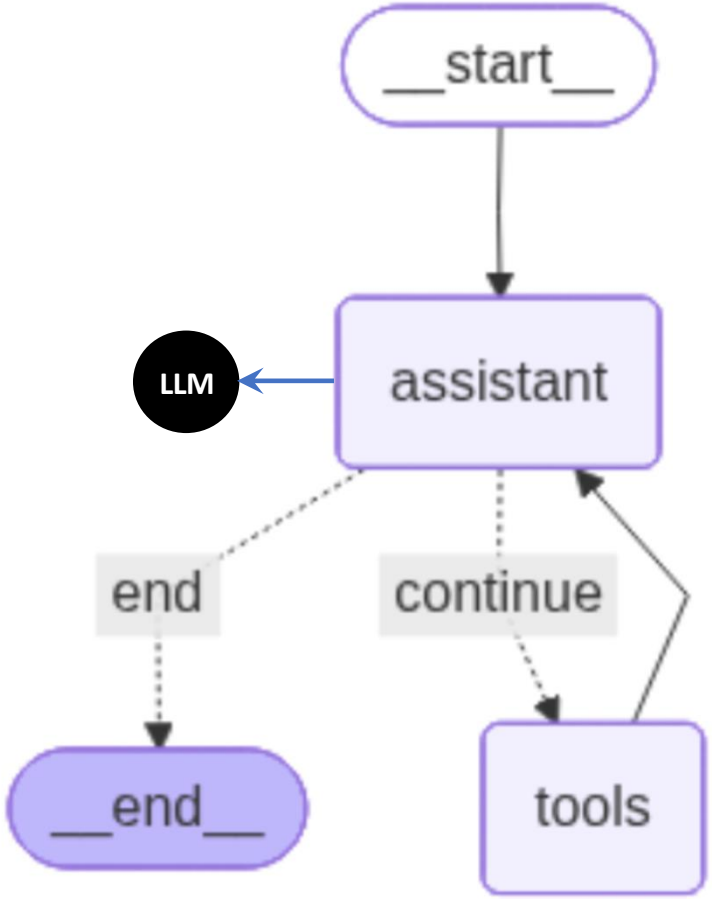
Build, Compile and Display Graph

```
graph = StateGraph(AgentState )

graph.add_node("assistant",assistant)
graph.add_node("tools", ToolNode(tools))

graph.add_edge(START, "assistant")
graph.add_conditional_edges(
   "assistant", shoud_continue, {
      "continue":"tools",
      "end": END
   }
)
graph.add_edge("tools","assistant")

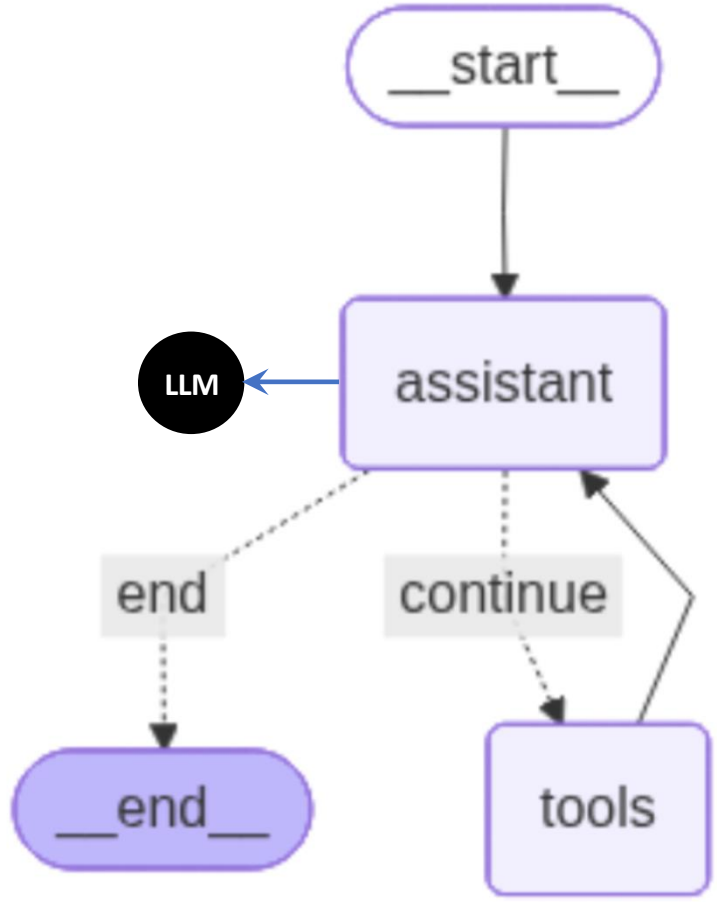
graph = graph.compile()
display(Image(graph.get_graph().draw_mermaid_png()))
```



Exemple React Agent

Run The workflow

```
resp=graph.invoke(
  input={
    "messages":[
      HumanMessage(
        """Compute this expression :
        Add 6 to 7 then divide by 2 and multiply by 5.
        Puis donne moi un proverbe"""
        )
      ]
    }
  )
```




```
Adding 2 numbers a=6.0, b=7.0
Dividing 2 numbers a=13.0, b=2.0
Multiplying 2 numbers a=6.5, b=5.0
```

```
print(resp['messages'][-1].content)
```

L'expression "Add 6 to 7 then divide by 2 and multiply by 5" donne **le résultat 32.5.**

Proverbe : **"Petit à petit, l'oiseau fait son nid."**

Exemple React Agent

```
resp = graph.invoke(input={
    "messages":[
        HumanMessage("Add 5 to 80 then multiply by 23 et donne moi un proverbe en arabe")
        ]
        }
        )
```

✓ 4.3s

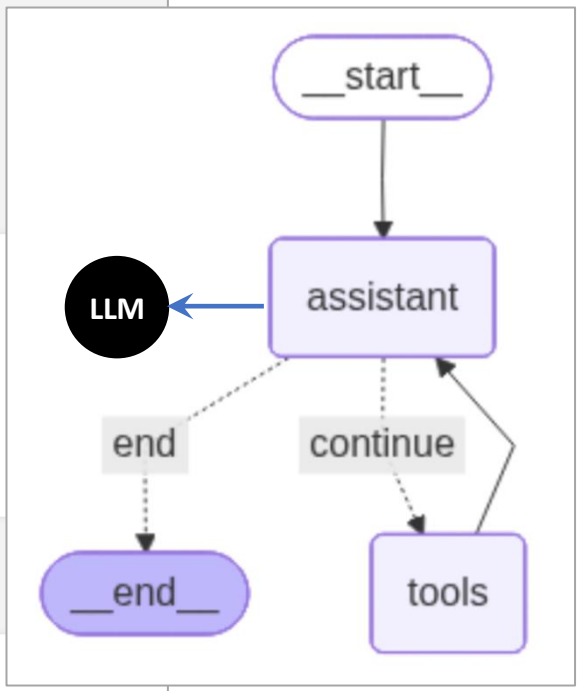
```
Add Numbers Tool Invoked for inputs:  a = 5.0, b=80.0
Multiply Numbers Tool Invoked for inputs:  a = 85.0, b=23.0
```

```
print(resp['messages'][-1].content)
```

✓ 0.0s

```
Le résultat de l'addition de 5 à 80 est 85, et le résultat de la multiplication par 23 est 1955.

Voici un proverbe en arabe : "الصبر مفتاح الفرج" qui signifie "La patience est la clé de la délivrance."
```



Exemple React Agent

```
stream = graph.stream(input={
    "messages":[
        HumanMessage("Add 5 to 80 then multiply by 23 et donne moi un proverbe en arabe")
        ]
        }, stream_mode="values")
```

0.0s

```
for item in stream:
    message = item['messages'][-1]
    if(isinstance(message, BaseMessage)):
        message.pretty_print()
```

4.9s

```
================================ Human Message =================================

Add 5 to 80 then multiply by 23 et donne moi un proverbe en arabe
================================== Ai Message ==================================
Tool Calls:
  add_tool (call_8jHt3Feb1QPpQJC73f45viC4)
 Call ID: call_8jHt3Feb1QPpQJC73f45viC4
  Args:
    a: 5
    b: 80
  multiply_tool (call_KhZ4utmbXmcLokgJ4mhLC9ZP)
 Call ID: call_KhZ4utmbXmcLokgJ4mhLC9ZP
  Args:
    a: 85
    b: 23

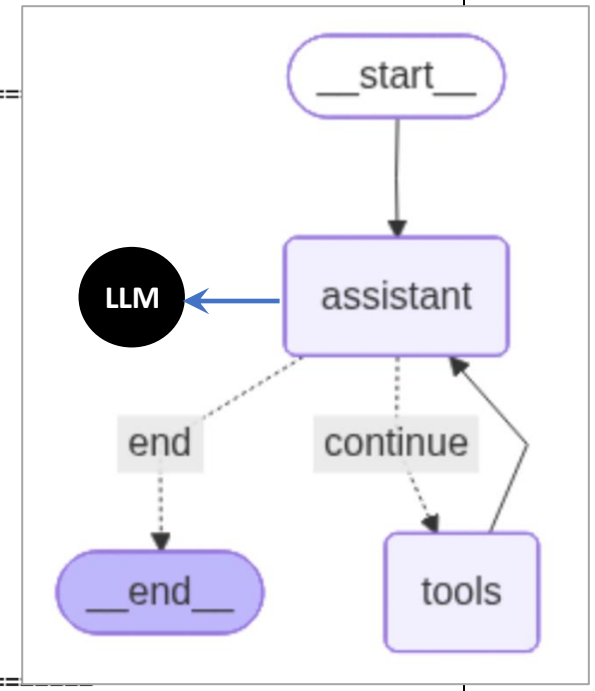
Add Numbers Tool Invoked for inputs:  a = 5.0, b=80.0

Multiply Numbers Tool Invoked for inputs:  a = 85.0, b=23.0
================================= Tool Message =================================
Name: multiply_tool

1955.0
================================== Ai Message ==================================

Le résultat de l'addition de 5 à 80 est 85, et le résultat de la multiplication par 23 est 1955.

Voici un proverbe en arabe : "الصبر مفتاح الفرج" qui signifie "La patience est la clé de la délivrance."
```

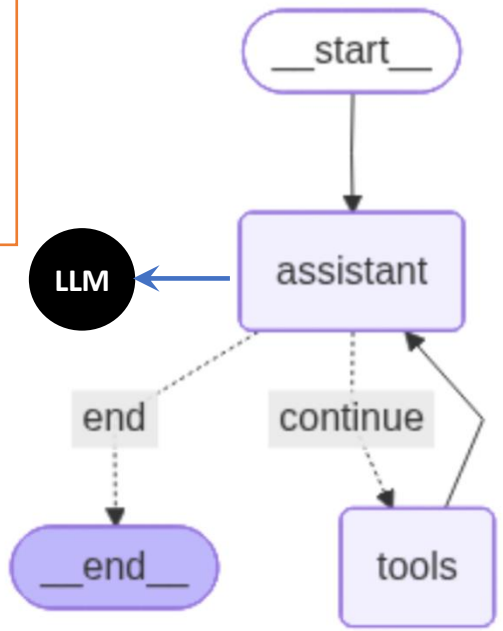


Exemple React Agent

Run The workflow

```
inputs = {
   "messages":[
      HumanMessage("""Add 6 to 7 then divide the result by 2.
                Donne moi un proverbe""")
         ]
      }
stream = graph.stream(inputs, stream_mode="values")
```

```
for s in stream:
   message = s['messages'][-1]
   if isinstance(message, tuple):
      print(message)
   else:
      message.pretty_print()
```




```
====================== Human Message ======================

Add 6 to 7 then divide the result by 2. Donne moi un proverbe
====================== Ai Message ==========================
Tool Calls:
  add (call_oBi0jRG9xnRsTHPvmUcdGMQJ)
 Call ID: call_oBi0jRG9xnRsTHPvmUcdGMQJ
  Args:
    a: 6
    b: 7
  divide (call_sLqTbTDqySHFHxmwxxZsQkAB)
 Call ID: call_sLqTbTDqySHFHxmwxxZsQkAB
  Args:
    a: 13
    b: 2
Adding 2 numbers a=6.0, b=7.0

Dividing 2 numbers a=13.0, b=2.0

======================Tool Message ======================
Name: divide

6.5
======================Ai Message ======================

Le résultat de l'addition de 6 et 7 est 13, et en divisant par 2, on obtient 6,5.

Proverbe : "Petit à petit, l'oiseau fait son nid."
……..……..……..
```

Exemple React With Memory and tools

Run The workflow

```
from langgraph.checkpoint.memory import InMemorySaver
```

```
memory = InMemorySaver()
graph_with_memory = workflow.compile(checkpointer=memory)
```

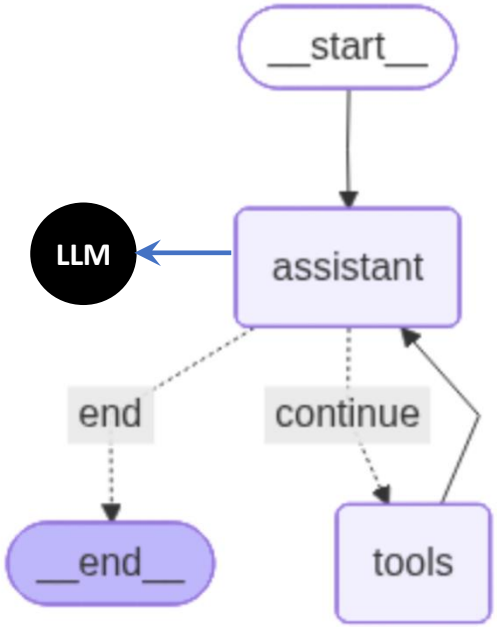
```
config = {"configurable":{"thread_id":"1"}}
result = graph_with_memory.invoke(
   {"messages": [HumanMessage(content="Mon nom est Mohamed")]},
   config=config
)
print(result['messages'][-1].content)
```

Bonjour Mohamed ! Comment puis-je vous aider aujourd'hui ?

```
result = graph_with_memory.invoke(
   {"messages": [HumanMessage(content="C'est quoi mon nom?")]},
   config=config
)
print(result['messages'][-1].content)
```

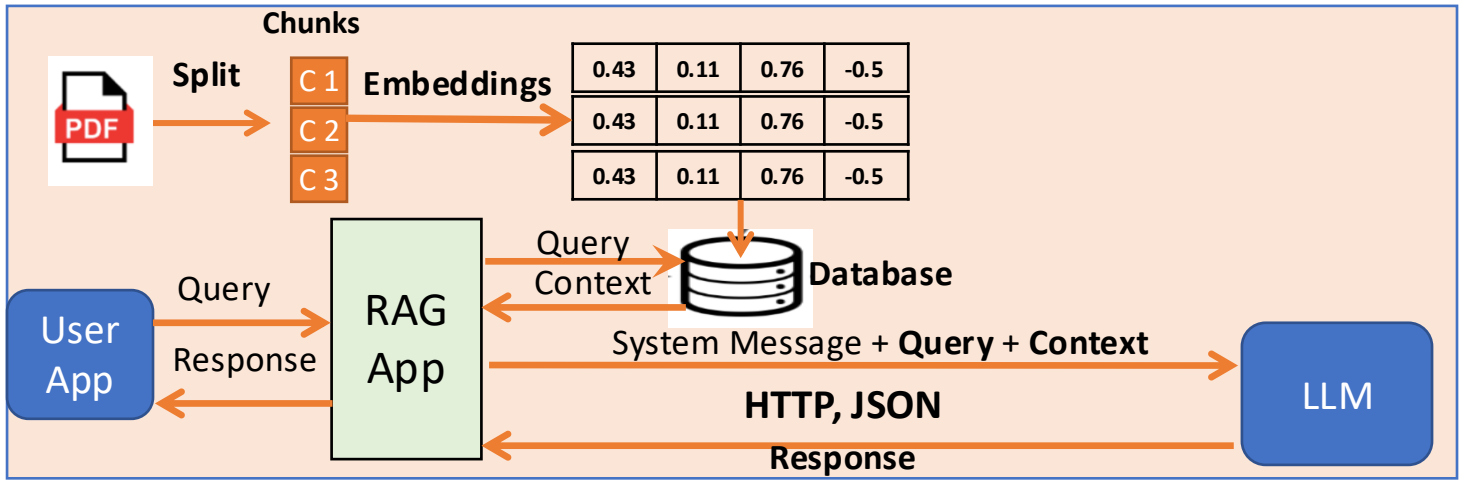
Votre nom est Mohamed.

```
current_state = graph_with_memory.get_state(config).values['messages']
current_state
```

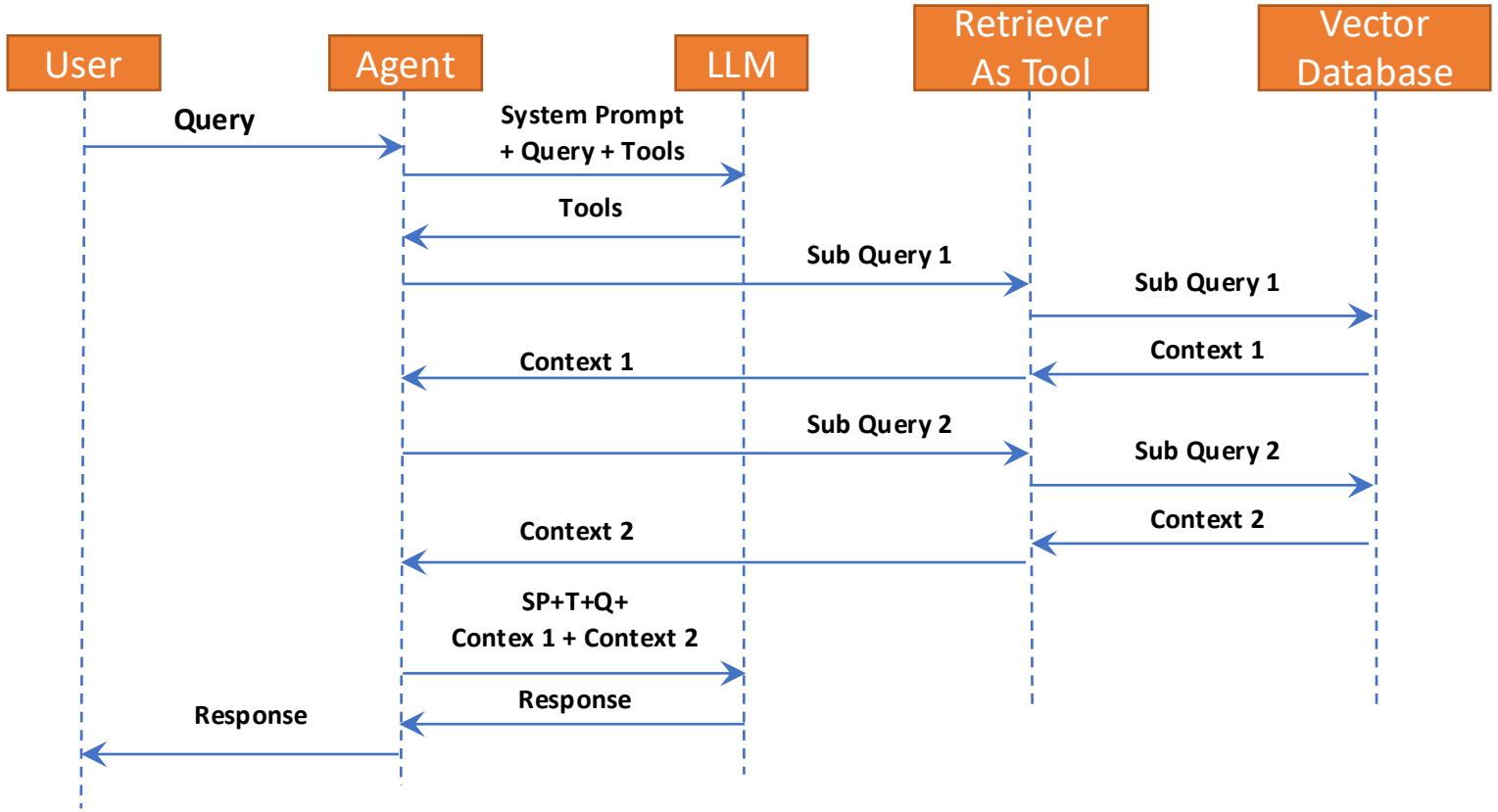


Agentic RAG

Classic RAG



Agentic RAG



Agentic RAG

Import

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import PyPDFDirectoryLoader
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings
from langchain_openai import ChatOpenAI
from IPython.display import Markdown
import os
```

Agentic RAG

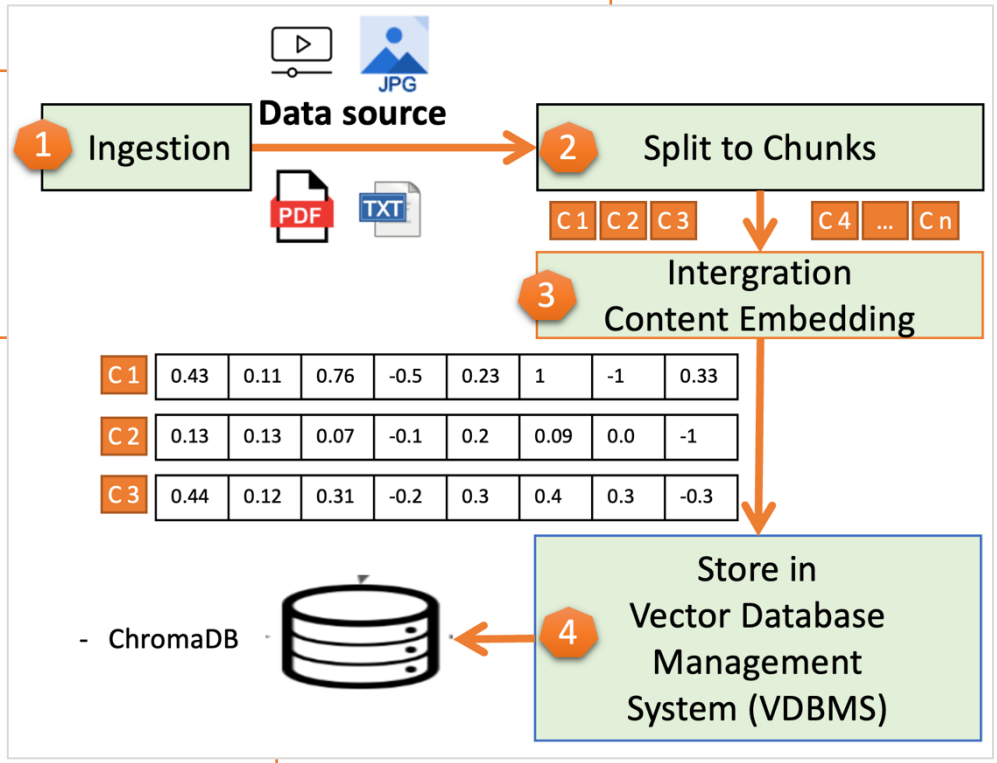
Load and Split Documents

```
from langchain_community.document_loaders import PyPDFDirectoryLoader
# Path of all pdfs
pdf_paths = [f"./pdfs/{file}" for file in os.listdir("./pdfs")]
pdf_paths
```

```
['./pdfs/these_doctorat.pdf']
```

```
# Creating  PDF loader object
loader = PyPDFDirectoryLoader(path="./pdfs/")
```

```
from langchain_text_splitters
import RecursiveCharacterTextSplitter
# Split Documents into chunks
text_splitter = RecursiveCharacterTextSplitter
  .from_tiktoken_encoder(
   encoding_name="cl100k_base",
   chunk_size=1000,
   chunk_overlap=200
)
# Load and split documents
chunks = loader.load_and_split(text_splitter=text_splitter)
```



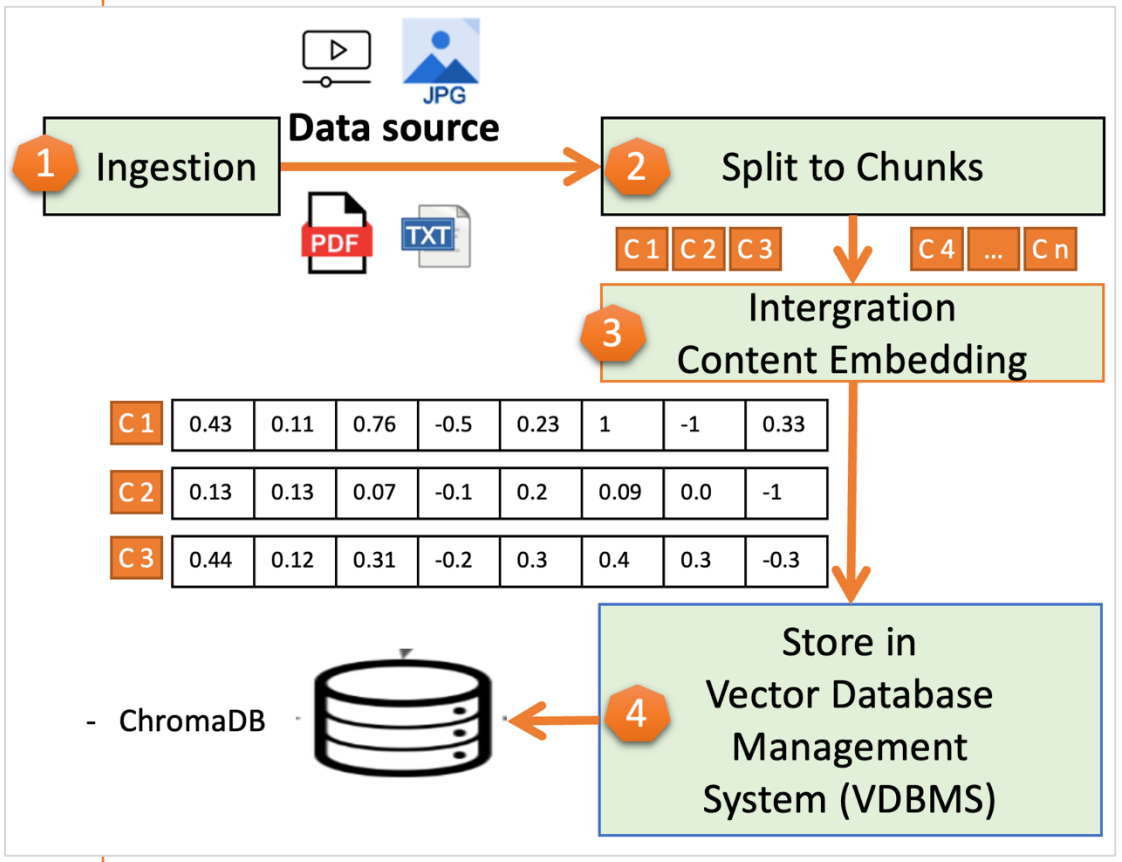
Agentic RAG

Step 4: Create Vector Store with Embeddings

```
### Step 4: Create Vector Store with Embeddings

from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma

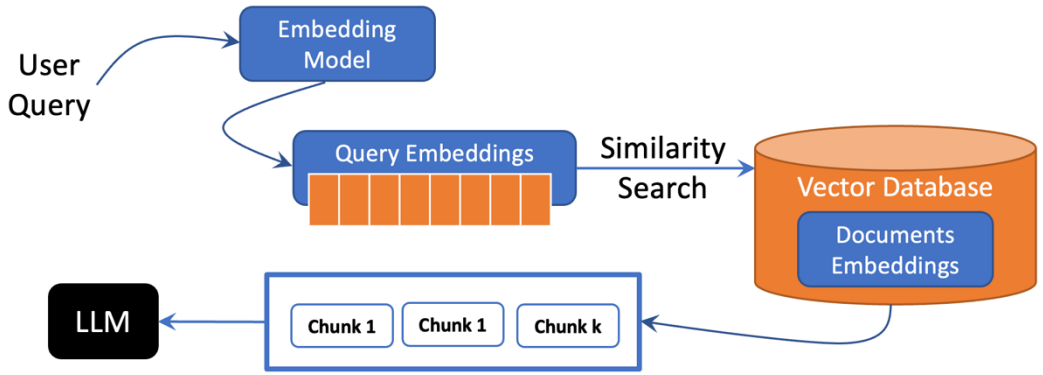
# Initialize OpenAI embedding model
embedding_model = OpenAIEmbeddings(
  model="text-embedding-ada-002”
)
# Create vector store from documents
vectorstore = Chroma.from_documents(
    documents=ai_chunks,
    embedding=embedding_model,
    collection_name=”these_doctorat"
)
# Create retriever for similarity search
# which fetches 10 relevant chunks
retriever = vectorstore.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 10}
)
```



Agentic RAG

Step 5: Test RAG Retrieval (Manual)

```
### Step 5: Test RAG Retrieval (Manual)
# Test query
test_query = "Sujet de thèse et Auteur de cette thèse de doctorat"
print(f"🔍 Test Query: {test_query}")
print("="*50)
# Retrieve relevant documents
relevant_docs = retriever.invoke(test_query)
print(f"\n Retrieved {len(relevant_docs)} relevant chunks:")
# Display top 3 results
for i, doc in enumerate(relevant_docs[:3], 1):
    print(f"\n Result {i}:")
    print(f"\n - Source: {doc.metadata.get('source', 'Unknown')}")
    print(f"\n - Page: {doc.metadata.get('page', 'Unknown')}")
    print(f"\n - Content: {doc.page_content[:100]}...")
    print("-"*50 + "\n")
```




```
🔍 Test Query: Sujet de thèse et Auteur de cette thèse de doctorat
==================================================
08:40:38 [INFO] HTTP Request: POST https://api.openai.com/v1/embeddings "HTTP/1.1 200 OK"
Retrieved 10 relevant chunks:
Result 1:
  - Source: pdfs/these_doctorat.pdf
  - Page: 3
  - Content: ACKNOWLEDGMENTS In this month of October, as the days grow colder and winter draws near, I arrive at...
--------------------------------------------------
Result 2:
  - Source: pdfs/these_doctorat.pdf
  - Page: 161
  - Content: Pasteur, SINERGIES laboratory. His research is focused on artificial intelligence, particularly in o...
--------------------------------------------------
Result 3:
  - Source: pdfs/these_doctorat.pdf
  - Page: 39
  - Content: Publications The research presented in this thesis has been the subject of the following publication...
--------------------------------------------------
```

Agentic RAG

Step 6: Implement `retriever_tool` Tool

```
@tool
def retriever_tool(query: str) -> str:
    """ Permet de chercher des informations sur une thèse de doctorat :
    - Titre de la thèse
    - Membres de jurys
    - Directeurs et co-directeurs de thèse
    - Chapitres
    - Résumé
Args:
    query: Question posée par l'utilisateur
Returns:
    str: Réponse détaillée
    """
    try:
        # Retrieve relevant document chunks
        relevant_document_chunks = retriever.invoke(query)
        context_list = [d.page_content for d in relevant_document_chunks]
        context_for_query = ". ".join(context_list)
        return context_for_query
    except Exception as e:
        return f"Error querying private database: {str(e)}"
```

```
model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
tools = [retriever_tool]
llm_with_tools = model.bind_tools(tools)
```

Agentic RAG

State et System_message

```
import logging
from typing import Annotated, Sequence, Literal
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode
from langchain.messages import SystemMessage, HumanMessage, AIMessage
```

```
# Configure logging to see agent's decision-making process
logging.basicConfig(
  level=logging.INFO,
  format='%(asctime)s [%(levelname)s] %(message)s',
  datefmt='%H:%M:%S'
)
logger = logging.getLogger(__name__)
```

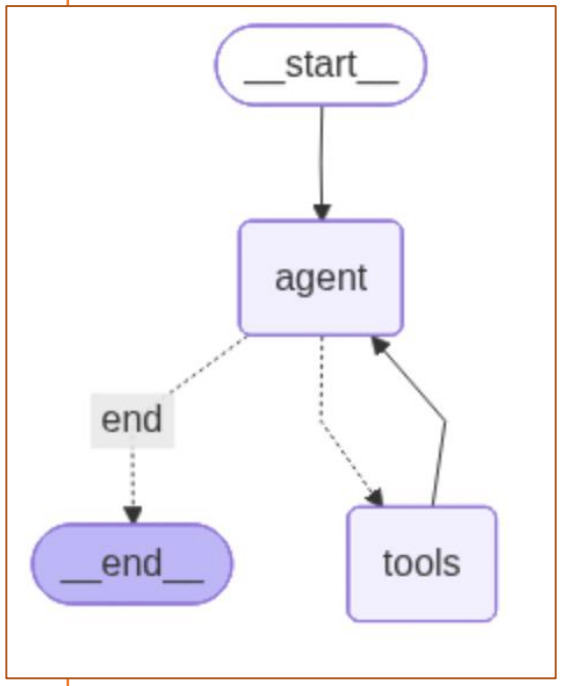
```
class SimpleAgentState(TypedDict):
  messages: Annotated[Sequence, add_messages]
```

```
agentic_rag_system_message ="""
Vous êtes un assistant spécialisé dans la recherche des informations concernant des thèses de doctorat.
Si la réponse ne peut pas être trouvée dans le contexte, répondez par : "Je ne sais pas - cette information n'est pas disponible dans nos rapports d'analystes."
"""
```

Agentic RAG

Agent Node

```
def agent_node(state: SimpleAgentState) -> dict:
  logger.info("AGENT NODE: Processing request...")
   system_msg = SystemMessage(content=agentic_rag_system_message)
   messages = [system_msg] + list(state["messages"])
   logger.info("   Calling LLM with tools...")
   response = llm_with_tools.invoke(messages)
   # Log if agent wants to use tools
   if hasattr(response, 'tool_calls') and response.tool_calls:
      logger.info(f"   ✓ Agent decided to use {len(response.tool_calls)} tool(s)")
      for i, tool_call in enumerate(response.tool_calls, 1):
         logger.info(f"      {i}. {tool_call['name']}")
         logger.info(f" {i}. {tool_call['args']}")
   else:
      logger.info("   ✓ Agent generated final response (no tools needed)")
   return {"messages": [response]}
```

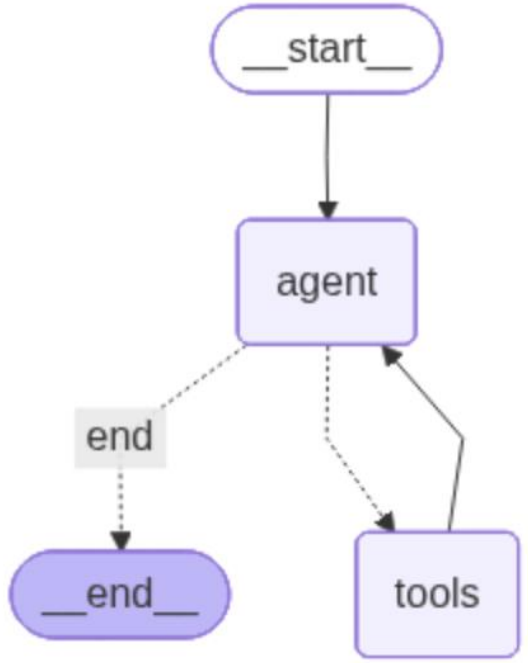


Agentic RAG

Condition de routage

```
def should_continue(state: SimpleAgentState) -> Literal["tools", "end"]:
    """Determines whether to continue to tools or end."""
    last_message = state["messages"][-1]

    if hasattr(last_message, 'tool_calls') and last_message.tool_calls:
        logger.info("🔀 ROUTING: Continuing to TOOLS node")
        return "tools"
    logger.info("ROUTING: Ending workflow (final response ready)")
    return "end"
```



Agentic RAG

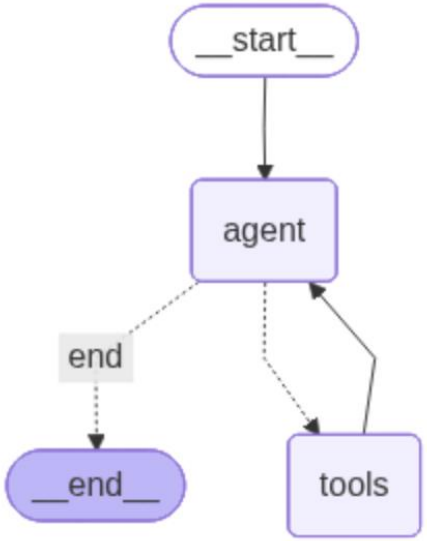
Créer , Compiler et visualiser le Graph

```
# Create workflow
workflow = StateGraph(SimpleAgentState)
# Create tool node with logging
original_tool_node = ToolNode(tools)

def tool_node_with_logging(state):
    logger.info("TOOL NODE: Executing tools...")
    result = original_tool_node.invoke(state)
    logger.info(f"Tools executed successfully")
    return result

# Add nodes to graph
workflow.add_node("agent", agent_node)
workflow.add_node("tools", tool_node_with_logging)
# Set entry point
workflow.set_entry_point("agent")
```

```
# Add conditional edge from agent
workflow.add_conditional_edges(
    "agent",
    should_continue,
    {
        "tools": "tools",
        "end": END
    }
)
# Add edge from tools back to agent
workflow.add_edge("tools", "agent")
graph = workflow.compile()
logger.info("✅ Enhanced agent created successfully\n")
display(Image(graph.get_graph().draw_mermaid_png()))
```

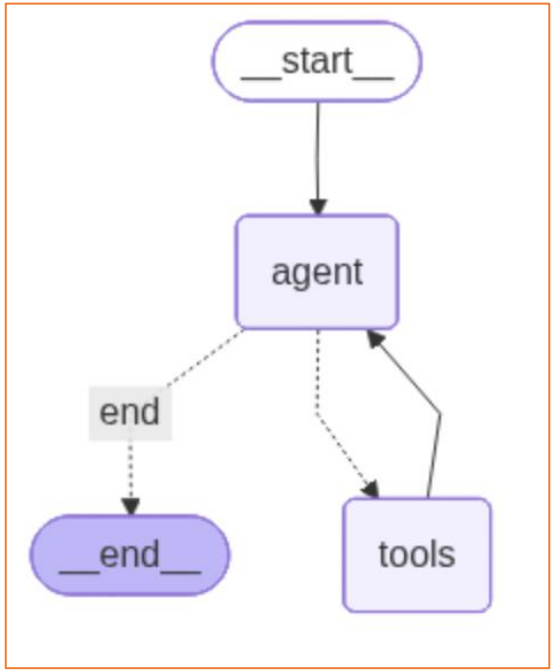


Agentic RAG

Tester l'Agentic RAG

```
resp = graph.invoke({"messages":[
  HumanMessage(""" Sujet de la thèse, auteur, liste des membres de jury et directeurs de la thèse""")
  ]})
```

09:00:45 [INFO] AGENT NODE: Processing request... 09:00:45
[INFO] Calling LLM with tools... 09:00:47
[INFO] HTTP Request: POST https://api.openai.com/v1/chat/completions "HTTP/1.1 200 OK" 09:00:47
[INFO] ✓ Agent decided to use 1 tool(s) 09:00:47
[INFO] 1. retriever_tool 09:00:47
[INFO] 1. {'query': 'Sujet de la thèse, auteur, liste des membres de jury et directeurs de la thèse'} 09:00:47
[INFO] 🔀 ROUTING: Continuing to TOOLS node 09:00:47
[INFO] TOOL NODE: Executing tools... 09:00:49
[INFO] HTTP Request: POST https://api.openai.com/v1/embeddings "HTTP/1.1 200 OK" 09:00:49
[INFO] Tools executed successfully 09:00:49
[INFO] AGENT NODE: Processing request... 09:00:49 [INFO] Calling LLM with tools... 09:00:54
[INFO] HTTP Request: POST https://api.openai.com/v1/chat/completions "HTTP/1.1 200 OK" 09:00:54 [INFO] ✓ Agent generated final response (no tools needed) 09:00:54 [INFO] ROUTING: Ending workflow (final response ready)




```
print(display(Markdown(resp['messages'][-1].content)))
```

Voici les informations concernant la thèse :

- **Sujet de la thèse** : Optimisation de la gestion des ressources préhospitalières et détection des émotions dans les appels d'urgence à l'aide de l'intelligence artificielle.
- **Auteur** : Omar Elfahim
- **Directeurs de thèse** :
  - Pr. Mohamed Youssfi
  - Pr. Oussama Barakat
  - Pr. Mohamed Mestari (co-directeur)
- **Membres du jury** :
  - Professeur Abdelhakim Artiba
  - Professeur Mohamed Kissi
  - Professeur Elhabib Nfaoui (rapporteurs)
  - Professeur Omar Bouattane
  - Professeur Mohamed Embarki (examiners)

Agentic RAG Application

RAG Agentique

```python
from langchain_community.document_loaders import PyPDFLoader
from langchain_openai.embeddings import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma
from dotenv.ipython import load_dotenv
from langchain_text_splitters import RecursiveCharacterTextSplitter
import tiktoken
from langchain.tools import tool
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI
from langchain.messages import HumanMessage

load_dotenv(override=True)
loader = PyPDFLoader("CV Pr Mohamed YOUSSFI V9.pdf")
tokennizer = tiktoken.encoding_for_model("gpt-4o-mini")
splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
    encoding_name=tokennizer.name, chunk_size=300, chunk_overlap=20
)
chunks = loader.load_and_split(splitter)
embedding_model = OpenAIEmbeddings()
vecror_store = Chroma.from_documents(
    documents=chunks, embedding=embedding_model, collection_name="cv_data_collection"
)

retriever = vecror_store.as_retriever(kwargs={"k": 10})
```

```python
@tool
def retriever_tool(query: str) -> str:
    """
    Permet de chercher des informations sur des candidats :
    - Nom, Prénom, Diplômes
    - Expériences
    - Copétences
    """
    relevant_chunks = retriever.invoke(query)
    context_list = [d.page_content for d in relevant_chunks]
    context = ". ".join(context_list)
    return context
@tool
def get_company_infos(company_name: str):
    """
    Consulter des infomrations sur l'entreprise donnée
    """
    return {"company_name": company_name, "domain": "IT", "turnover": 120_870_000}


llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

my_agent = create_agent(
    model=llm,
    tools=[retriever_tool, get_company_infos],
    system_prompt="Répond à la question de l'utilisateur en utilisant les tools fournis",
)
```

Agentic RAG : Tester avec LangGraph Studio

Exemplede RAG Agentique

Langgraph.json

```
{
  "dependencies": ["langchain_openai", "./"],
  "graphs": {
    "agent_rag_graph": "./app.py:my_agent"
  },
  "env": "./.env"
}
```

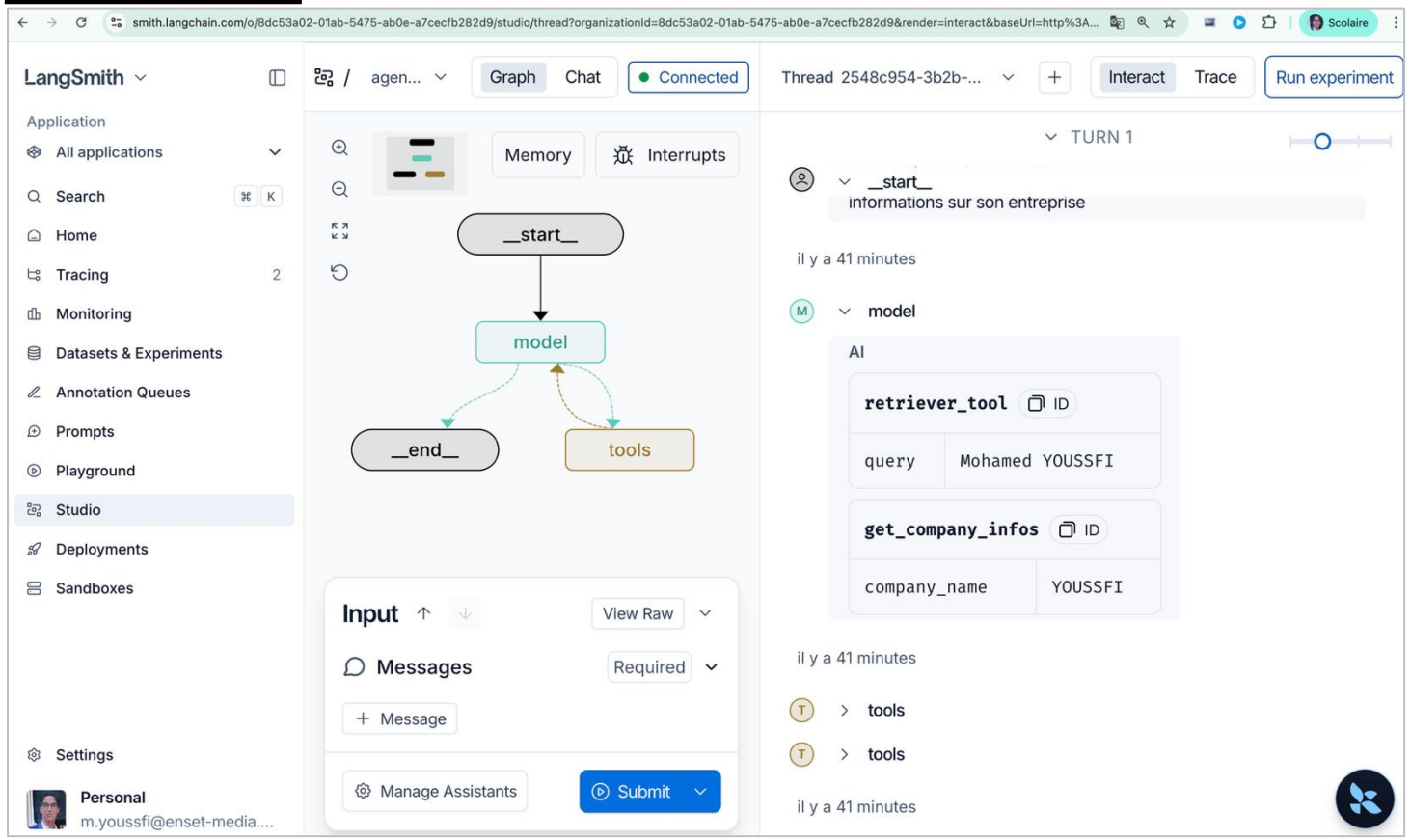
Project.toml

```
[project]
name = "ag-rag-lg"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.13"
dependencies = [
    "chromadb>=1.5.7",
    "ipykernel>=7.2.0",
    "langchain>=1.2.15",
    "langchain-community>=0.4.1",
    "langchain-openai>=1.1.12",
    "langchain-text-splitters>=1.1.1",
    "langgraph>=1.1.6",
    "langgraph-cli[inmem]>=0.4.21",
    "pypdf>=6.9.2",
    "python-dotenv>=1.2.2",
    "tiktoken>=0.12.0",
]
```

.env

OPENAI_API_KEY=sk-proj-.....
LANGSMITH_API_KEY=lsv2_pt_.....

$ langgraph dev



Agentic RAG : Tester avec LangGraph Studio

Exemplede RAG Agentique

Langgraph.json

```
{
  "dependencies": ["langchain_openai", "./"],
  "graphs": {
    "agent_rag_graph": "./app.py:my_agent"
  },
  "env": "./.env"
}
```

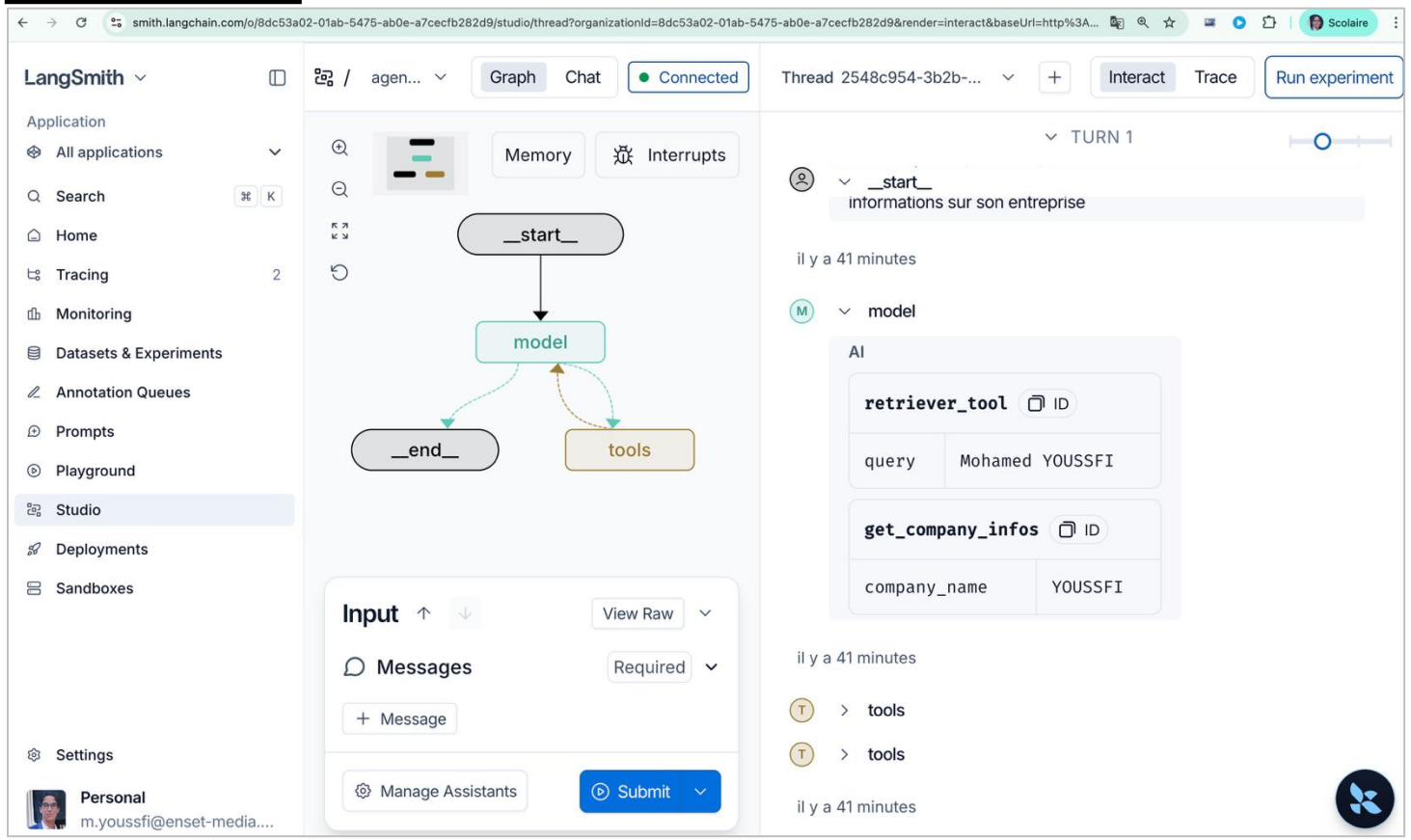
Project.toml

```
[project]
name = "ag-rag-lg"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.13"
dependencies = [
    "chromadb>=1.5.7",
    "ipykernel>=7.2.0",
    "langchain>=1.2.15",
    "langchain-community>=0.4.1",
    "langchain-openai>=1.1.12",
    "langchain-text-splitters>=1.1.1",
    "langgraph>=1.1.6",
    "langgraph-cli[inmem]>=0.4.21",
    "pypdf>=6.9.2",
    "python-dotenv>=1.2.2",
    "tiktoken>=0.12.0",
]
```

.env

OPENAI_API_KEY=sk-proj-.....
LANGSMITH_API_KEY=lsv2_pt_.....

$ langgraph dev

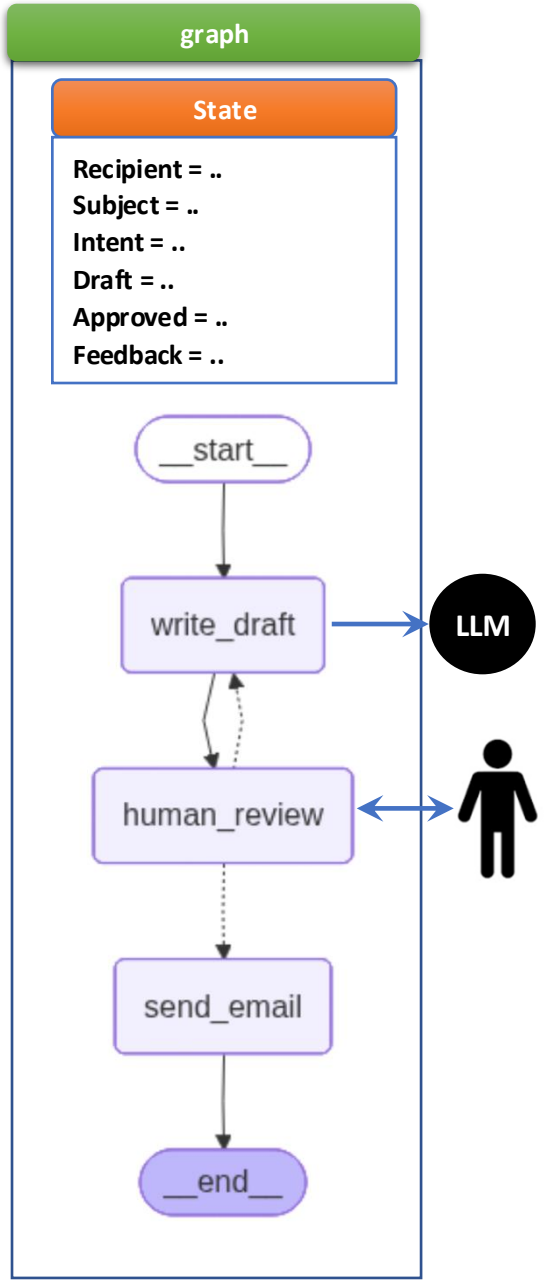


Human In The Loop

```
import os
from typing_extensions import TypedDict, Literal
from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, START, END
from langgraph.types import interrupt, Command
from langgraph.checkpoint.memory import InMemorySaver
```

```
class EmailState(TypedDict, total=False):
    recipient: str
    subject: str
    intent: str
    draft: str
    approved: bool
    feedback: str
```

```
llm = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0.4,
    api_key=os.environ["OPENAI_API_KEY"],
)
```



Human In The Loop

Construction du graph

```
builder = StateGraph(EmailState)

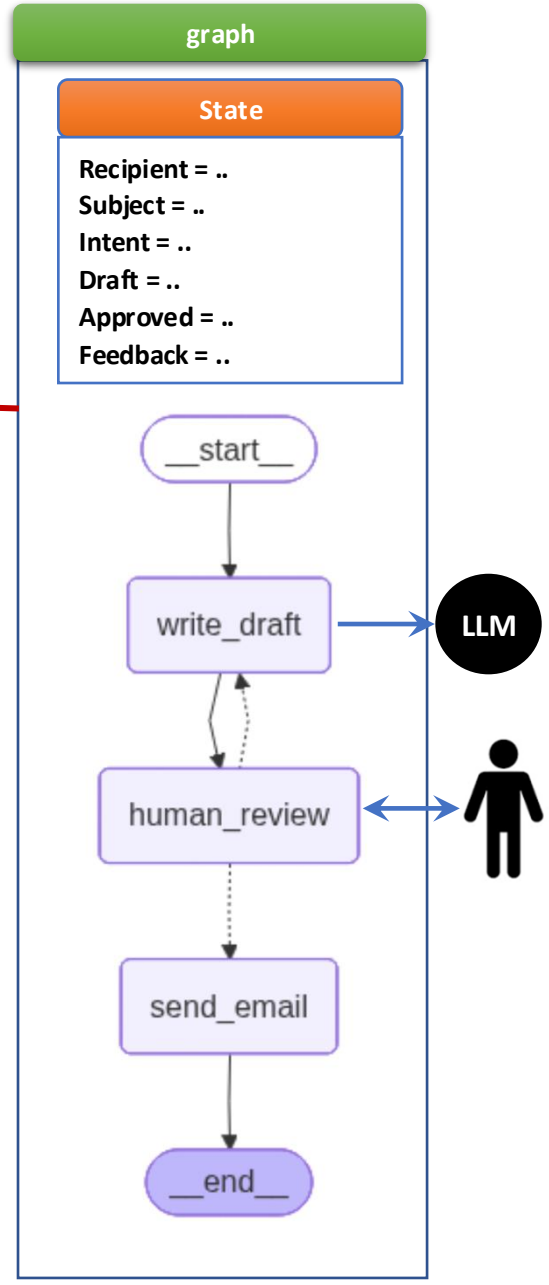
builder.add_node("write_draft", write_draft)
builder.add_node("human_review", human_review)
builder.add_node("send_email", send_email)

builder.add_edge(START, "write_draft")
builder.add_edge("write_draft", "human_review")
builder.add_conditional_edges("human_review", route_after_review)
builder.add_edge("send_email", END)

graph = builder.compile(checkpointer=InMemorySaver())
display(Image(graph.get_graph().draw_mermaid_png()))
```

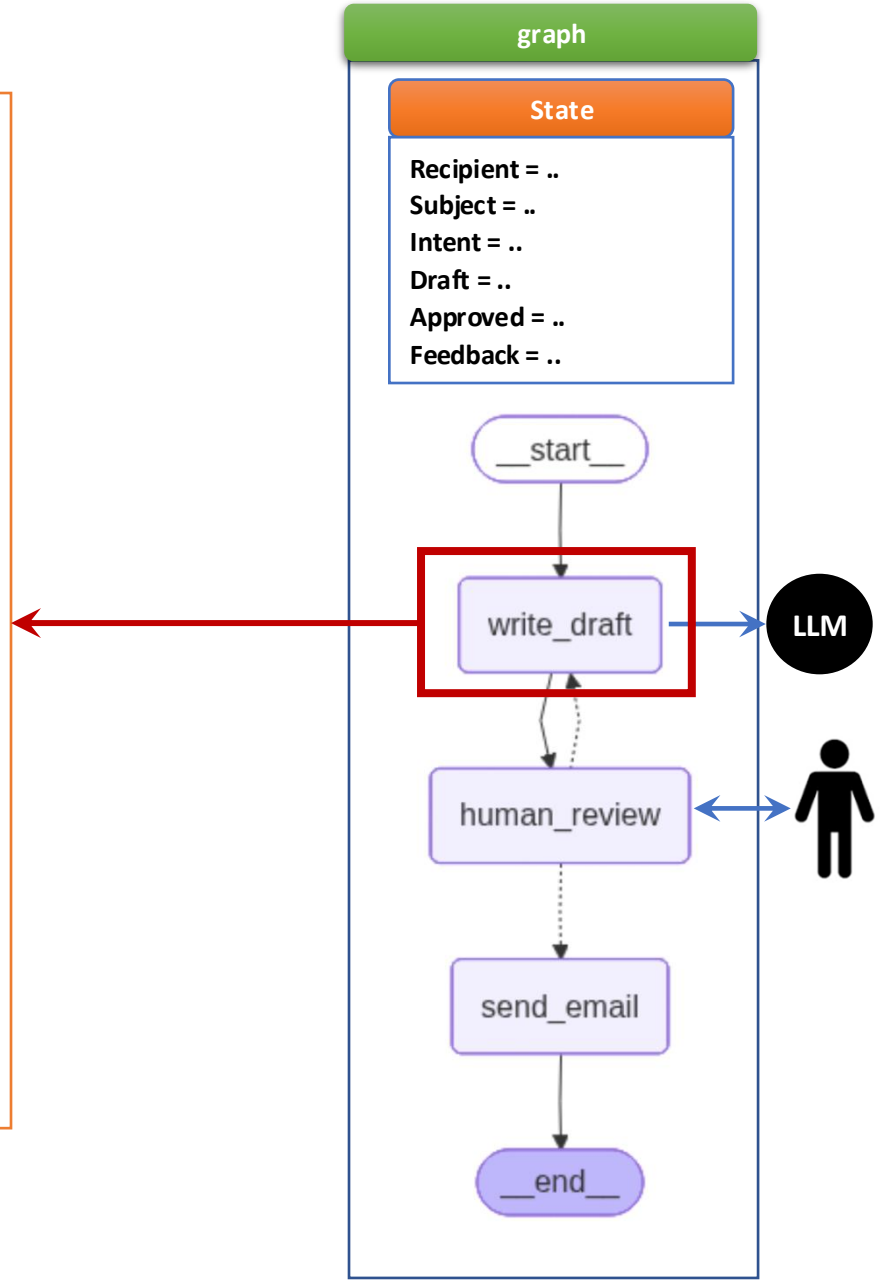
Exécution du graph

```
config = {"configurable": {"thread_id": "email-demo-1"}}
initial_input = {
    "recipient": "client@entreprise.com",
    "subject": "Proposition de rendez-vous",
    "intent": "Écrire un email au format markdown pour proposer un rendez-vous la semaine prochaine afin de présenter notre offre.",
}
result = graph.invoke(initial_input, config=config)
```



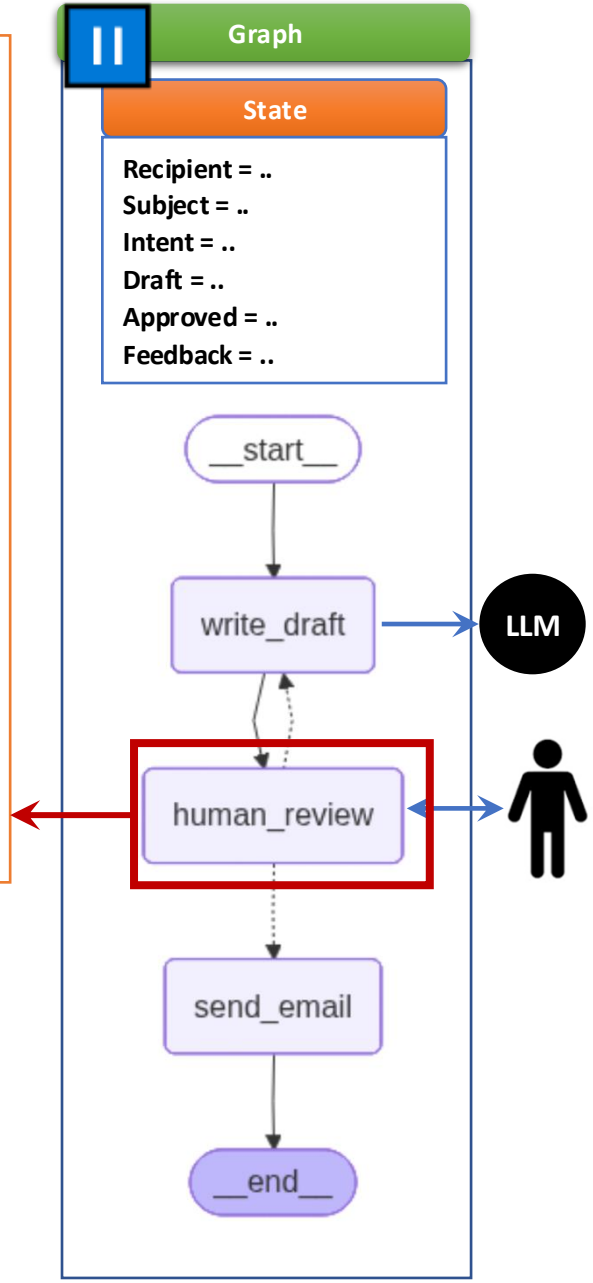
Human In The Loop

```
def write_draft(state: EmailState):
    feedback = state.get("feedback", "")
    previous_draft = state.get("draft", "")
    prompt = f"""
Tu es un assistant qui rédige des emails professionnels en français.
Destinataire: {state["recipient"]}
Sujet: {state["subject"]}
Objectif: {state["intent"]}
Brouillon précédent:
{previous_draft if previous_draft else "(aucun)"}
Commentaire humain:
{feedback if feedback else "Aucun commentaire. Rédige un premier brouillon."}
Instructions:
- Rédige un email clair, poli et concis
- Si un commentaire humain est fourni, réécris le brouillon en en tenant compte
- Retourne uniquement le corps de l'email
"""
    response = llm.invoke(prompt)
    return {"draft": response.content}
```



Human In The Loop

```python
def human_review(state: EmailState):
    review = interrupt({
        "type": "human_review",
        "message": "Merci de valider ou rejeter le brouillon",
        "draft": state["draft"],
        "expected_response": {
            "approved": "bool",
            "feedback": "str (optionnel, utile si rejet)"
        }
    })
    return {
        "approved": review["approved"],
        "feedback": review.get("feedback", "")
    }
```

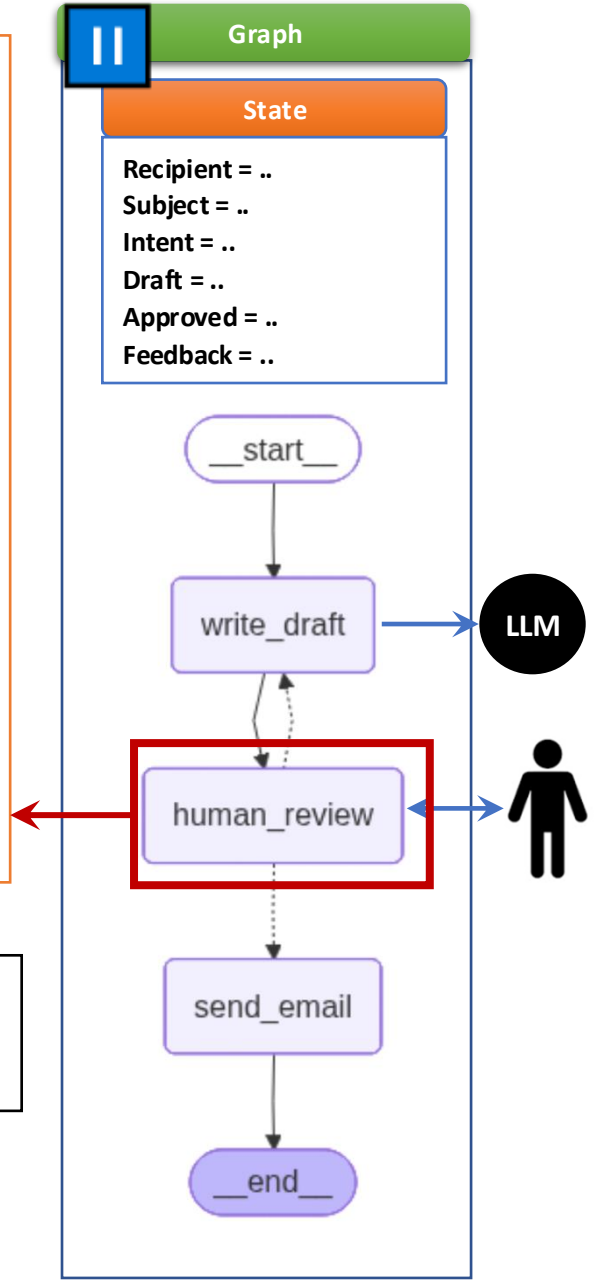


Human In The Loop

```python
def human_review(state: EmailState):
    review = interrupt({
        "type": "human_review",
        "message": "Merci de valider ou rejeter le brouillon",
        "draft": state["draft"],
        "expected_response": {
            "approved": "bool",
            "feedback": "str (optionnel, utile si rejet)"
        }
    })
    return {
        "approved": review["approved"],
        "feedback": review.get("feedback", "")
    }
```

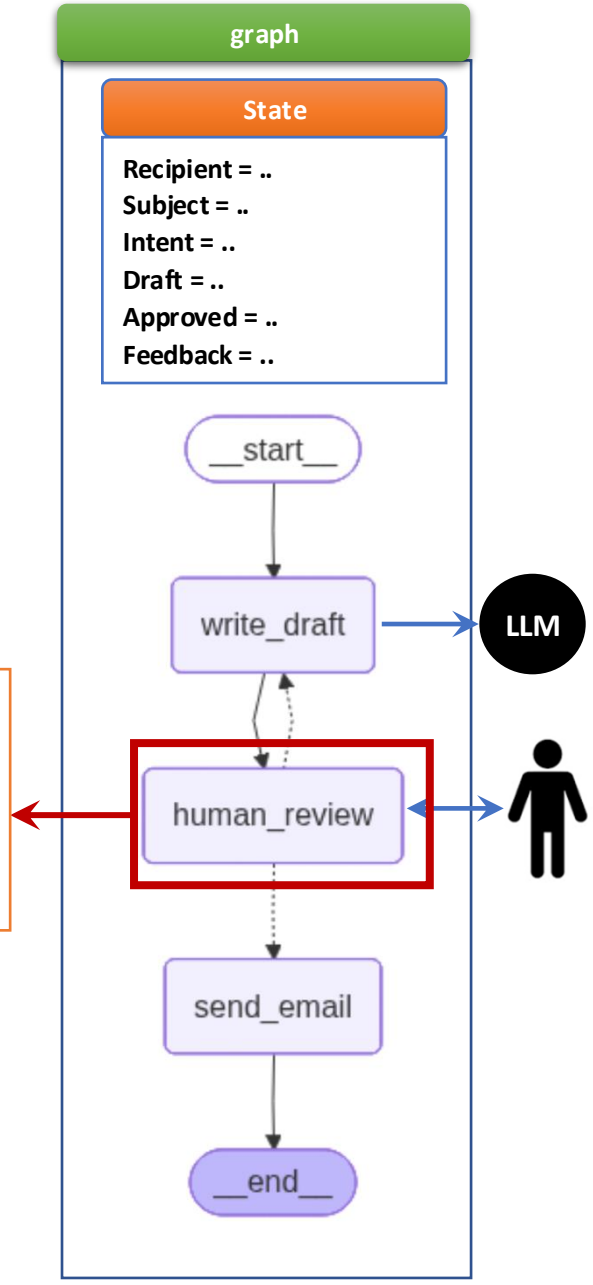


```python
result = graph.invoke(
  Command(resume={"approved": False, "feedback": "..."}),config=config
 )
```



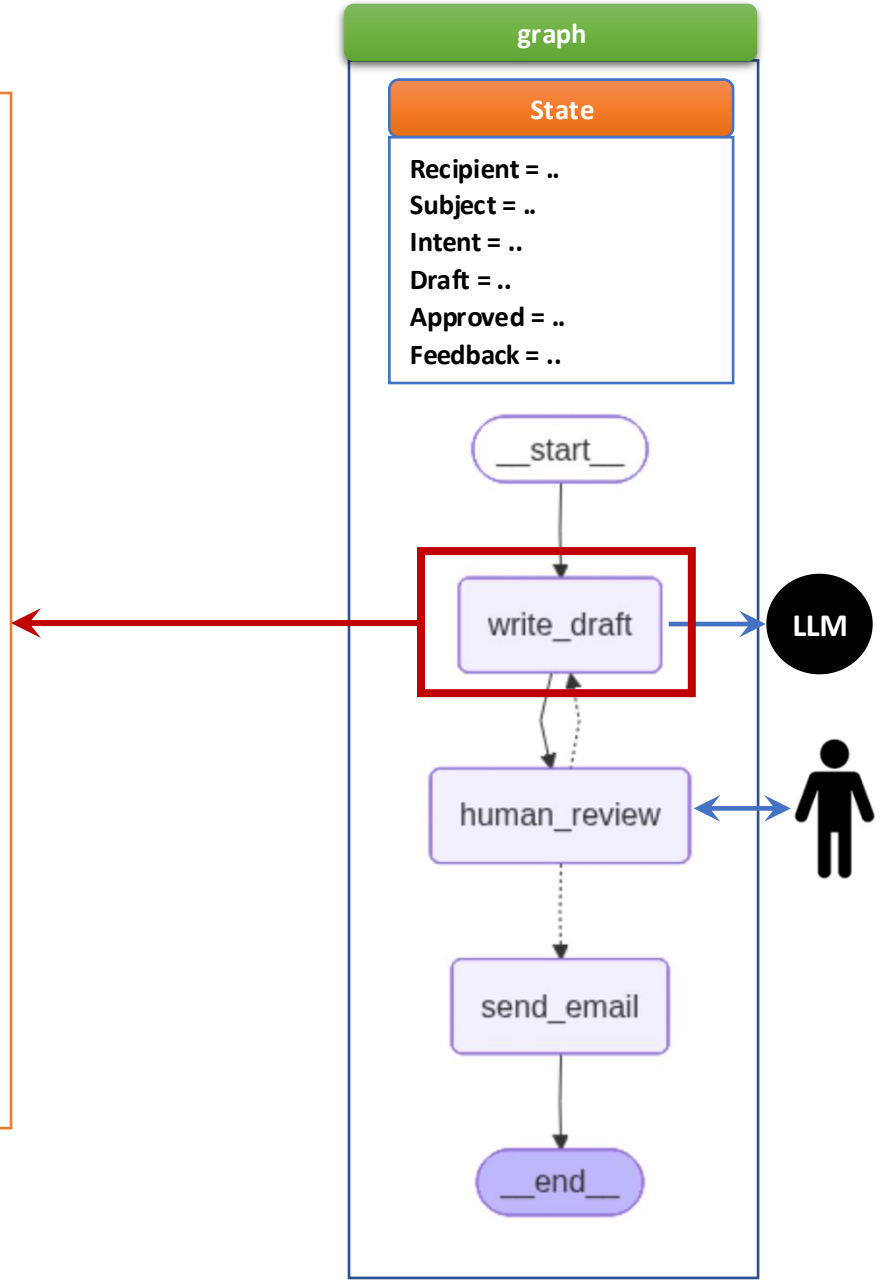
Human In The Loop

```python
def route_after_review(state: EmailState) -> Literal["send_email", "write_draft"]:
    if state["approved"]:
        return "send_email"
    return "write_draft"
```



Human In The Loop

```
def write_draft(state: EmailState):
    feedback = state.get("feedback", "")
    previous_draft = state.get("draft", "")
    prompt = f"""
Tu es un assistant qui rédige des emails professionnels en français.
Destinataire: {state["recipient"]}
Sujet: {state["subject"]}
Objectif: {state["intent"]}
Brouillon précédent:
{previous_draft if previous_draft else "(aucun)"}
Commentaire humain:
{feedback if feedback else "Aucun commentaire. Rédige un premier brouillon."}
Instructions:
- Rédige un email clair, poli et concis
- Si un commentaire humain est fourni, réécris le brouillon en en tenant compte
- Retourne uniquement le corps de l'email
"""
    response = llm.invoke(prompt)
    return {"draft": response.content}
```

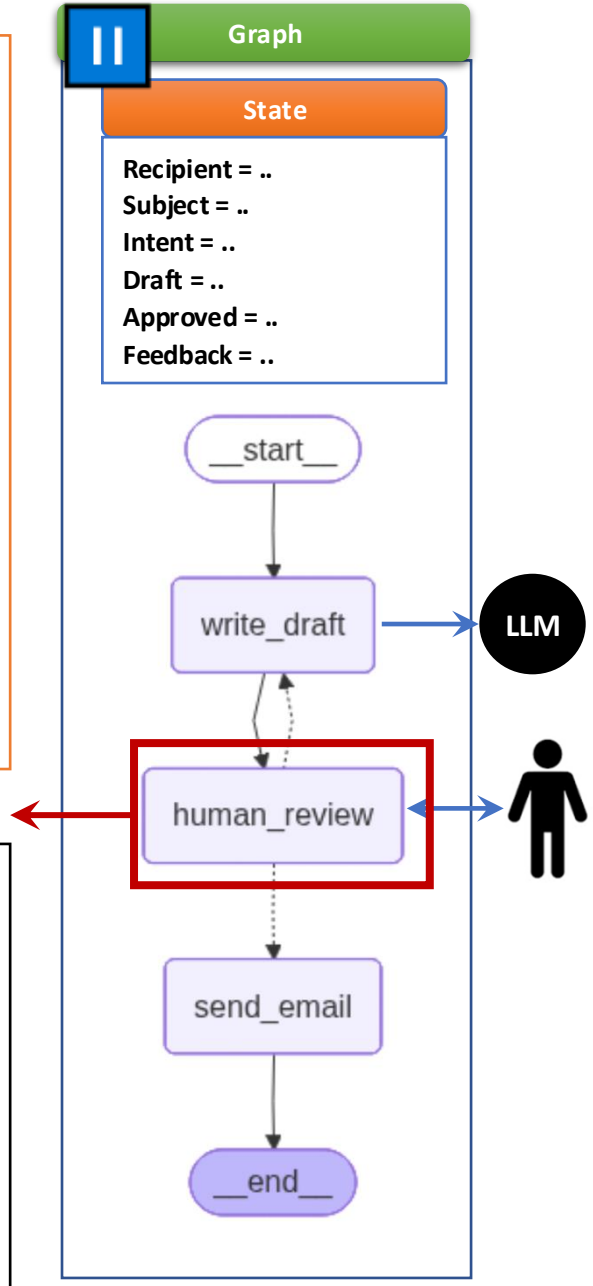


Human In The Loop

```
def human_review(state: EmailState):
    review = interrupt({
        "type": "human_review",
        "message": "Merci de valider ou rejeter le brouillon",
        "draft": state["draft"],
        "expected_response": {
            "approved": "bool",
            "feedback": "str (optionnel, utile si rejet)"
        }
    })
    return {
        "approved": review["approved"],
        "feedback": review.get("feedback", "")
    }
```

```
print(result)
```

```
{
 'recipient': 'client@entreprise.com', 'subject': 'Proposition de rendez-vous',
 'intent': 'Écrire un email au format….',
 'draft': "Bonjour,\n\nJ'espère que vous allez …",
 '__interrupt__': [ Interrupt(
     value={'type': 'human_review',
          'message': 'Merci de valider ou rejeter le brouillon',
          'draft': "Bonjour, ……",
          'expected_response': {
                        'approved': 'bool',
                        'feedback': 'str (optionnel, utile si rejet)'}}, id='1192a863d6a10fb47415d89151458412')]}
```

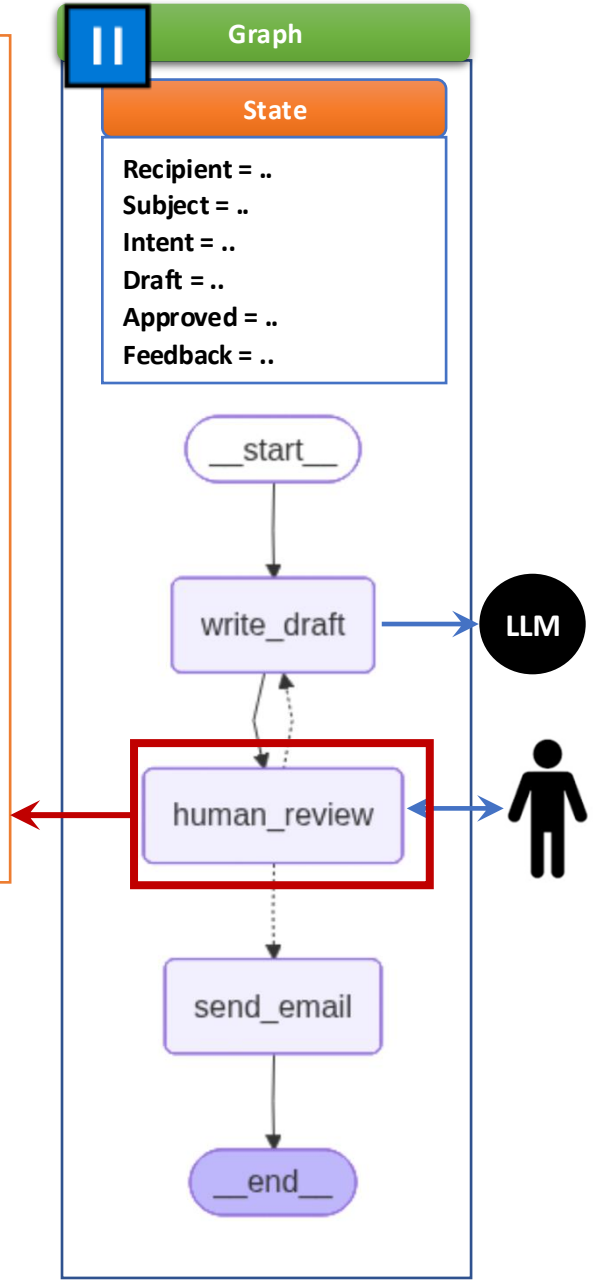


Human In The Loop

```python
def human_review(state: EmailState):
    review = interrupt({
        "type": "human_review",
        "message": "Merci de valider ou rejeter le brouillon",
        "draft": state["draft"],
        "expected_response": {
            "approved": "bool",
            "feedback": "str (optionnel, utile si rejet)"
        }
    })
    return {
        "approved": review["approved"],
        "feedback": review.get("feedback", "")
    }
```

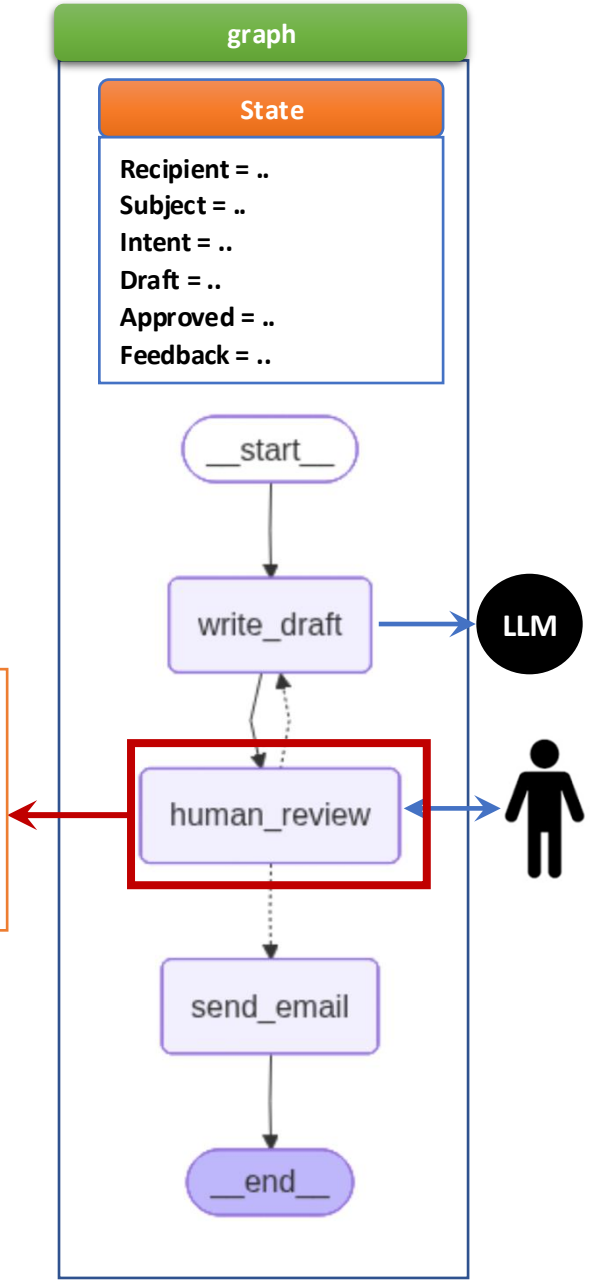


```python
result = graph.invoke(
  Command(resume={"approved": True, "feedback": ""}),config=config
 )
```



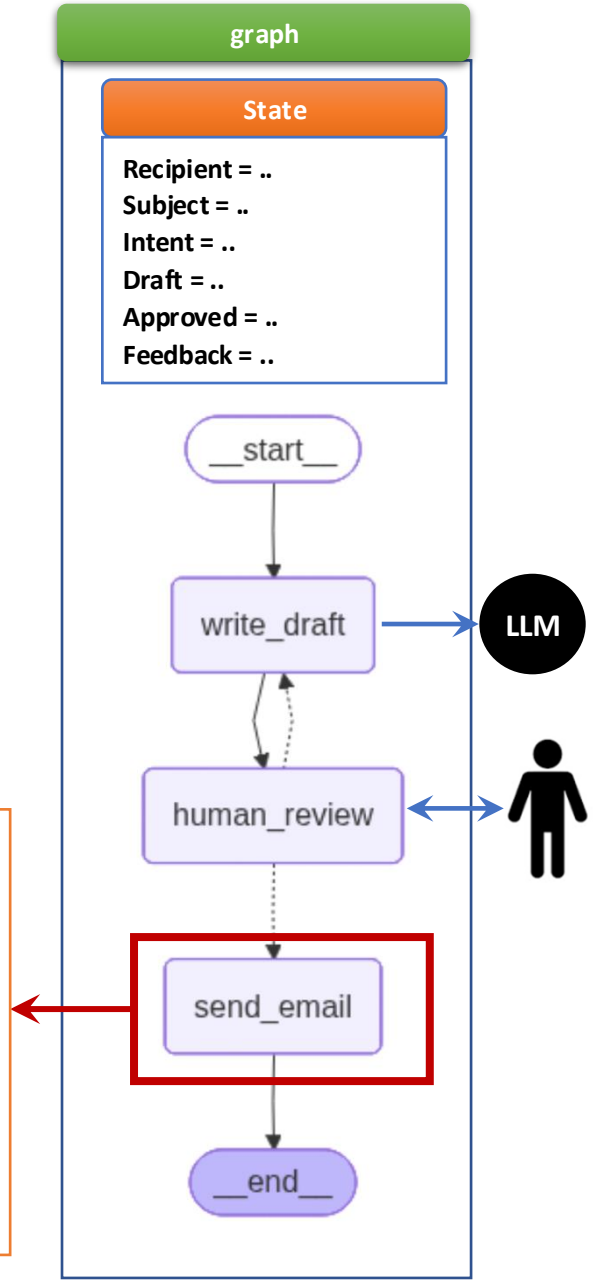
Human In The Loop

```python
def route_after_review(state: EmailState) -> Literal["send_email", "write_draft"]:
    if state["approved"]:
        return "send_email"
    return "write_draft"
```



Human In The Loop

```python
def send_email(state: EmailState):
    print("\n=== EMAIL ENVOYÉ ===")
    print(f"À      : {state['recipient']}")
    print(f"Sujet  : {state['subject']}")
    print("Corps  :")
    print(state["draft"])
    print("====================\n")
    return {}
```



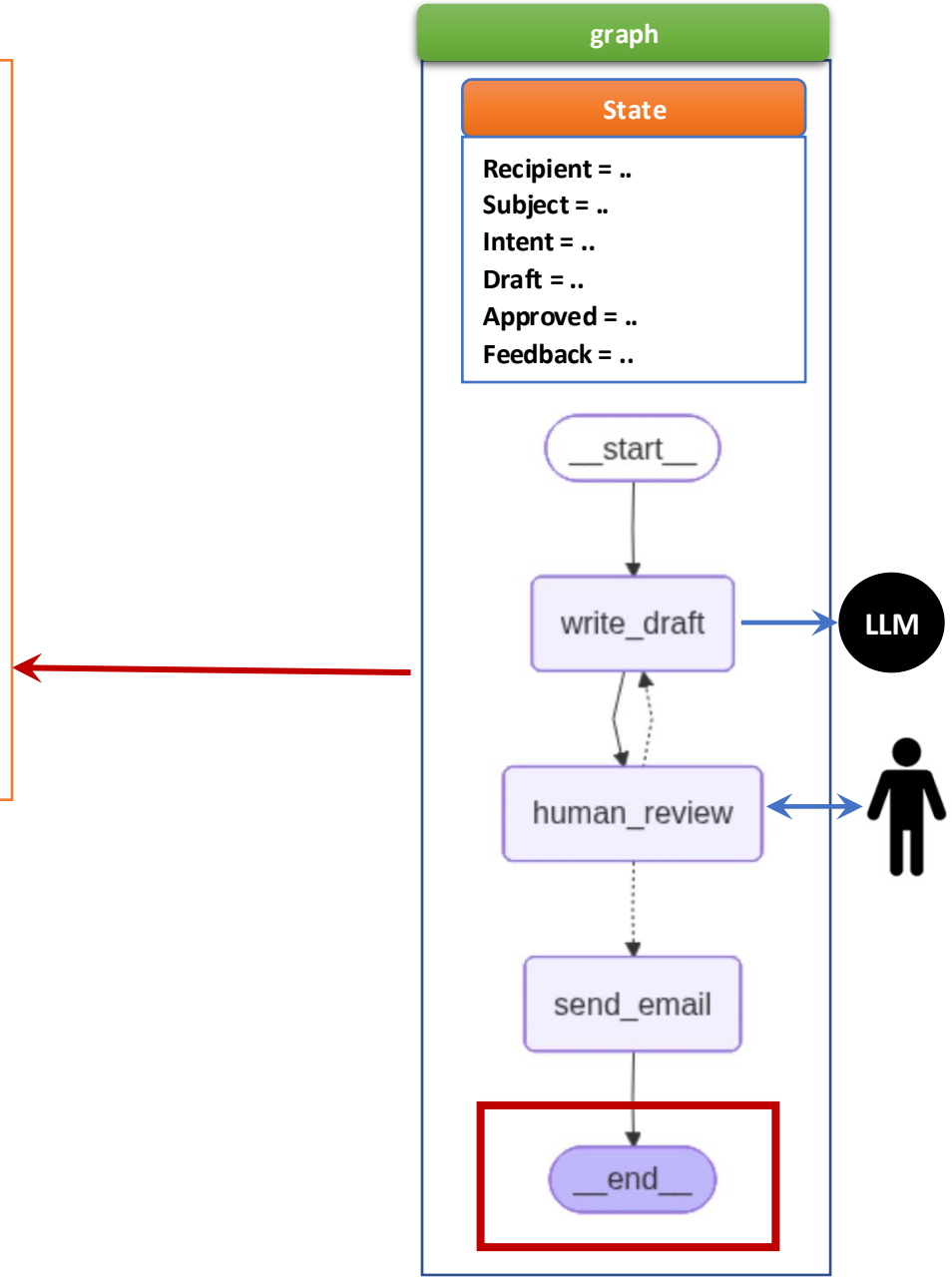
Human In The Loop

```
builder = StateGraph(EmailState)

builder.add_node("write_draft", write_draft)
builder.add_node("human_review", human_review)
builder.add_node("send_email", send_email)

builder.add_edge(START, "write_draft")
builder.add_edge("write_draft", "human_review")
builder.add_conditional_edges("human_review", route_after_review)
builder.add_edge("send_email", END)

graph = builder.compile(checkpointer=InMemorySaver())
display(Image(graph.get_graph().draw_mermaid_png()))
```



Human In The Loop

```
--- BROUILLON À VALIDER ---
```

Bonjour,

J'espère que vous allez bien.

Je me permets de vous contacter pour vous proposer un rendez-vous la semaine prochaine afin de vous prése
peuvent vous être bénéfiques.

Pourriez-vous me faire savoir vos disponibilités ? Je suis flexible et peux m'adapter à votre emploi du temps.

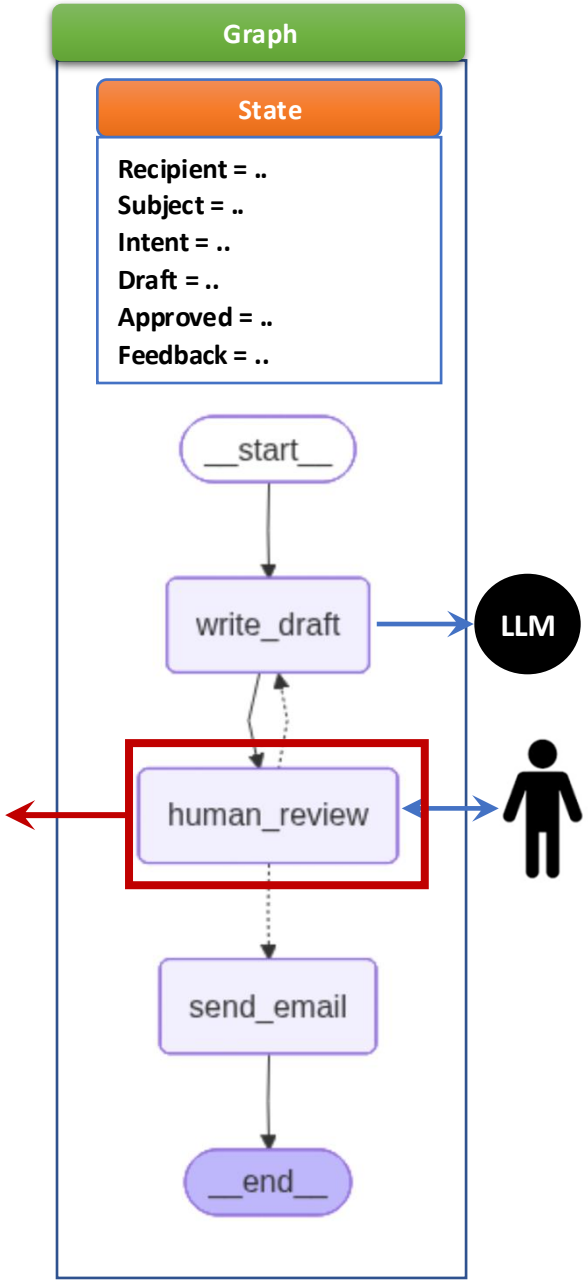
Dans l'attente de votre retour, je vous souhaite une excellente journée.

Cordialement,

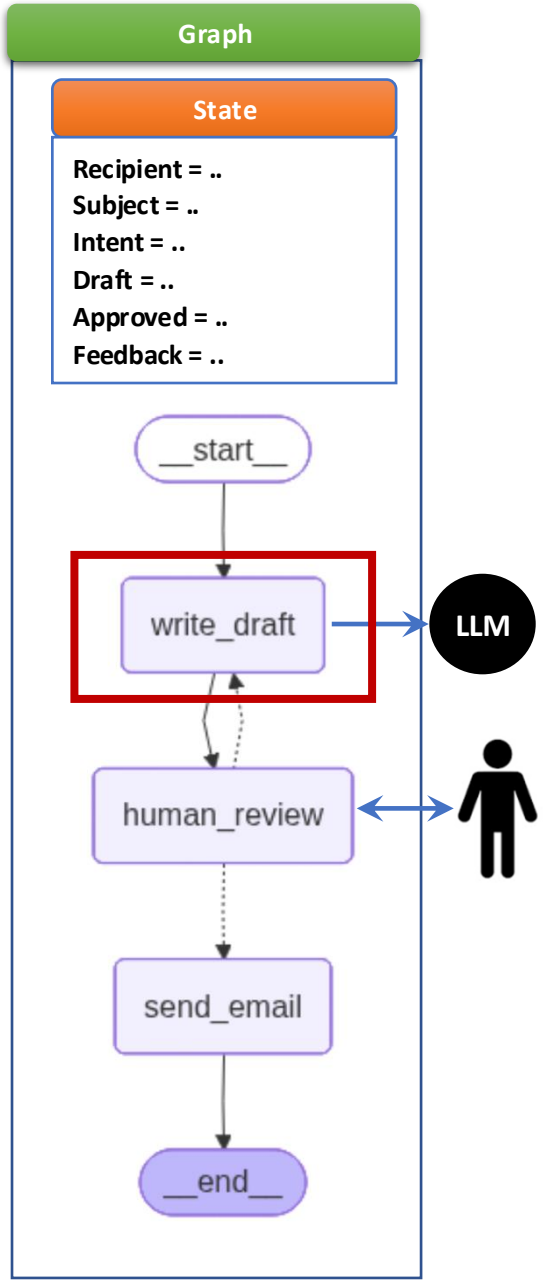
[Votre Nom]
[Votre Poste]
[Votre Entreprise]
[Votre Numéro de Téléphone]
[Votre Adresse Email]

```
None
--------------------------


 Merci de valider ou rejeter le brouillon
Réponse :  n
Commentaire :  à la fin tu ajoute, Bien crodialement : M.Youssfi
```



Human In The Loop



Human In The Loop

```
--- BROUILLON À VALIDER ---

Bonjour,

J'espère que vous allez bien.

Je me permets de vous contacter pour vous proposer un rendez-vous la semaine prochaine afin de vous préser
peuvent vous être bénéfiques.

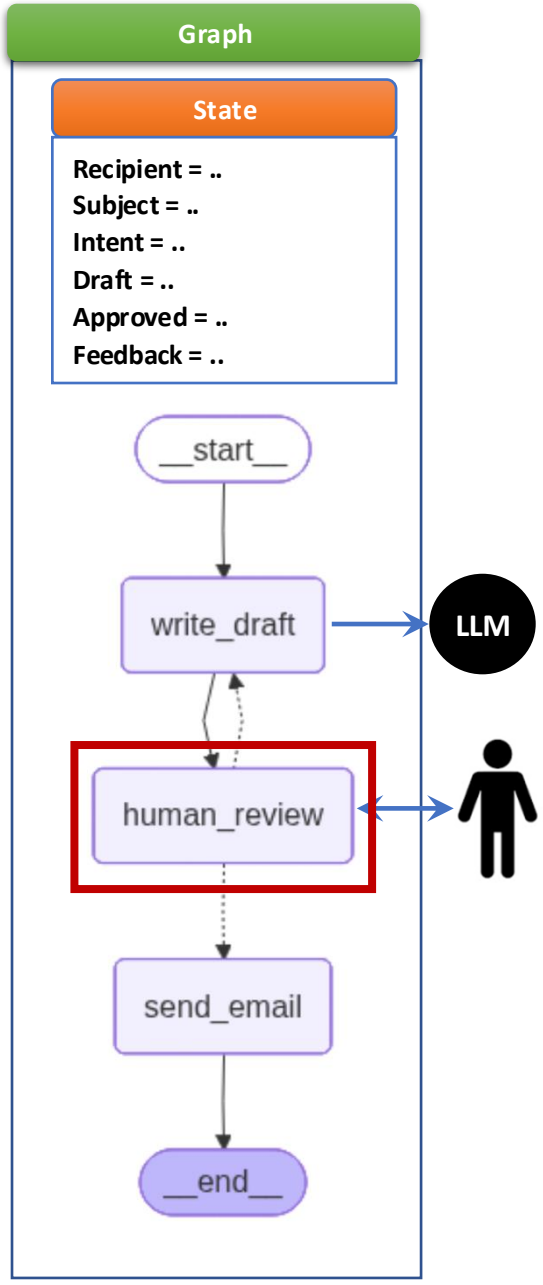
Pourriez-vous me faire savoir vos disponibilités ? Je suis flexible et peux m'adapter à votre emploi du temps.

Dans l'attente de votre retour, je vous souhaite une excellente journée.

Bien cordialement,
M. Youssfi
[Votre Poste]
[Votre Entreprise]
[Votre Numéro de Téléphone]
[Votre Adresse Email]

None
--------------------------

 Merci de valider ou rejeter le brouillon
Réponse :  o
```



Human In The Loop

```
=== EMAIL ENVOYÉ ===
À       : client@entreprise.com
Sujet   : Proposition de rendez-vous
Corps   :
Bonjour,

J'espère que vous allez bien.

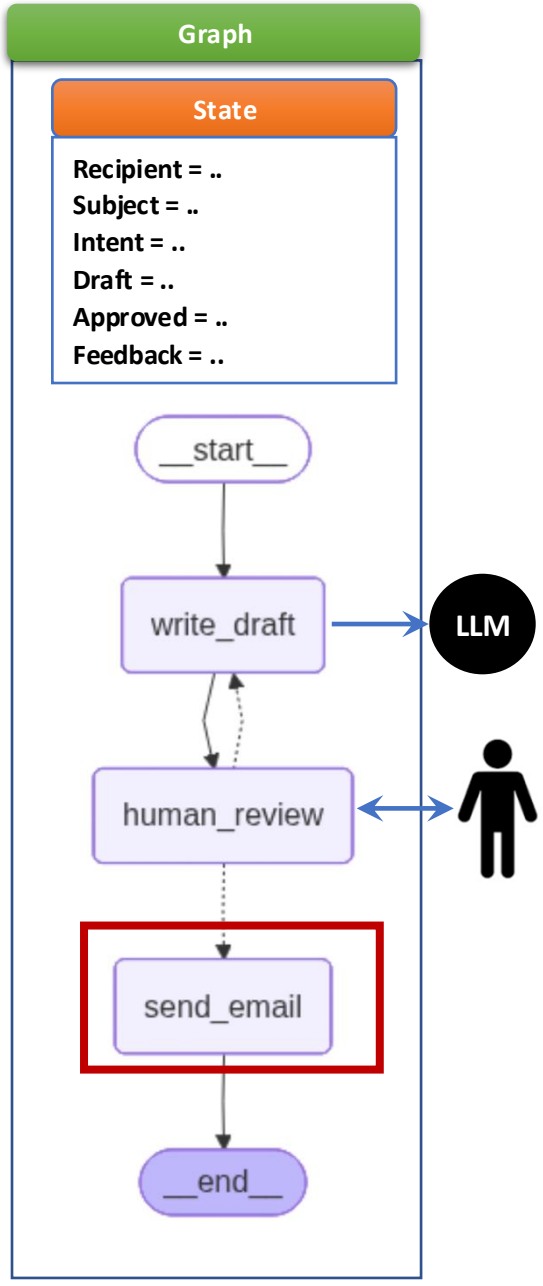
Je me permets de vous contacter pour vous proposer un rendez-vous la semai

Pourriez-vous me faire savoir vos disponibilités ? Je suis flexible et peu

Dans l'attente de votre retour, je vous souhaite une excellente journée.

Bien cordialement,
M. Youssfi
[Votre Poste]
[Votre Entreprise]
[Votre Numéro de Téléphone]
[Votre Adresse Email]
====================

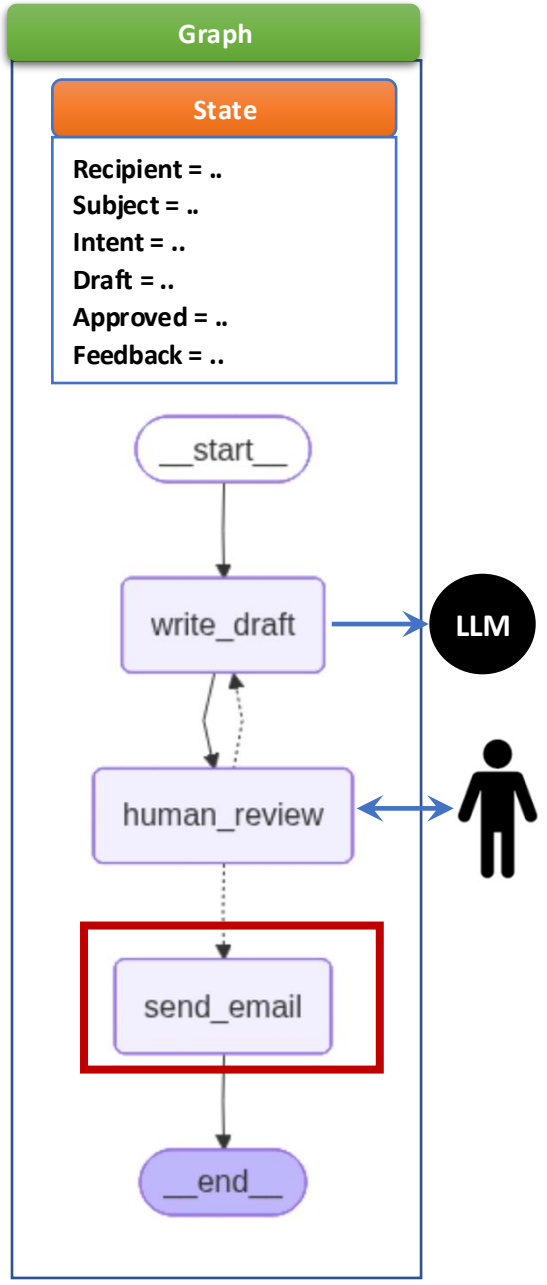
Workflow terminé
```



Human In The Loop

```
print(result)
```

```
{
  'recipient': 'client@entreprise.com',
  'subject': 'Proposition de rendez-vous',
  'intent': 'Écrire un email au format..... ',
  'draft': "Bonjour,\n\nJ'espère que vous .....",
  'approved': True,
  'feedback': ''
}
```



Human In The Loop

```python
import os
from typing_extensions import TypedDict, Literal
 from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, START, END
from langgraph.types import interrupt, Command
from langgraph.checkpoint.memory import InMemorySaver

class EmailState(TypedDict, total=False):
    recipient: str
    subject: str
    intent: str
    draft: str
    approved: bool
    feedback: str
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.4)
def write_draft(state: EmailState):
    feedback = state.get("feedback", "")
    previous_draft = state.get("draft", "")

    prompt = f"""
Tu es un assistant qui rédige des emails professionnels en français.

Destinataire: {state["recipient"]}
Sujet: {state["subject"]}
Objectif: {state["intent"]}

Brouillon précédent:
{previous_draft if previous_draft else "(aucun)"}

Commentaire humain:
{feedback if feedback else "Aucun commentaire. Rédige un premier brouillon."}

Instructions:
- Rédige un email clair, poli et concis
- Si un commentaire humain est fourni, réécris le brouillon en en tenant compte
- Retourne uniquement le corps de l'email
"""

    response = llm.invoke(prompt)
    return {"draft": response.content}
```

```python
def human_review(state: EmailState):
    review = interrupt(
        {
            "type": "human_review",
            "message": "Merci de valider ou rejeter le brouillon",
            "draft": state["draft"],
            "expected_response": {
                "approved": "bool",
                "feedback": "str (optionnel, utile si rejet)",
            },
        }
    )

    return {"approved": review["approved"], "feedback": review.get("feedback", "")}


def route_after_review(state: EmailState) -> Literal["send_email", "write_draft"]:

    if state["approved"]:
        return "send_email"
    return "write_draft"


def send_email(state: EmailState):
    print("\n=== EMAIL ENVOYÉ ===")
    print(f"À      : {state['recipient']}")
    print(f"Sujet  : {state['subject']}")
    print("Corps  :")
    print(state["draft"])
    print("====================\n")

    return {}
```

Human In The Loop

app.py

```
builder = StateGraph(EmailState)

builder.add_node("write_draft", write_draft)
builder.add_node("human_review", human_review)
builder.add_node("send_email", send_email)

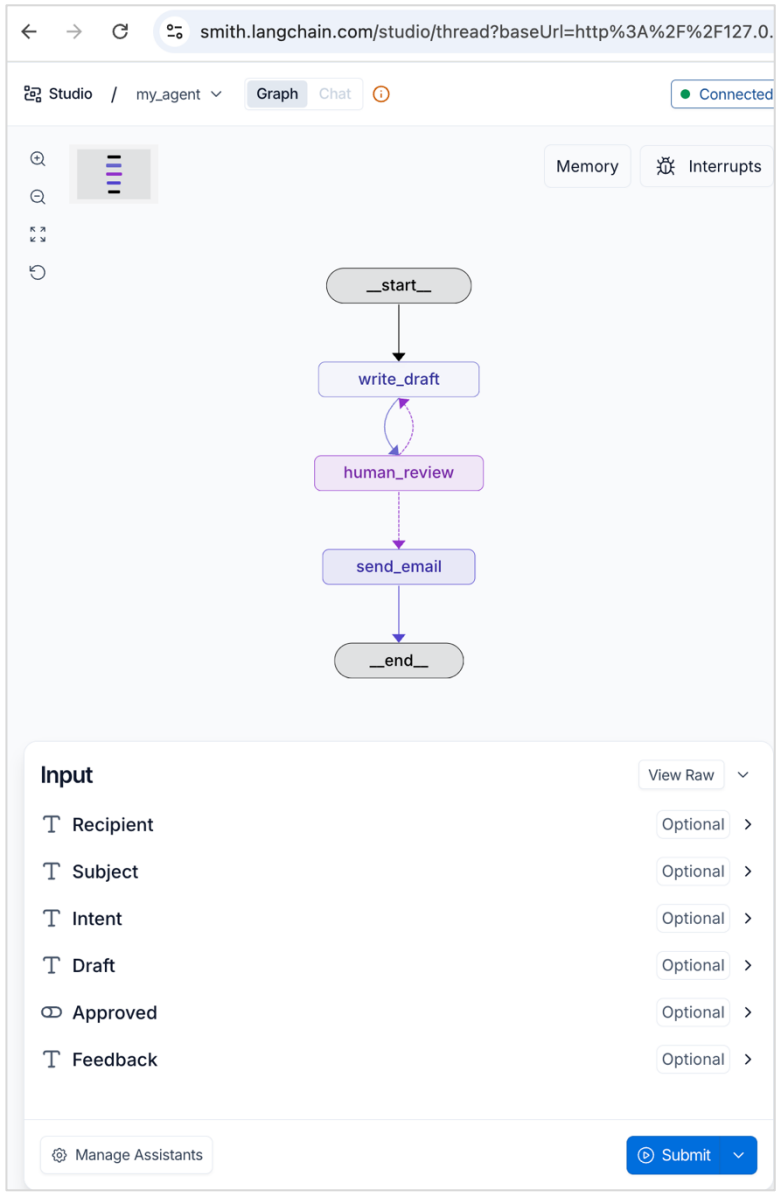
builder.add_edge(START, "write_draft")
builder.add_edge("write_draft", "human_review")
builder.add_conditional_edges("human_review", route_after_review)
builder.add_edge("send_email", END)

graph = builder.compile()
```

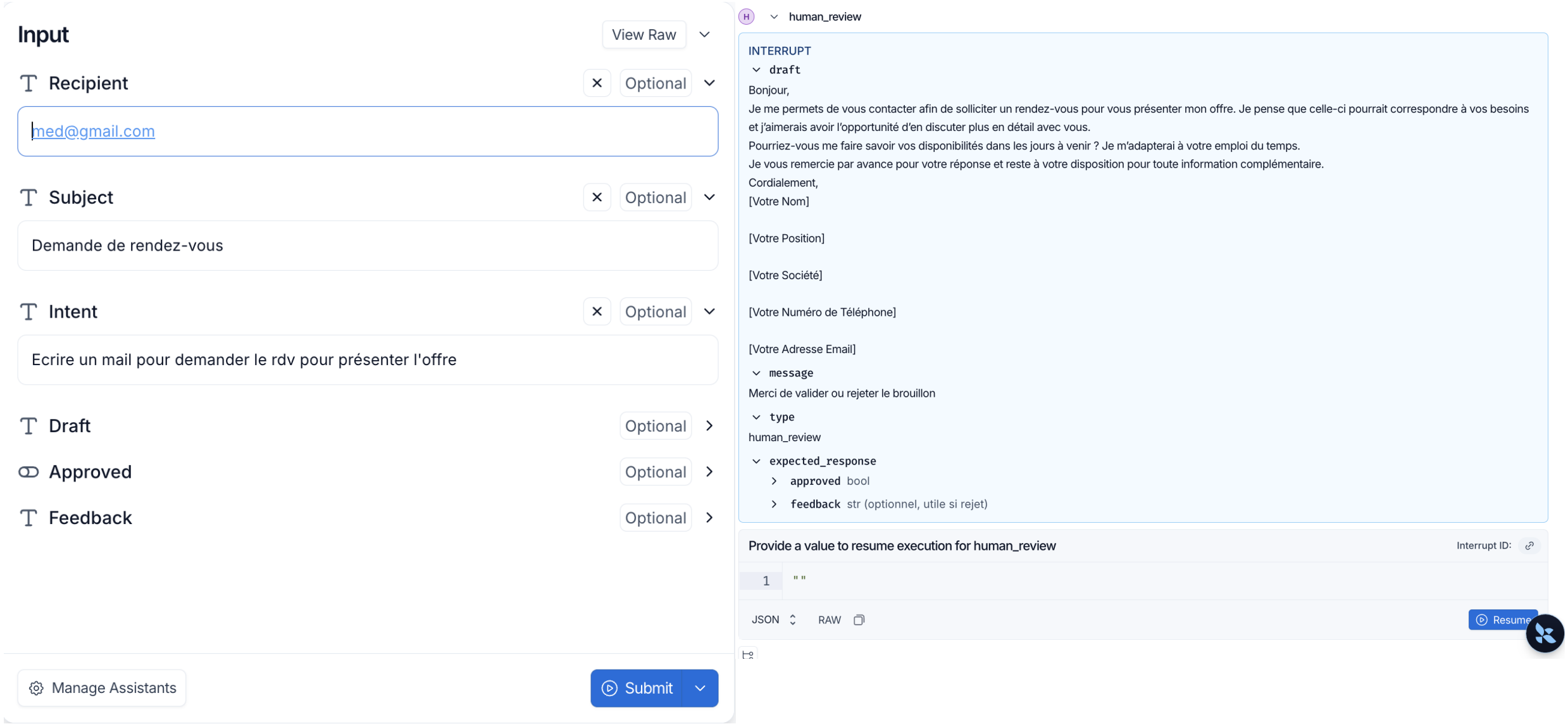
Langgraph.json

```
{
  "dependencies": ["langchain_openai", "./"],
  "graphs": {
    "my_agent": "./app.py:graph"
  },
  "env": "./.env"
}
```

$ langgraph dev



Human In The Loop

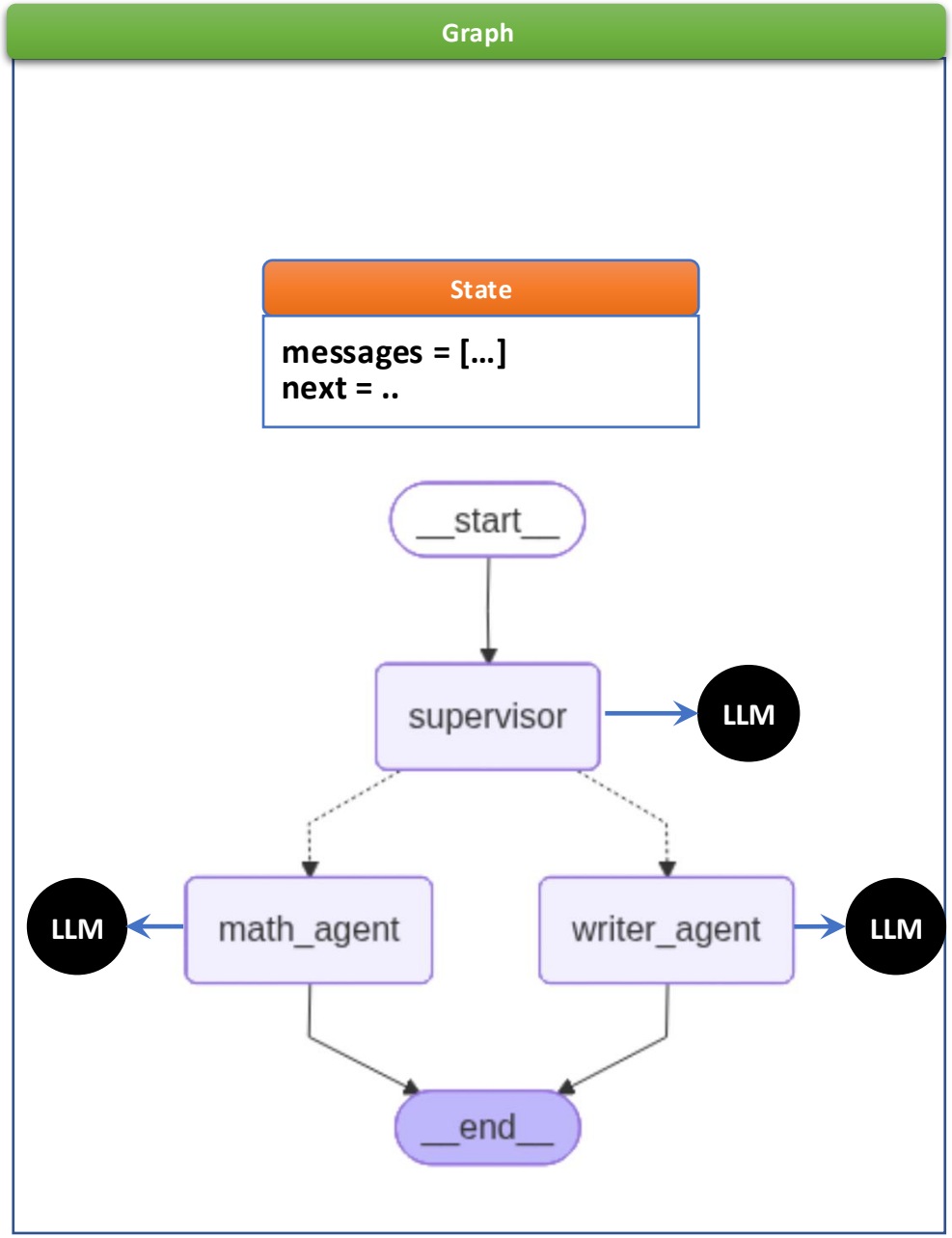


Human In The Loop

Multi Agent System

```
from typing import Annotated
from typing_extensions import TypedDict, Literal
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, AIMessage, SystemMessage
from IPython.display import Image
```

```
# Etat partagé
class AgentState(TypedDict):
    messages: Annotated[list, add_messages]
    next: str
```



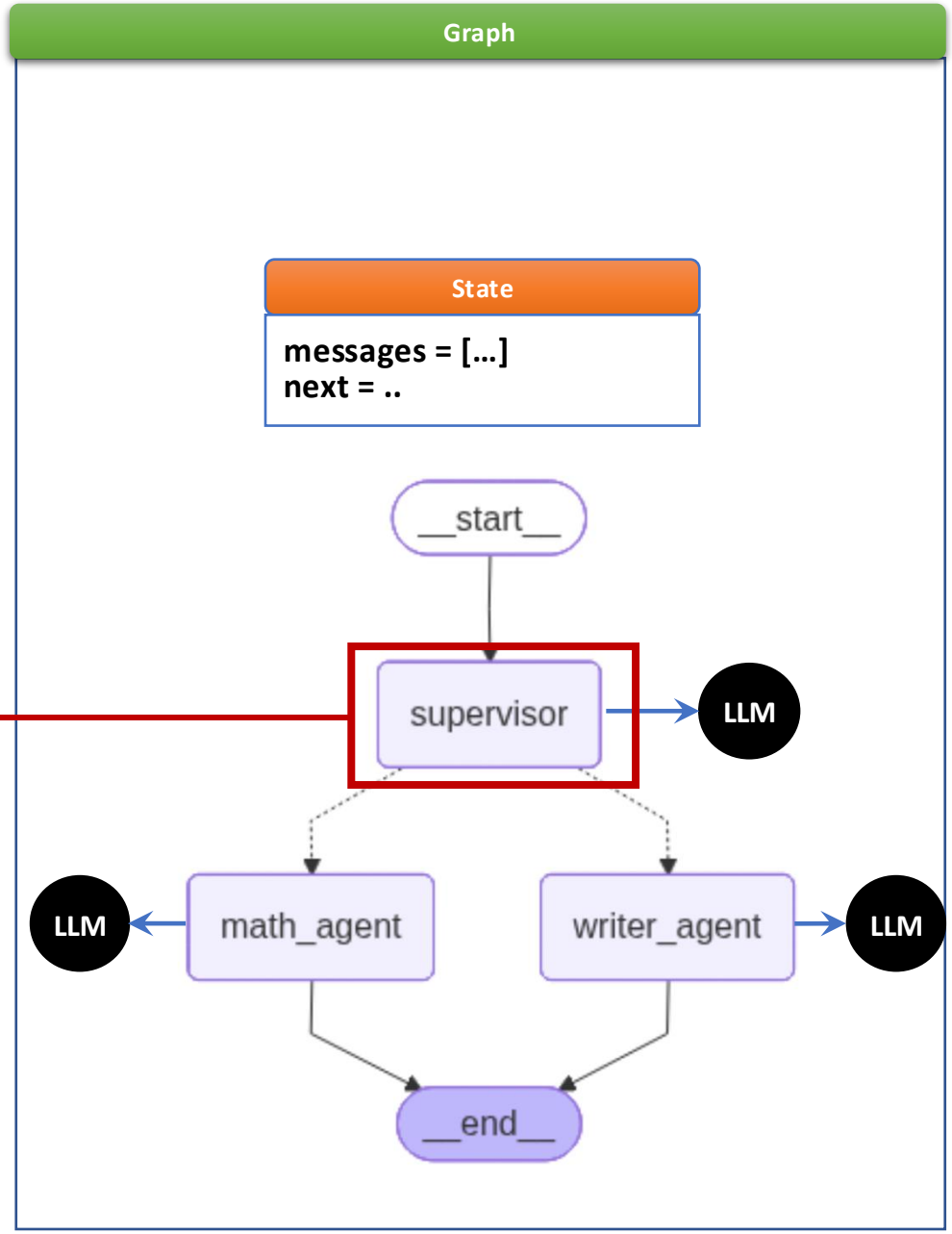
Multi Agent System

```
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
# LLM avec sortie structurée
router_llm = llm.with_structured_output(Router)
```

```
# Sortie attendue du supervisor
class Router(TypedDict):
    next: Literal["math_agent", "writer_agent"]
```

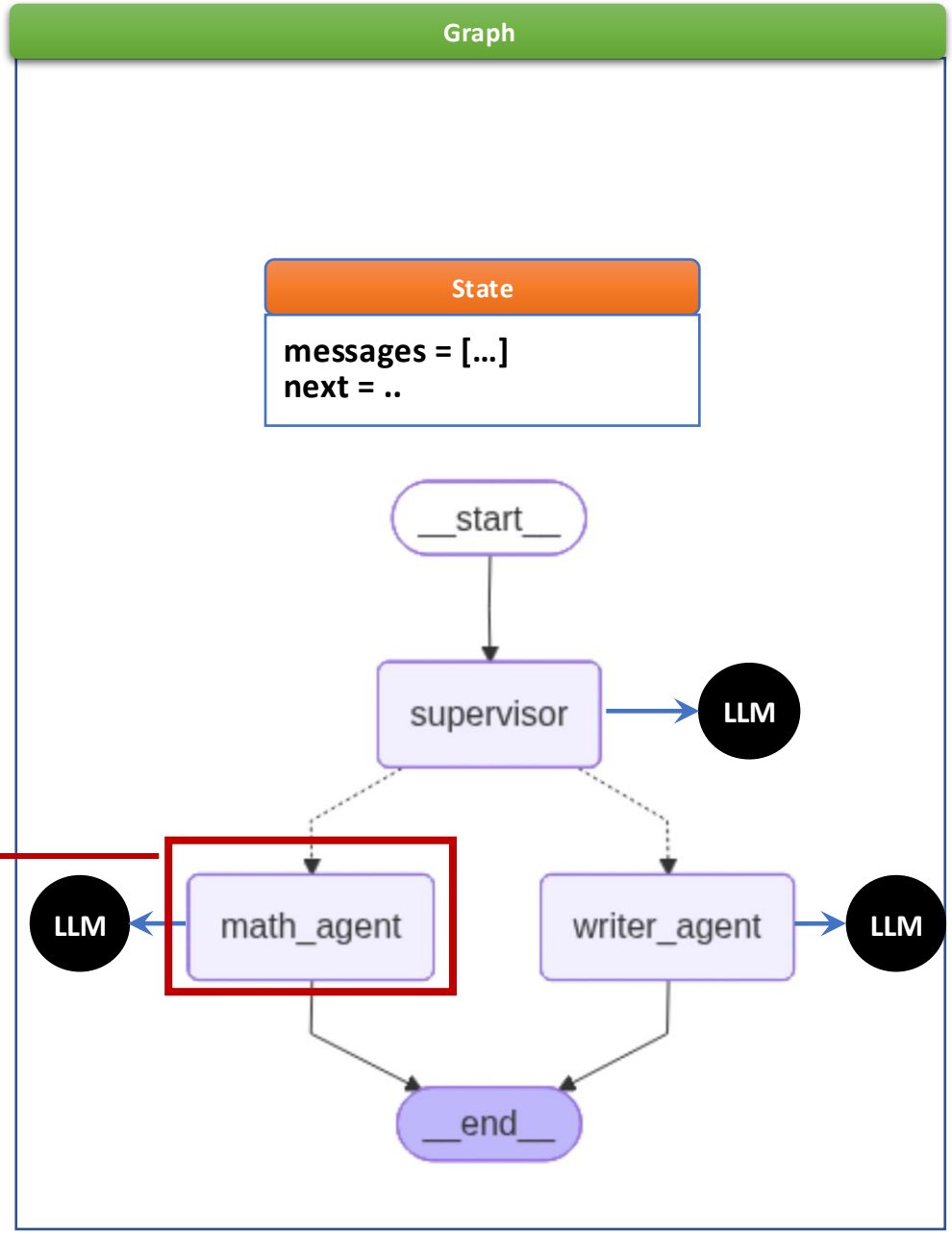
```
# Supervisor : utilise le LLM pour décider du prochain agent
def supervisor_node(state: AgentState):
    decision = router_llm.invoke([
        SystemMessage(content=(
    "Tu es un supervisor dans un système multi-agents.\n"
    "Choisis l'agent le plus adapté selon la demande utilisateur.\n"
    "- math_agent : pour calculs, équations, problèmes mathématiques\n"
    "- writer_agent : pour rédaction, reformulation, emails, textes\n"
    "Réponds uniquement avec le champ structuré attendu."
        )), *state["messages"]
    ])
    return {"next": decision["next"]}
```

```
# Fonction de routage
def route_after_supervisor(state: AgentState):
    return state["next"]
```



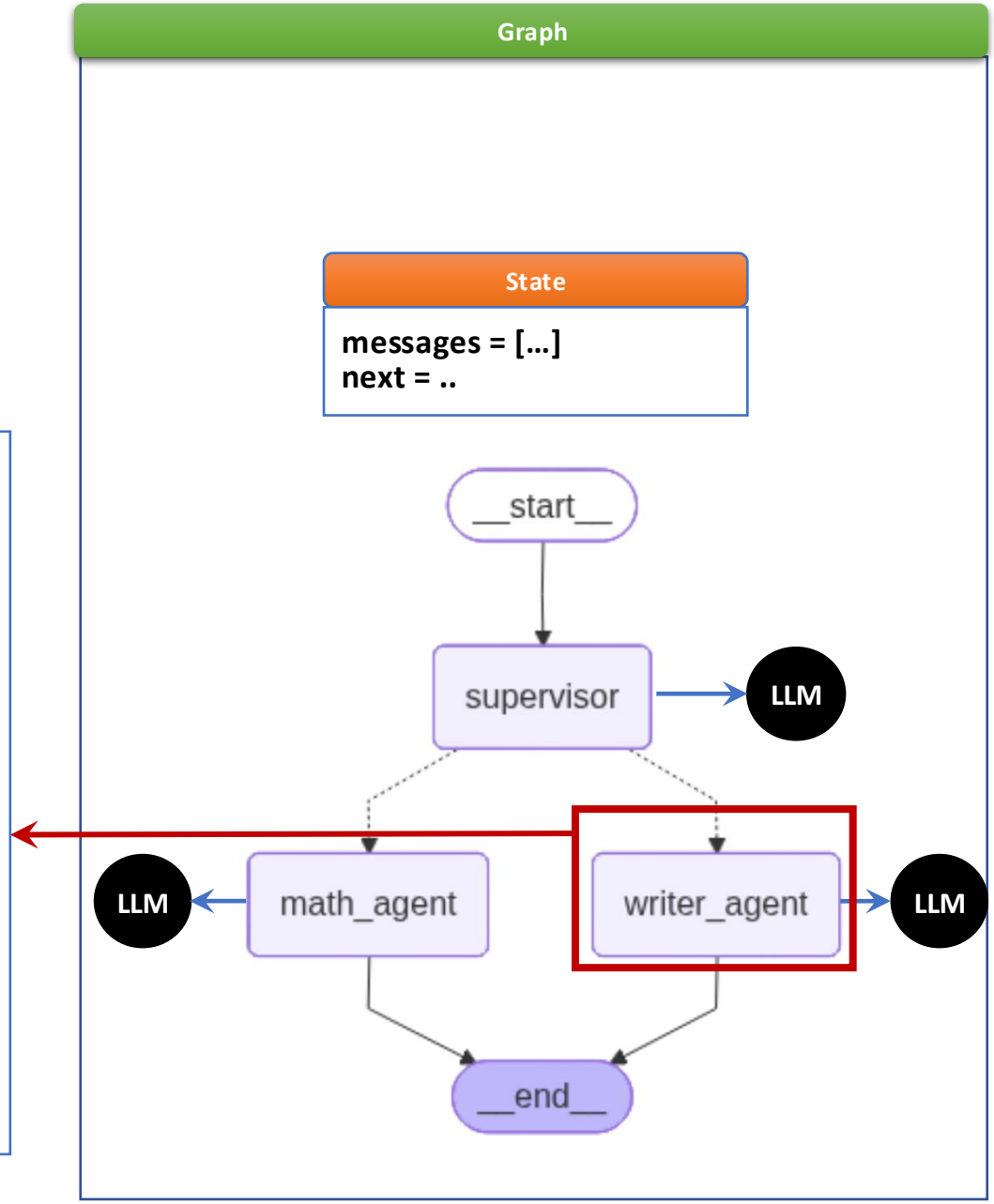
Multi Agent System

```
# Sous-agent math
def math_agent_node(state: AgentState):
    response = llm.invoke([
     SystemMessage(content="Tu es un agent expert en mathématiques. Réponds clairement et correctement."),
        *state["messages"]
    ])
    return {
    "messages": [
      AIMessage(
        content=response.content, name="math_agent »
      )
     ]
    }
```



Multi Agent System

```
# Sous-agent rédaction
def writer_agent_node(state: AgentState):
   response = llm.invoke([
      SystemMessage(content="Tu es un agent expert en rédaction. Rédige de manière claire et professionnelle."),
      *state["messages"]
   ])
   return {
      "messages": [
       AIMessage(
        content=response.content, name="writer_agent »
       )
     ]
   }
```

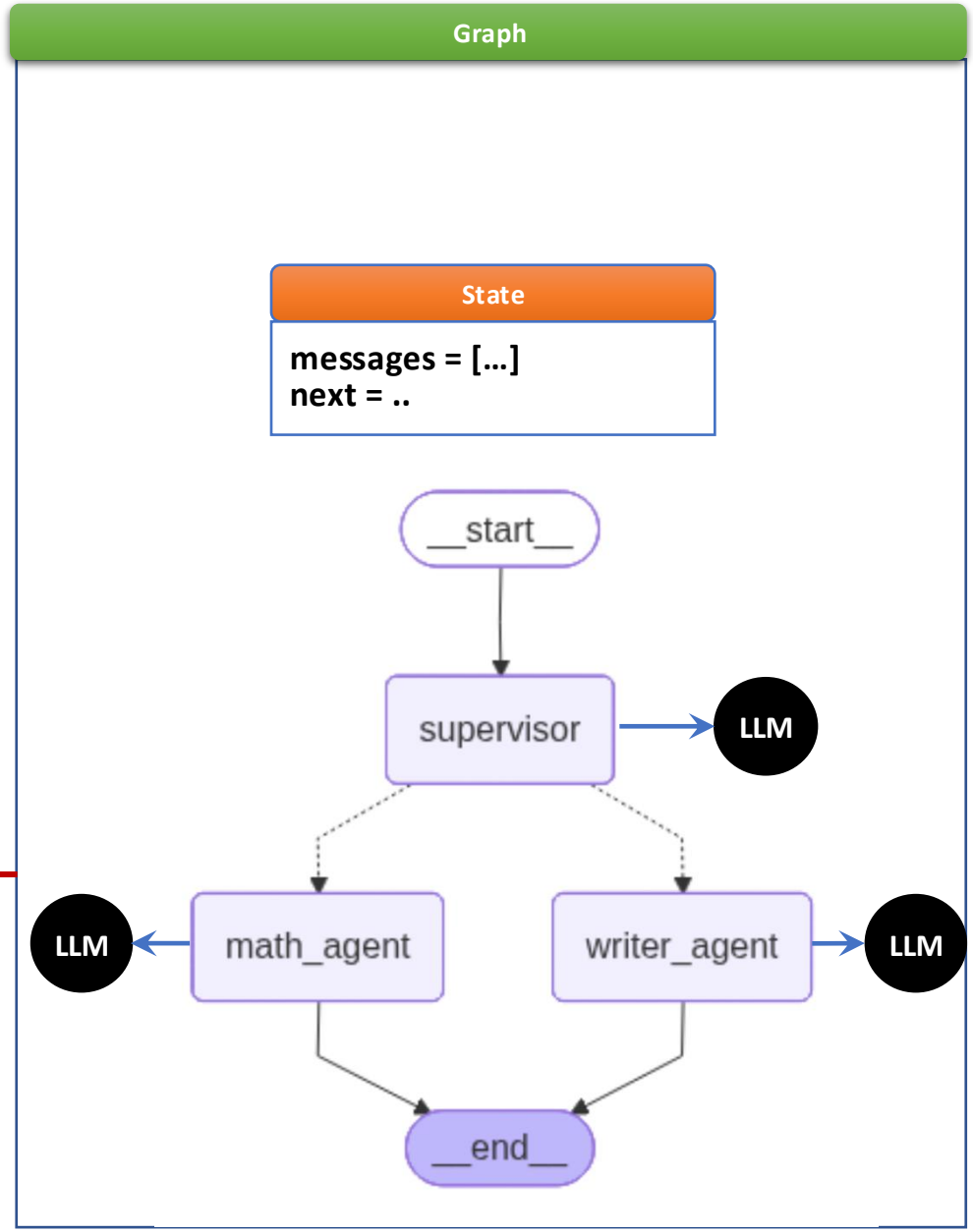


Multi Agent System

```
# Construction du graphe
builder = StateGraph(AgentState)
builder.add_node("supervisor", supervisor_node)
builder.add_node("math_agent", math_agent_node)
builder.add_node("writer_agent", writer_agent_node)

builder.add_edge(START, "supervisor")

builder.add_conditional_edges(
    "supervisor",
    route_after_supervisor,
    {
        "math_agent": "math_agent",
        "writer_agent": "writer_agent",
    }
)
builder.add_edge("math_agent", END)
builder.add_edge("writer_agent", END)
graph = builder.compile()
display(Image(graph.get_graph().draw_mermaid_png()))
```



Multi Agent System

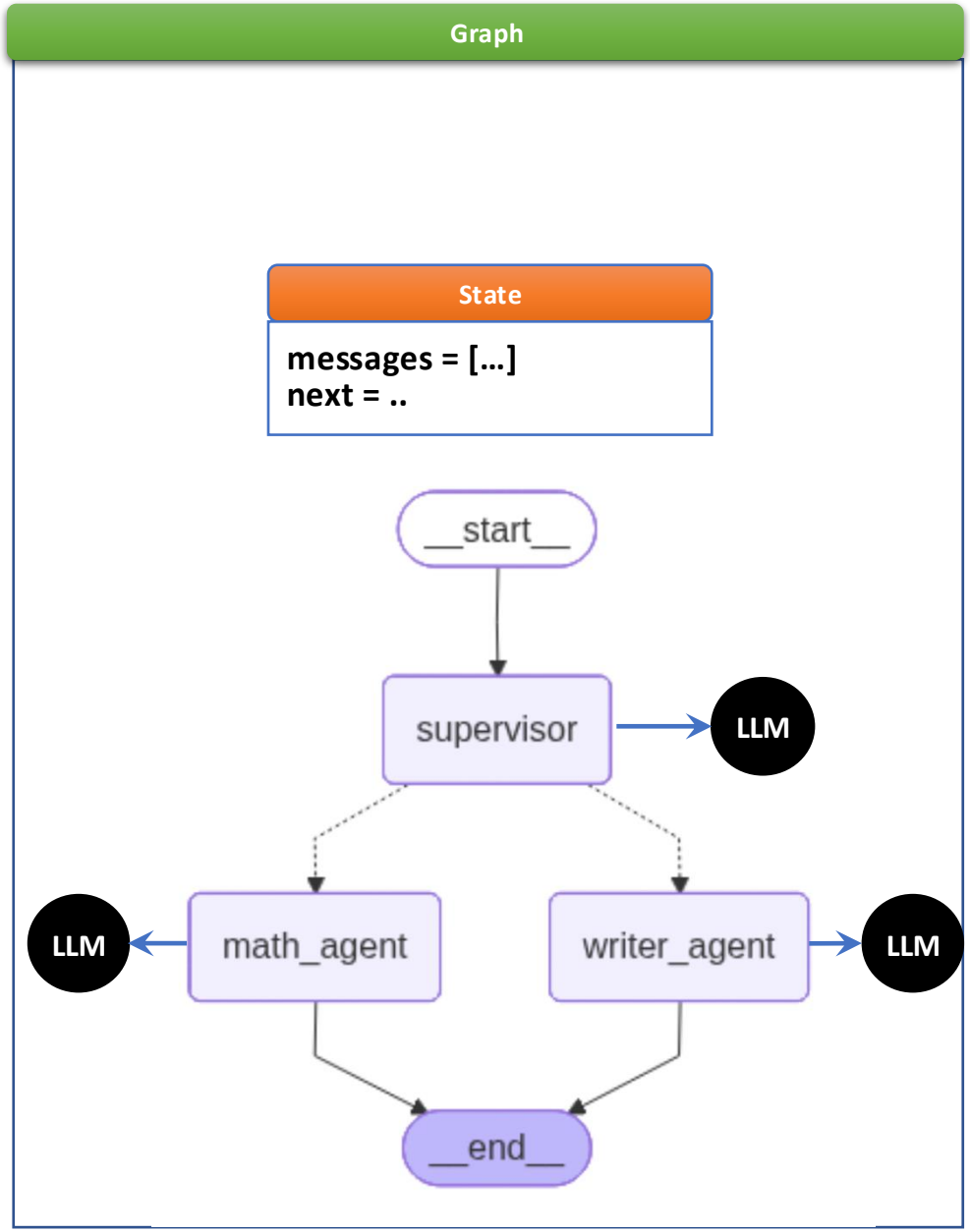
```
result = graph.invoke({
 "messages": [HumanMessage(content="Calcule 18 * 27")]
})
print(result["messages"][-1].content)
```

18 * 27 = 486.

```
result = graph.invoke({
"messages": [HumanMessage(content="Rédige un email pour demander un rendez-vous")]
})

print(result["messages"][-1].content)
```

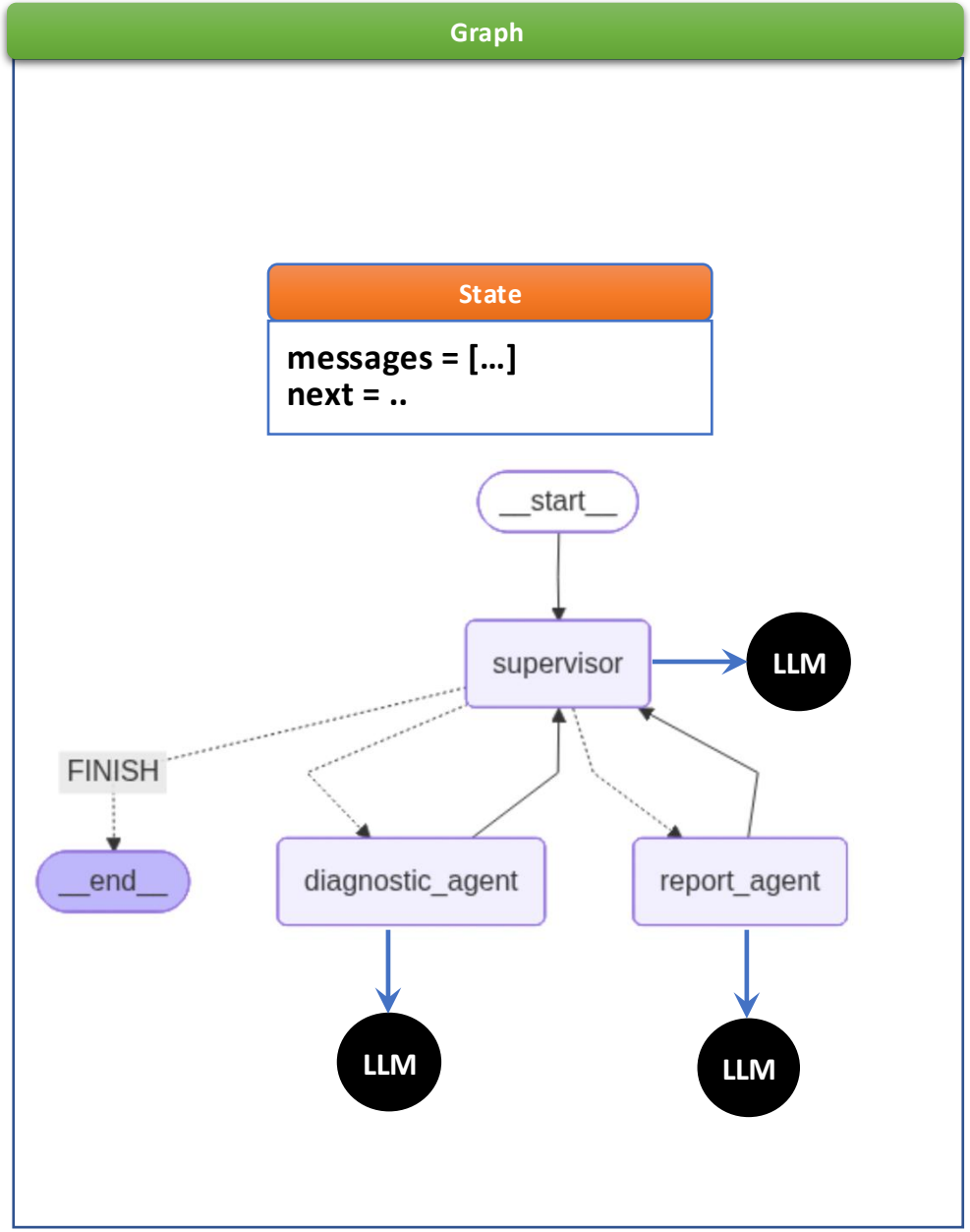
Bonjour [Nom du destinataire], J'espère que vous allez bien. Je me permets de vous contacter afin de solliciter un rendez-vous pour discuter de [préciser l'objet de la rencontre, par exemple : "notre collaboration", "un projet en cours", "des opportunités d'affaires", etc.]. Je pense qu'un échange direct serait bénéfique pour [expliquer brièvement l'intérêt de la rencontre]. Je suis disponible aux dates suivantes : [proposer quelques créneaux horaires], mais je reste flexible et je m'adapterai à votre emploi du temps. Je vous remercie par avance pour votre réponse et reste à votre disposition pour toute information complémentaire. Cordialement, [Votre nom] [Votre poste] [Votre entreprise] [Votre numéro de téléphone] [Votre adresse e-mail]



Multi Agent System

```
# ====================================================
# 1) Etat partagé du graphe
# ====================================================
class MedicalState(TypedDict, total=False):
    messages: Annotated[list, add_messages]
    next: Literal["diagnostic_agent", "report_agent", "FINISH"]
    diagnostic_summary: str
    final_report: str
```

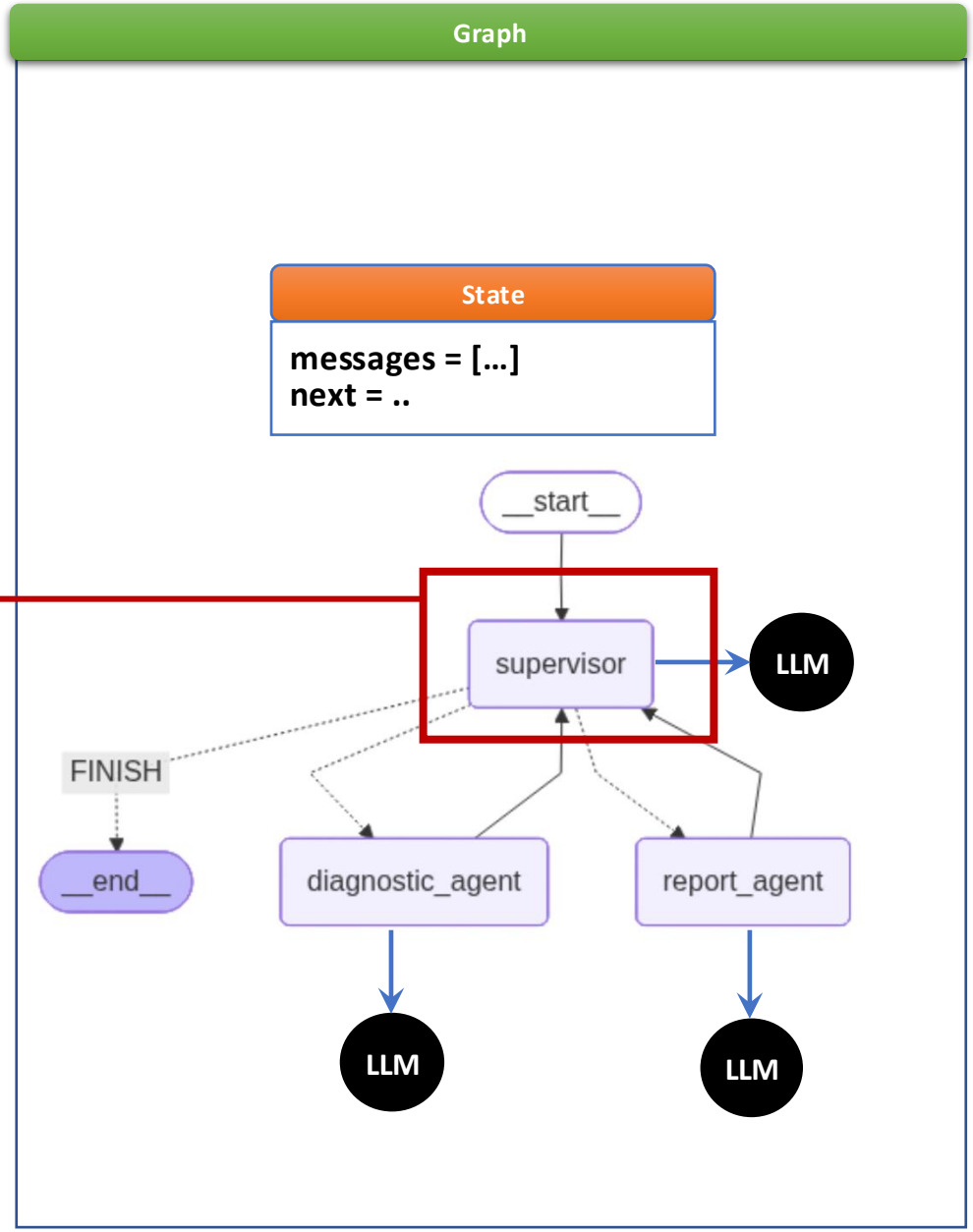
```
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
```



Multi Agent System

```python
def supervisor_node(state: MedicalState):
    if "diagnostic_summary" not in state:
        return {
            "next": "diagnostic_agent",
            "messages": [
                AIMessage(
                    content="Le supervisor envoie le dossier à diagnostic_agent.",
                    name="supervisor"
                )
            ]
        }
    if "final_report" not in state:
        return {
            "next": "report_agent",
            "messages": [
                AIMessage(
                    content="Le supervisor transmet l'analyse à report_agent pour rédaction finale.",
                    name="supervisor"
                )
            ]
        }
    return {
        "next": "FINISH",
        "messages": [
            AIMessage(
                content="Le supervisor clôt le workflow.",
                name="supervisor"
            )
        ]
    }
```

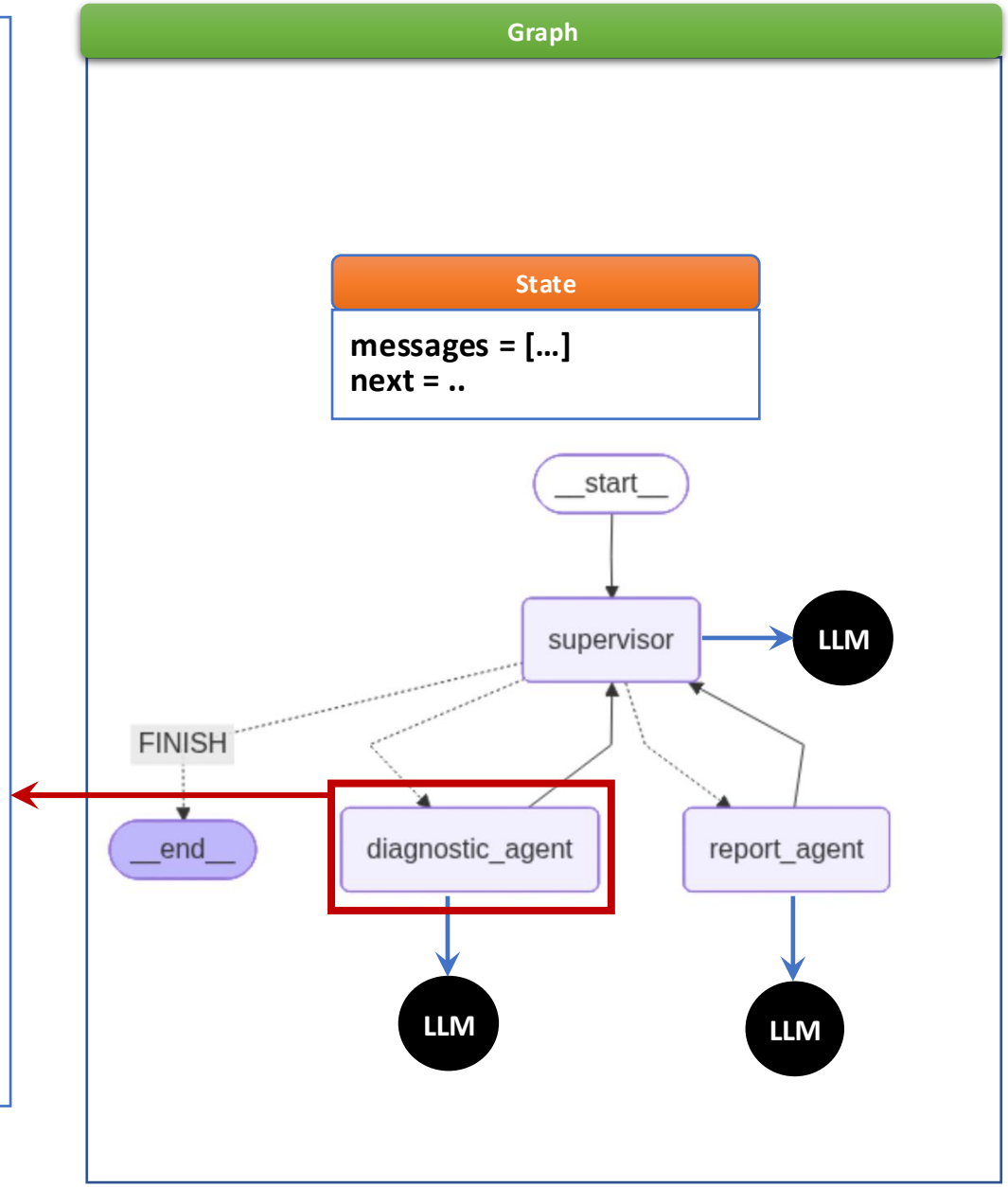
```python
def route_supervisor(state: MedicalState):
    return state["next"]
```



Multi Agent System

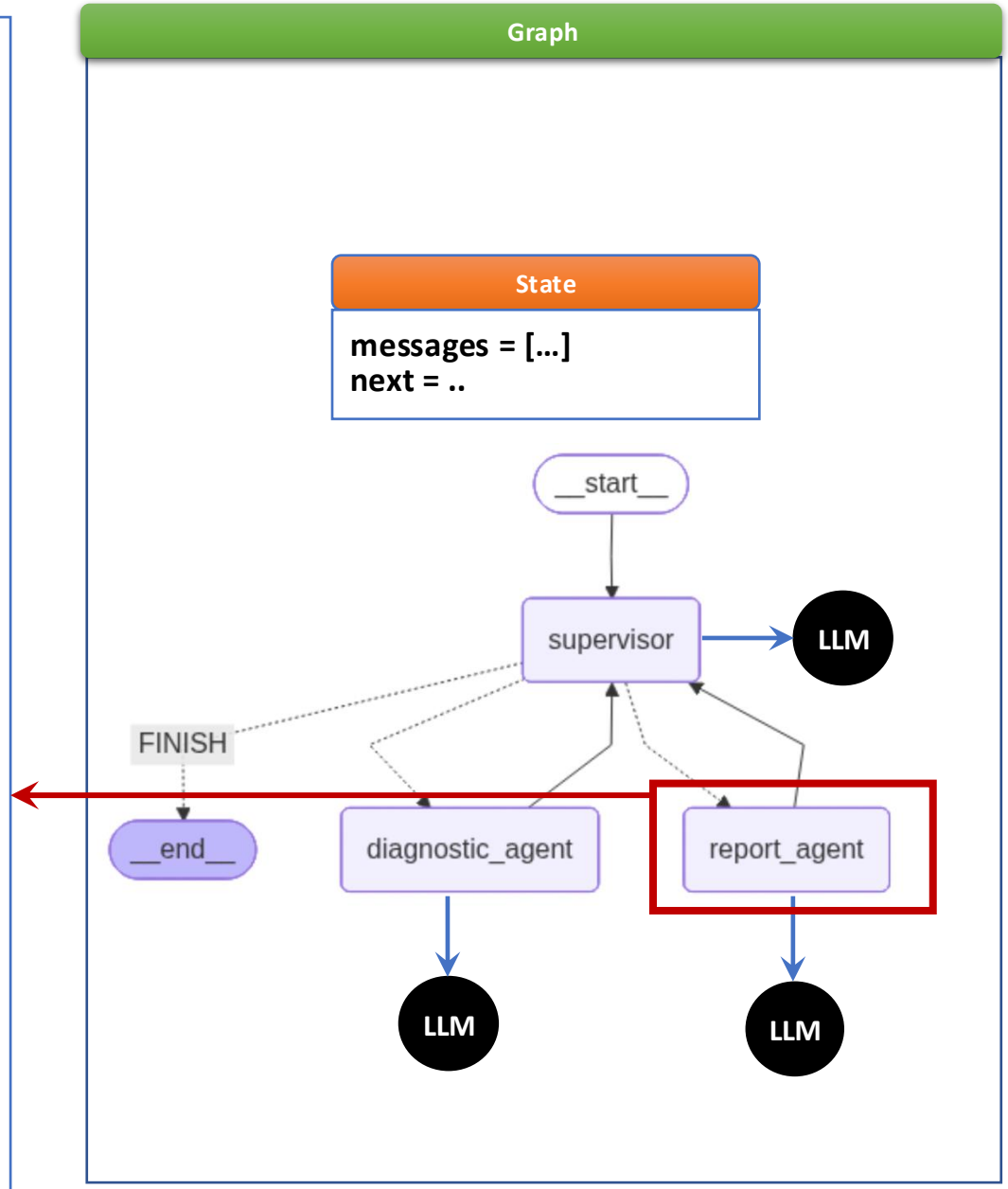
```
def diagnostic_agent_node(state: MedicalState):
    response = llm.invoke([
        SystemMessage(content=(
            "Tu es un agent d'orientation clinique générale.\n"
            "Tu analyses les symptômes décrits par l'utilisateur et tu produis une synthèse
clinique préliminaire.\n"
            "Tu ne poses PAS de diagnostic définitif.\n"
            "Tu dois fournir :\n"
            "1. Résumé clinique\n"
            "2. Hypothèses possibles\n"
            "3. Signes d'alerte éventuels\n"
            "4. Niveau d'urgence estimé\n"
            "5. Recommandations générales de consultation\n"
            "Sois prudent, clair et structuré."
        )),
        *state["messages"]
    ])

    return {
        "diagnostic_summary": response.content,
        "messages": [
            AIMessage(
                content=response.content,
                name="diagnostic_agent"
            )
        ]
    }
```



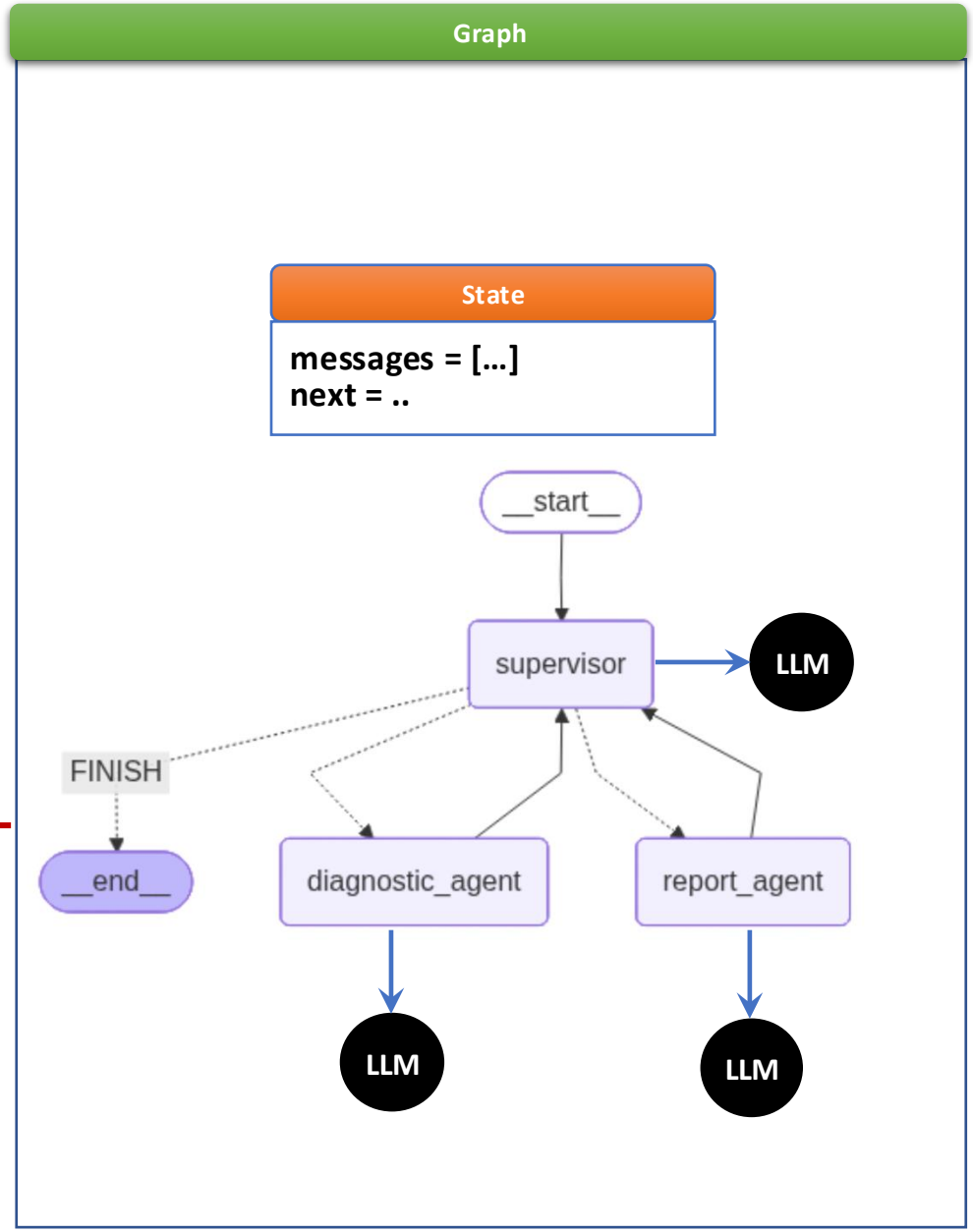
Multi Agent System

```python
def report_agent_node(state: MedicalState):
    response = llm.invoke([
        SystemMessage(content=(
            "Tu es un agent rapporteur médical.\n"
            "Ta mission est de transformer l'analyse fournie par diagnostic_agent "
            "en un rapport final clair, professionnel et lisible.\n"
            "Le rapport doit contenir les sections suivantes :\n"
            "- Motif principal\n"
            "- Symptômes rapportés\n"
            "- Synthèse clinique préliminaire\n"
            "- Hypothèses générales\n"
            "- Signes d'alerte\n"
            "- Niveau d'urgence\n"
            "- Conseils généraux\n"
            "- Mention de prudence : ce rapport ne remplace pas un avis médical.\n"
            "Le ton doit être clair et structuré."
        )),
        HumanMessage(content=(
            "Voici la demande initiale du patient :\n\n"
            f"{state['messages'][0].content}\n\n"
            "Voici l'analyse produite par diagnostic_agent :\n\n"
            f"{state['diagnostic_summary']}"
        ))
    ])
    return {
        "final_report": response.content,
        "messages": [
            AIMessage(
                content=response.content,
                name="report_agent"
            )
        ]
    }
```



Multi Agent System

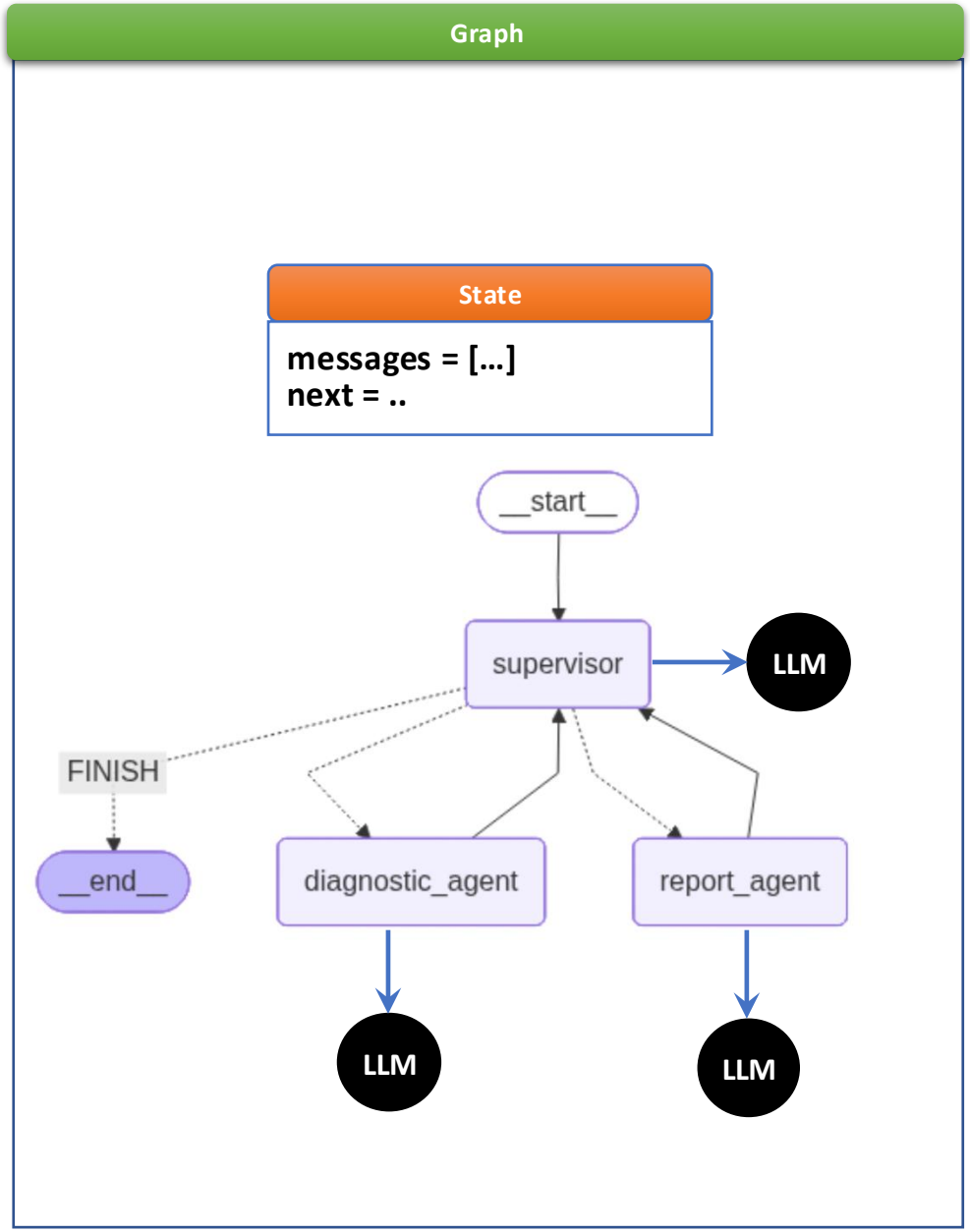
```
builder = StateGraph(MedicalState)
builder.add_node("supervisor", supervisor_node)
builder.add_node("diagnostic_agent", diagnostic_agent_node)
builder.add_node("report_agent", report_agent_node)
builder.add_edge(START, "supervisor")
builder.add_conditional_edges(
   "supervisor",
   route_supervisor,
   {
      "diagnostic_agent": "diagnostic_agent",
      "report_agent": "report_agent",
      "FINISH": END,
   }
)
builder.add_edge("diagnostic_agent", "supervisor")
builder.add_edge("report_agent", "supervisor")
graph = builder.compile()
display(Image(graph.get_graph().draw_mermaid_png()))
```



Multi Agent System

```
result = graph.invoke({
   "messages": [
      HumanMessage(
         content=(
"Patient de 45 ans présentant depuis 2 jours une toux, "
"une fièvre modérée à 38,2°C, fatigue, et légère douleur thoracique "
"à l'inspiration. Pas d'antécédent cardiaque connu."
         )
      )
   ]
})
```

```
print("=== Diagnostic / orientation clinique (A1) ===")
print(display(Markdown((result["diagnostic_summary"]))))
print("=== Rapport final (A2) ===")
print(display(Markdown(result["final_report"])))
```

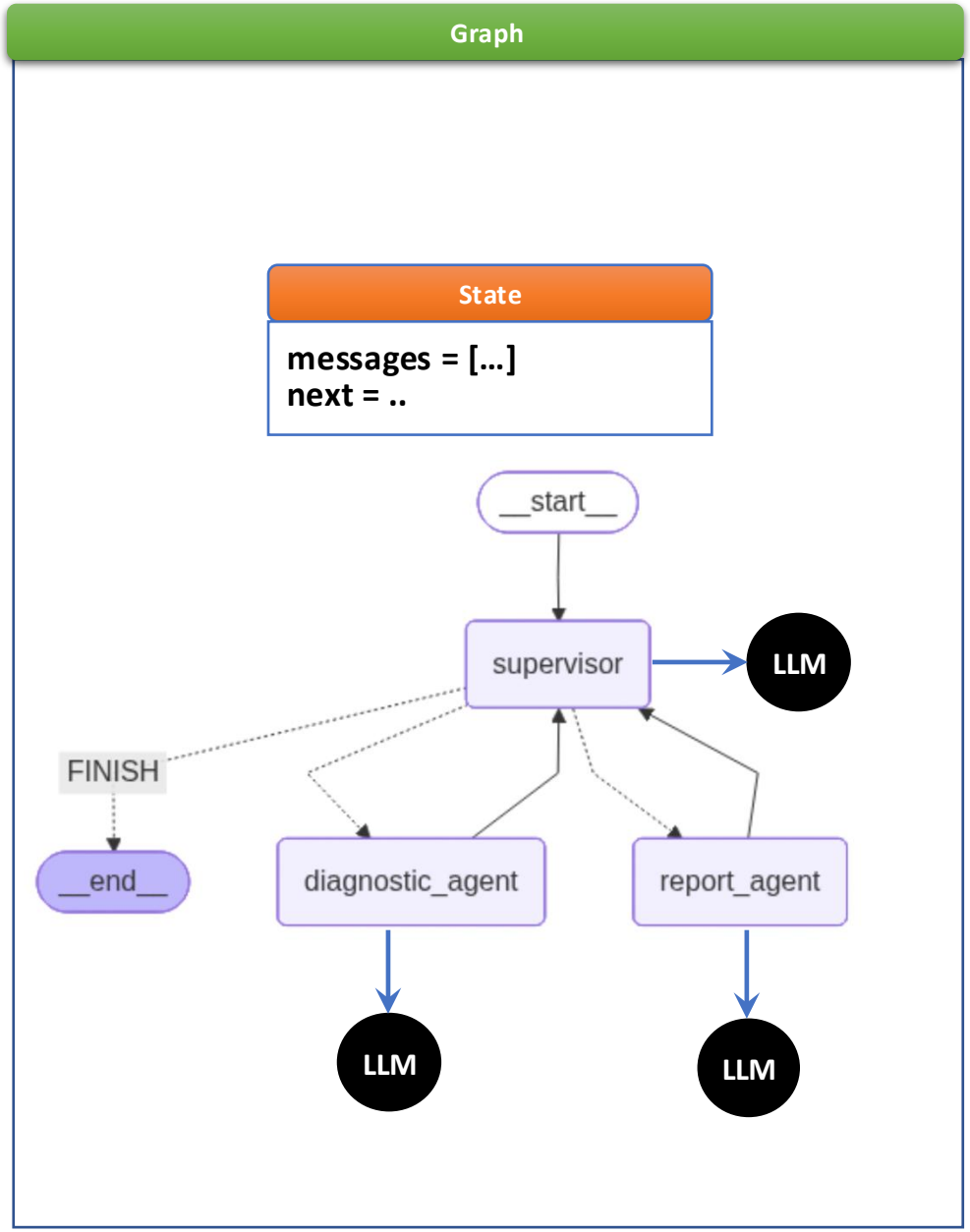


Multi Agent System

```
print("=== Diagnostic / orientation clinique (A1) ===")
print(display(Markdown((result["diagnostic_summary"]))))
```

```
=== Diagnostic / orientation clinique (A1) ===
```

1. **Résumé clinique :**
   - Patient de 45 ans.
   - Symptômes présents depuis 2 jours : toux, fièvre modérée (38,2°C), fatigue, légère douleur thoracique à l'inspiration.
   - Pas d'antécédent cardiaque connu.
2. **Hypothèses possibles :**
   - Infection respiratoire virale (ex. : bronchite, grippe).
   - Pneumonie (bactérienne ou virale).
   - Pleurésie (inflammation de la plèvre).
   - Éventuellement, une pathologie allergique ou irritative (moins probable compte tenu de la fièvre).
3. **Signes d'alerte éventuels :**
   - Aggravation de la douleur thoracique.
   - Difficulté respiratoire ou essoufflement.
   - Fièvre persistante ou élevée (supérieure à 39°C).
   - Expectoration de sang ou de mucus purulent.
   - Altération de l'état général (confusion, fatigue extrême).
4. **Niveau d'urgence estimé :**
   - Modéré. Bien que les symptômes ne soient pas immédiatement alarmants, la douleur thoracique et la fièvre nécessitent une évaluation médicale pour exclure des conditions plus graves.
5. **Recommandations générales de consultation :**
   - Consulter un médecin généraliste ou un service d'urgence si les symptômes s'aggravent ou si des signes d'alerte apparaissent.
   - En cas de symptômes persistants au-delà de 3 jours sans amélioration, une consultation est recommandée pour évaluer la nécessité d'examens complémentaires (radiographie thoracique, analyses sanguines, etc.).
   - Repos, hydratation et éventuellement des antipyrétiques pour gérer la fièvre et la douleur.



Multi Agent System

```
print("=== Rapport final (A2) ===")
print(display(Markdown(result["final_report"])))
```

```
=== Rapport final (A2) ===
```

Rapport médical

Motif principal :
Patient de 45 ans présentant des symptômes respiratoires et fébriles depuis 2 jours.

Symptômes rapportés :

- Toux
- Fièvre modérée à 38,2°C
- Fatigue
- Légère douleur thoracique à l'inspiration

Synthèse clinique préliminaire :
Le patient, âgé de 45 ans, présente des symptômes respiratoires et une fièvre modérée depuis 2 jours. Il ne rapporte pas d'antécédents cardiaques connus, ce qui pourrait influencer l'évaluation de sa douleur thoracique.

Hypothèses générales :
Les symptômes peuvent être attribués à plusieurs causes possibles, notamment :

- Infection respiratoire virale (ex. : bronchite, grippe)
- Pneumonie (bactérienne ou virale)
- Pleurésie (inflammation de la plèvre)
- Pathologie allergique ou irritative (moins probable compte tenu de la fièvre)

Signes d'alerte :
Le patient doit être vigilant et consulter un professionnel de santé si l'un des signes suivants apparaît :

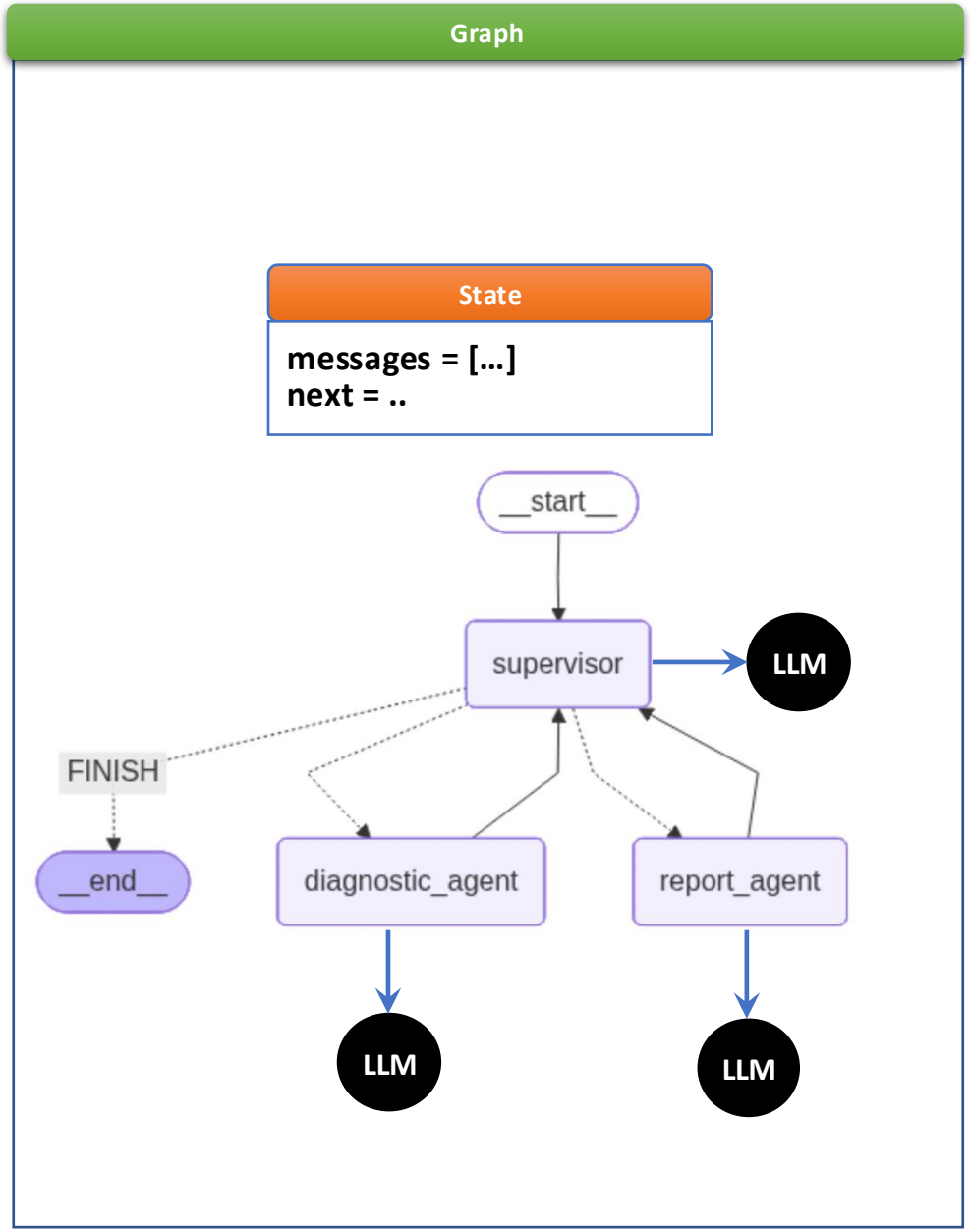
- Aggravation de la douleur thoracique
- Difficulté respiratoire ou essoufflement
- Fièvre persistante ou élevée (supérieure à 39°C)
- Expectoration de sang ou de mucus purulent
- Altération de l'état général (confusion, fatigue extrême)

Niveau d'urgence :
Le niveau d'urgence est estimé comme modéré. Bien que les symptômes ne soient pas immédiatement alarmants, la présence de douleur thoracique et de fièvre nécessite une évaluation médicale pour exclure des conditions plus graves.

Conseils généraux :

- Consulter un médecin généraliste ou un service d'urgence si les symptômes s'aggravent ou si des signes d'alerte apparaissent.
- En cas de symptômes persistants au-delà de 3 jours sans amélioration, une consultation est recommandée pour évaluer la nécessité d'examens complémentaires (radiographie thoracique, analyses sanguines, etc.).
- Repos, hydratation et éventuellement des antipyrétiques pour gérer la fièvre et la douleur.

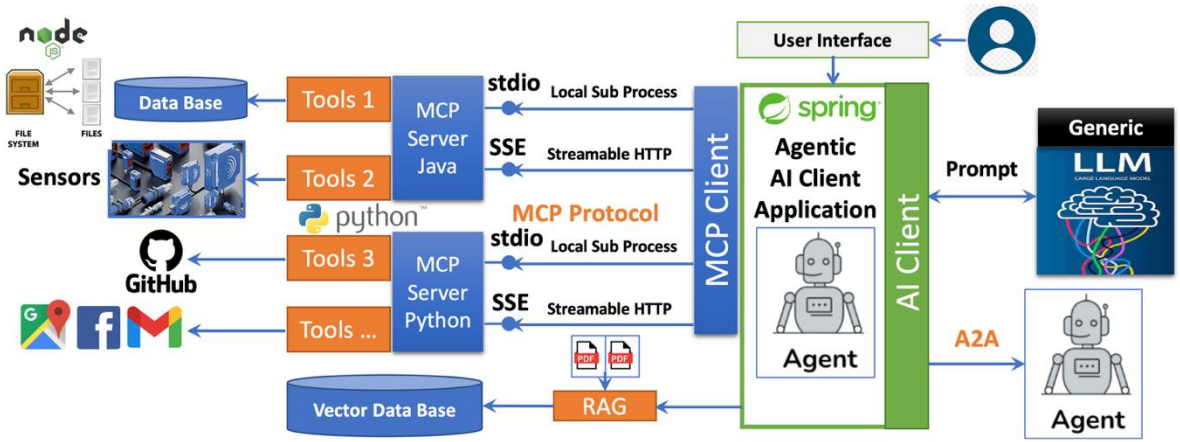
Mention de prudence :
Ce rapport ne remplace pas un avis médical. Il est essentiel de consulter un professionnel de santé pour un diagnostic et un traitement appropriés.

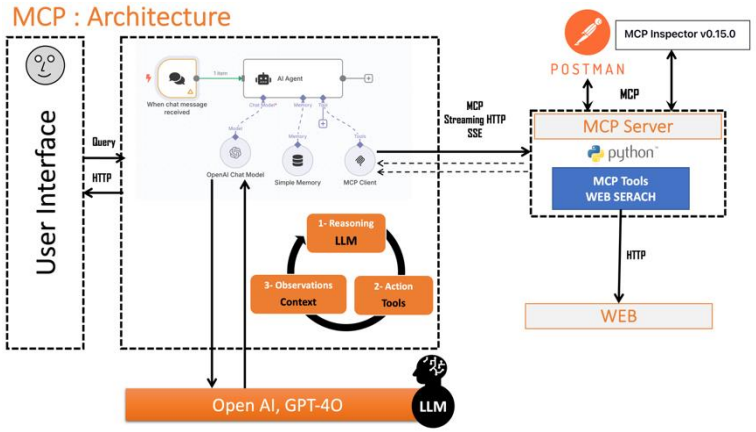


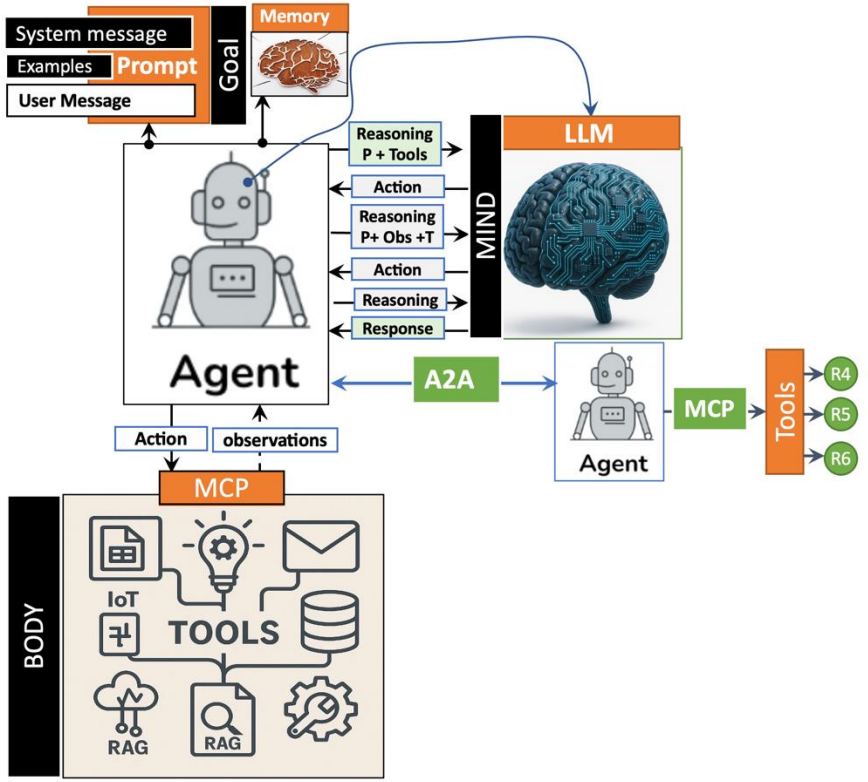
Model Context Protocol : MCP

Mohamed Youssfi, Enseignant Chercheur, ENSET Mohammedia, Directeur Laboratoire Informatique, Intelligence Artificielle et Cyber Sécurité









Agentic AI





Merci de faire la compagne de digital marketing

System message
Examples
Prompt
User Message

Goal

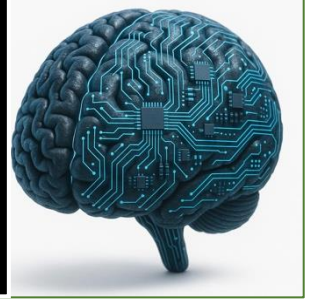
Memory



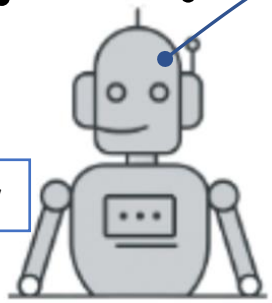
Reasoning P + Tools
Action
Reasoning P+ Obs +T
Action
Reasoning
Response

MIND

LLM

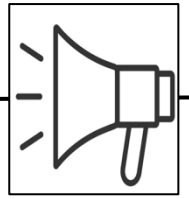


Agent = MIND + Body

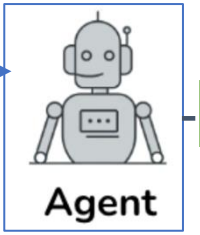


Agent

La compagne de digital marketing a été effectuée



A2A



Agent

MCP

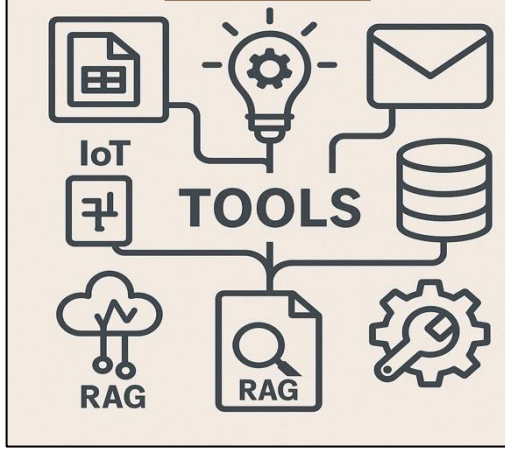
Tools

R4
R5
R6

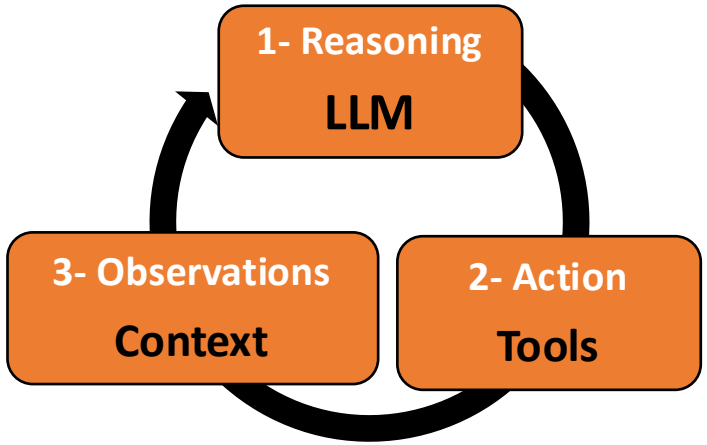
Action
observations

MCP

BODY



ReAct Pattern



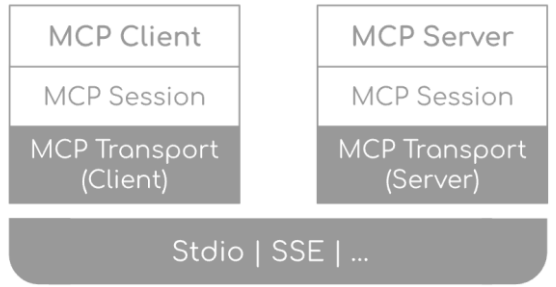
Frameworks an Agentic AI Platforms

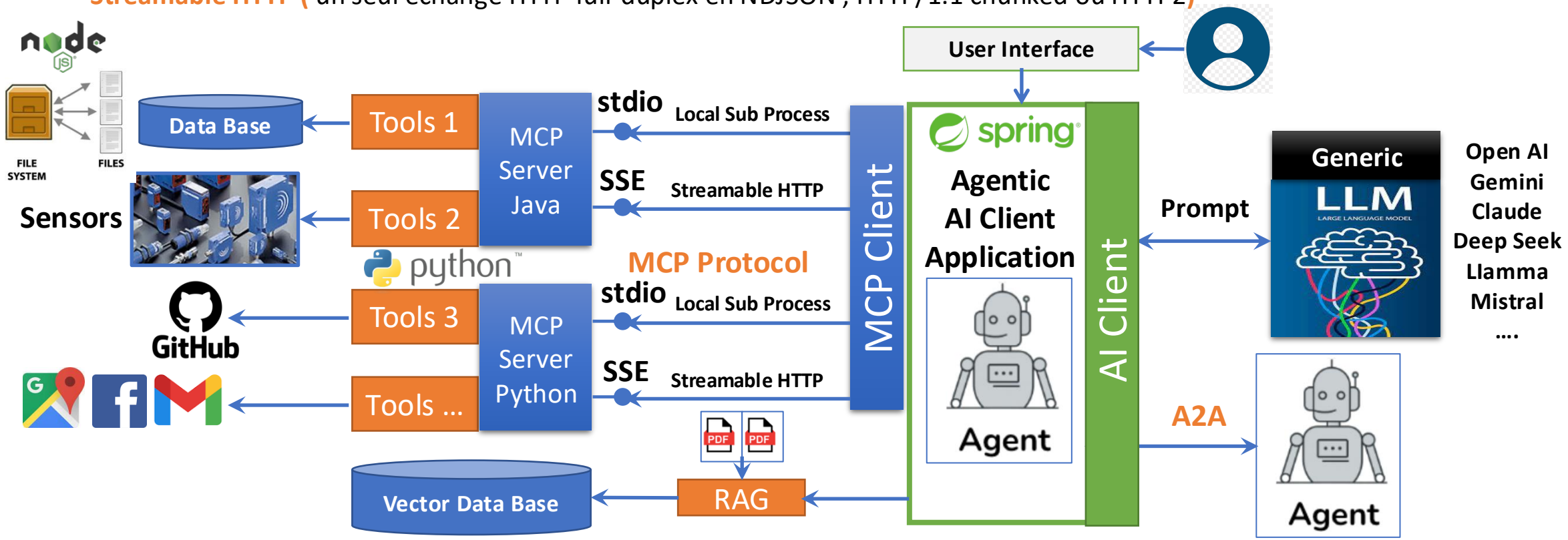
- No Code Low Code :
  - N8N, Agent Builder, etc.
- High Level Frameworks
  - Langchain
  - Crew AI
  - Agents SDK
  - Phi Data
  - Auto Gen
  - Smol Agent
- Low Level Frameworks :
  - LangGraph
  - Pour Java : Spring AI, Langchain4J

Model Context Protocol (MCP)

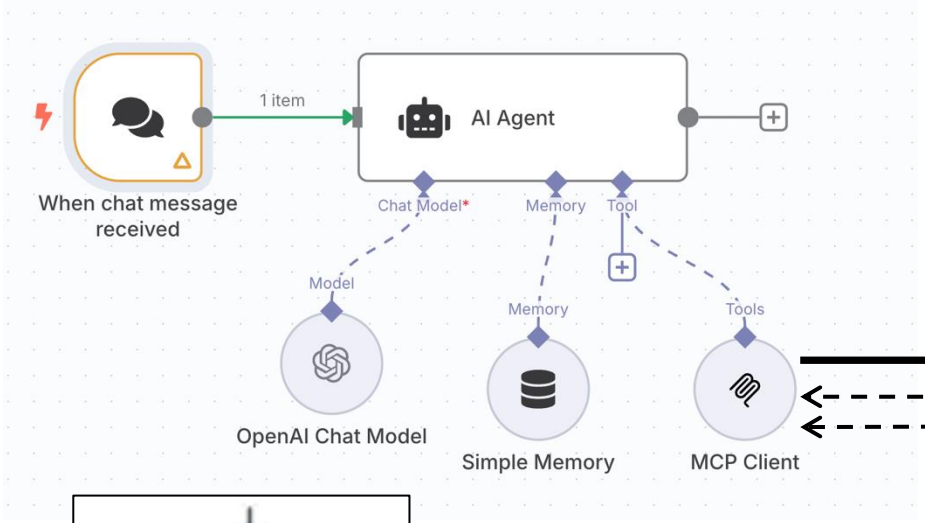
https://modelcontextprotocol.io

- MCP standardizes how to integrate **additional context and tools into the ecosystem of AI applications**
- The protocol uses **JSON-RPC 2.0 messages**
- The protocol defines two standard transport mechanisms for client-server communication:
  - **STDIO**, communication over **standard input** and **standard output**
  - **SSE (Server Sent Event :** serveur→client en streaming + POST séparés pour client→serveur
  - **Streamable HTTP (** un seul échange HTTP full-duplex en NDJSON , HTTP/1.1 chunked ou HTTP2**)**





MCP : Architecture



User Interface

Query

HTTP

Agent

1- Reasoning LLM

2- Action Tools

3- Observations Context

MCP Streaming HTTP SSE

MCP Inspector v0.15.0

POSTMAN

MCP

MCP Server

python

MCP Tools
WEB SERACH

HTTP

WEB

Open AI, GPT-4O

LLM

Mcp Server : Python

mcp_server.py

```
[project]
name = "lang-graph"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.13"
dependencies = [
    "ipykernel>=7.2.0",
    "ipython>=9.12.0",
    "langchain>=1.2.15",
    "langchain-mcp-adapters>=0.2.2",
    "langchain-openai>=1.2.0",
    "langchain-tavily>=0.2.18",
    "langgraph>=1.1.9",
    "langgraph-cli[inmem]>=0.4.24",

    "mcp[cli]>=1.27.0",

    "python-dotenv>=1.2.2",
]
```

$ uv run mcp_server.py

```
(lang_graph) $ uv run mcp-server.py
INFO:     Started server process [68385]
INFO:     Waiting for application startup.
[04/24/26 08:10:00] INFO     StreamableHTTP session manager started
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:23000 (Press CTRL+C to quit)
```

mcp_server.py

```python
from mcp.server.fastmcp import FastMCP
from langchain_tavily import TavilySearch
from dotenv.ipython import load_dotenv
load_dotenv(override=True)

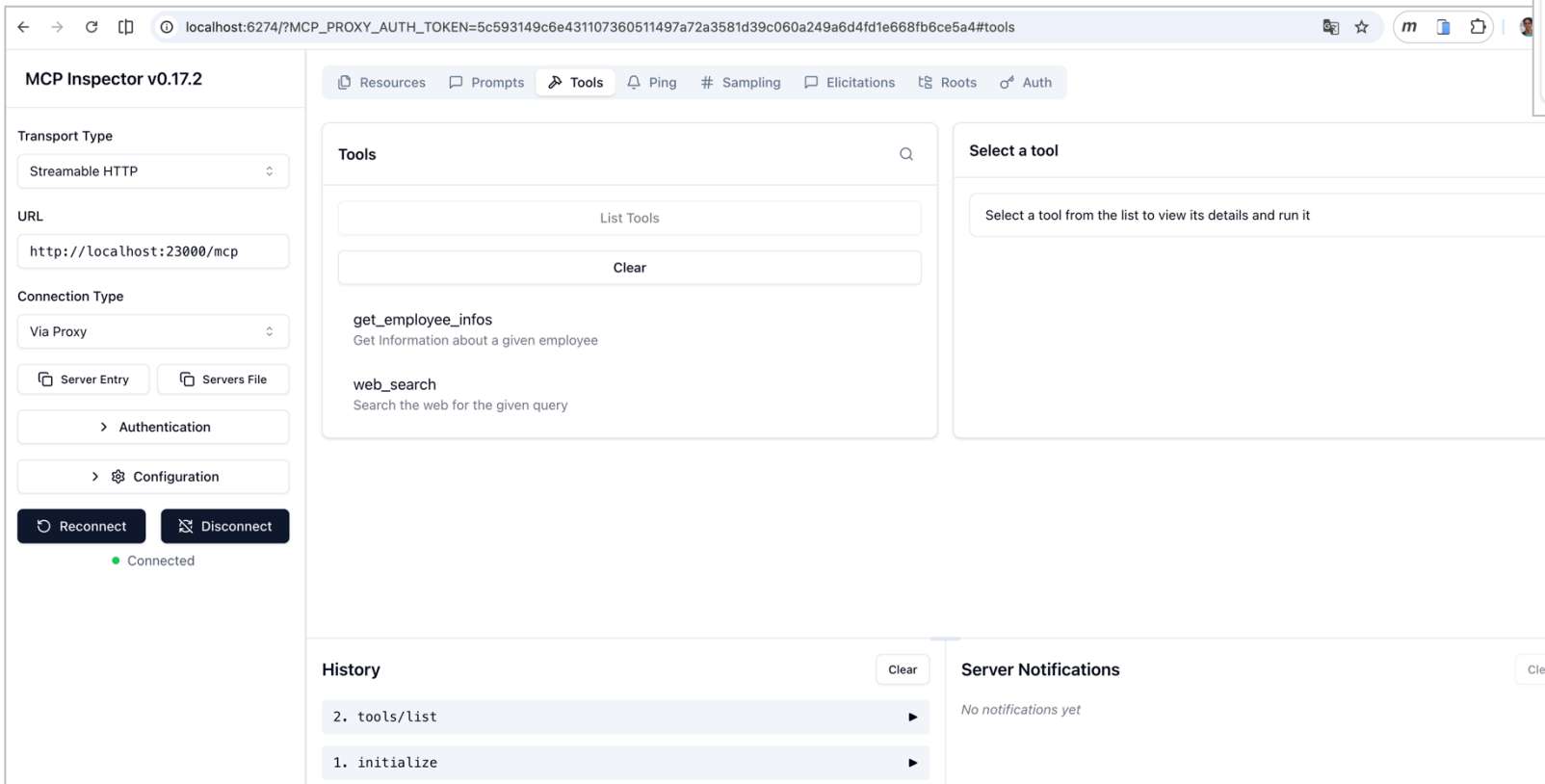
mcp = FastMCP(name="mcp-server", host="0.0.0.0", port=23000)
web_search_client = TavilySearch()
@mcp.tool()
def get_employee_infos(name: str):
    """
    Get Information about a given employee
    """
    return {"name": name, "salary": 21000, "seniority": 12}
@mcp.tool()
def web_search(query: str):
    """Search the web for the given query"""
    results = web_search_client.invoke({"query": query})
    return results
if __name__ == "__main__":
    mcp.run(transport="streamable-http")
```

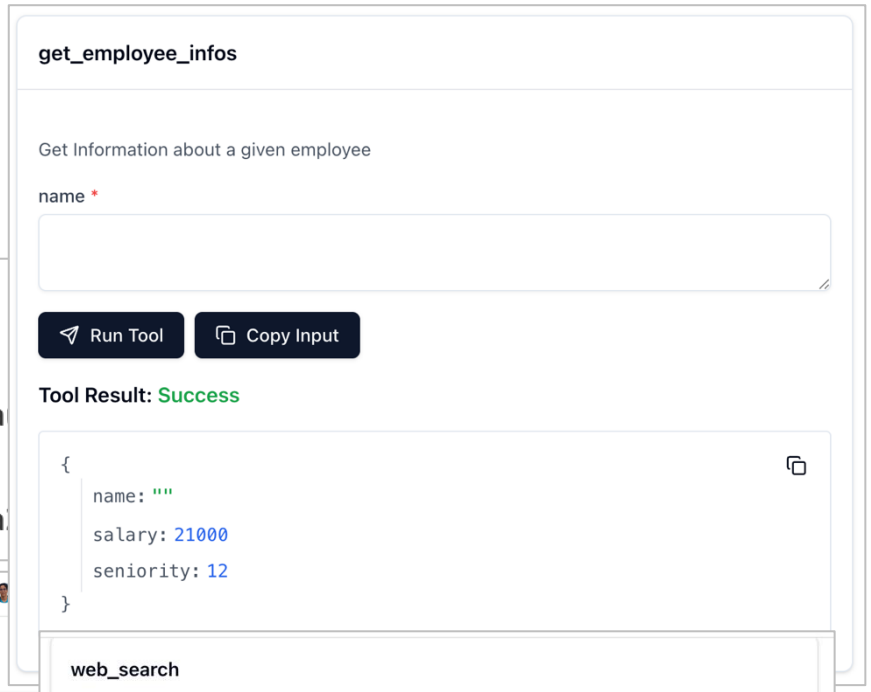
Tester un serveur MCP

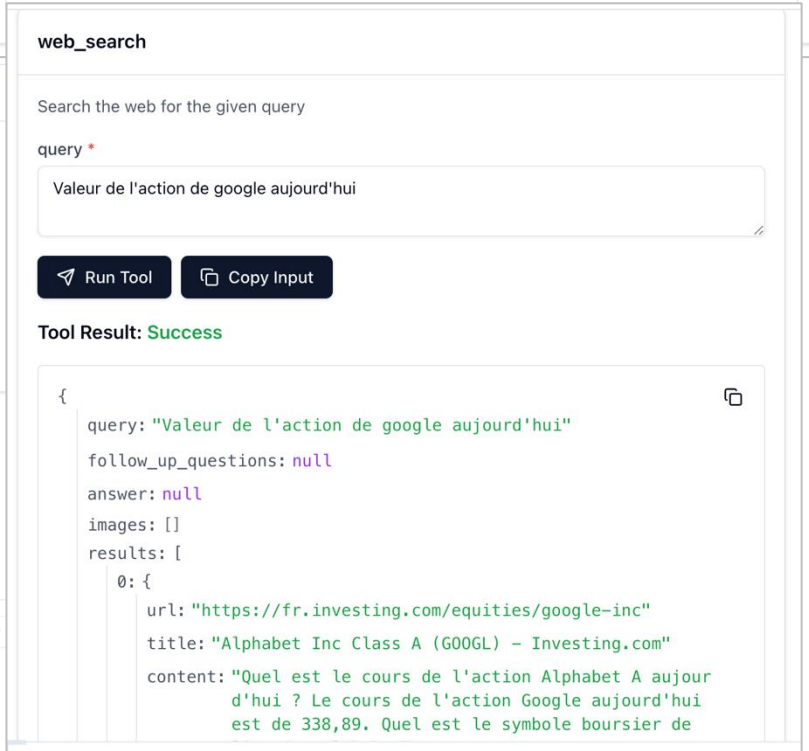
$ npx @modelcontextprotocol/inspector

```
(lang_graph) $ npx @modelcontextprotocol/inspector
Starting MCP inspector...
⚙️ Proxy server listening on localhost:6277
🔑 Session token: 5c593149c6e431107360511497a72a3581d39c060a249a6d4fd1e668fb6ce5a4
   Use this token to authenticate requests or set DANGEROUSLY_OMIT_AUTH=true to disable a

🚀 MCP Inspector is up and running at:
   http://localhost:6274/?MCP_PROXY_AUTH_TOKEN=5c593149c6e431107360511497a72a3581d39c060a
```







Python Mcp Client : Agent

mcp_server.py

```
[project]
name = "lang-graph"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.13"
dependencies = [
    "ipykernel>=7.2.0",
    "ipython>=9.12.0",
    "langchain>=1.2.15",
    "langchain-mcp-adapters>=0.2.2",
    "langchain-openai>=1.2.0",
    "langchain-tavily>=0.2.18",
    "langgraph>=1.1.9",
    "langgraph-cli[inmem]>=0.4.24",

    "mcp[cli]>=1.27.0",

    "python-dotenv>=1.2.2",
]
```

graph.py

```python
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI
from dotenv.ipython import load_dotenv
from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain.messages import HumanMessage
import asyncio
load_dotenv(override=True)

mcp_client = MultiServerMCPClient(
    {
        "mcp_server": {
            "transport": "streamable_http",
            "url": "http://localhost:23000/mcp",
        }
    }
)

async def main():
    tools = await mcp_client.get_tools()
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    agent = create_agent(
        model=llm, tools=tools,
        system_prompt="answer the user question using provided tools",
    )
    while True:
        user_query = input("Question:")
        if user_query == "exit":
            break
        response = await agent.ainvoke({"messages": [HumanMessage(user_query)]})
        print(response["messages"][-1].content)
if __name__ == "__main__":
    asyncio.run(main())
```

$ uv run graph.py

Question:Salaire de Mohamed
Le salaire de Mohamed est de **21 000.**

Question:Valeur de l'action de google aujourd'hui

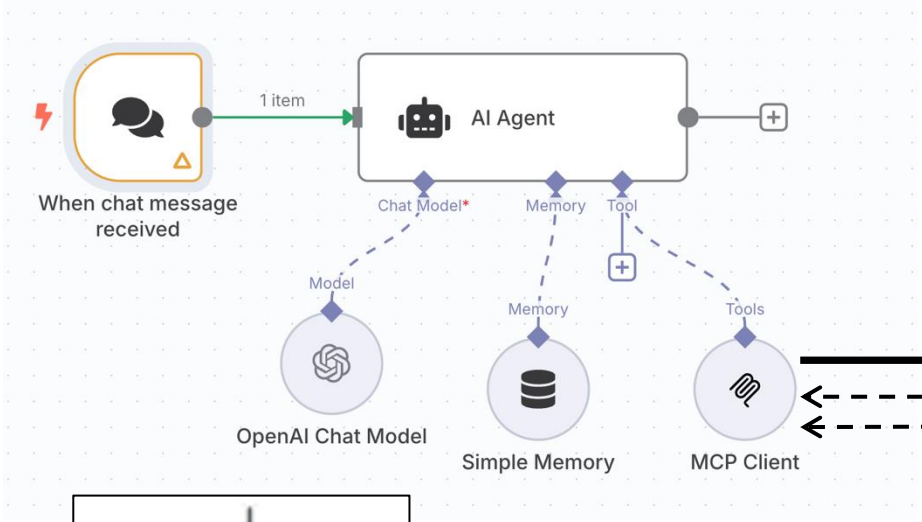
Aujourd'hui, la valeur de l'action de Google (Alphabet Inc) est d'environ **338,89 USD**. Pour plus de détails, vous pouvez consulter
[Investing.com](https://fr.investing.com/equities/google-inc)
Question:

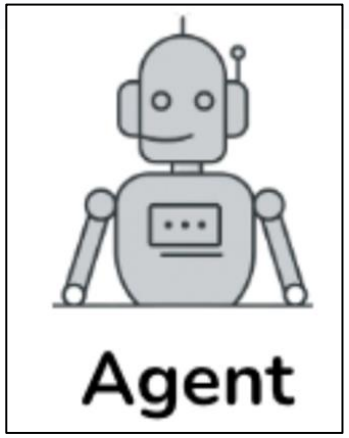
MCP Client : N8N

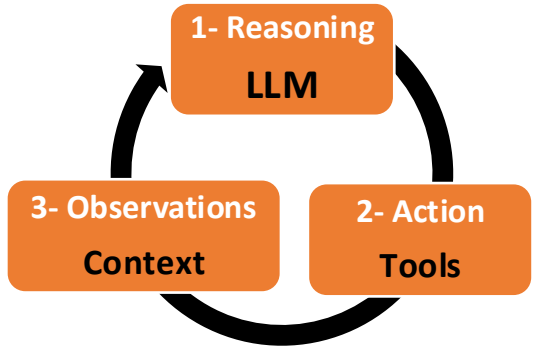
User Interface

Query

HTTP







MCP
Streaming HTTP
SSE



POSTMAN

MCP Inspector v0.15.0

MCP

MCP Server

python

MCP Tools
WEB SERACH

HTTP

WEB

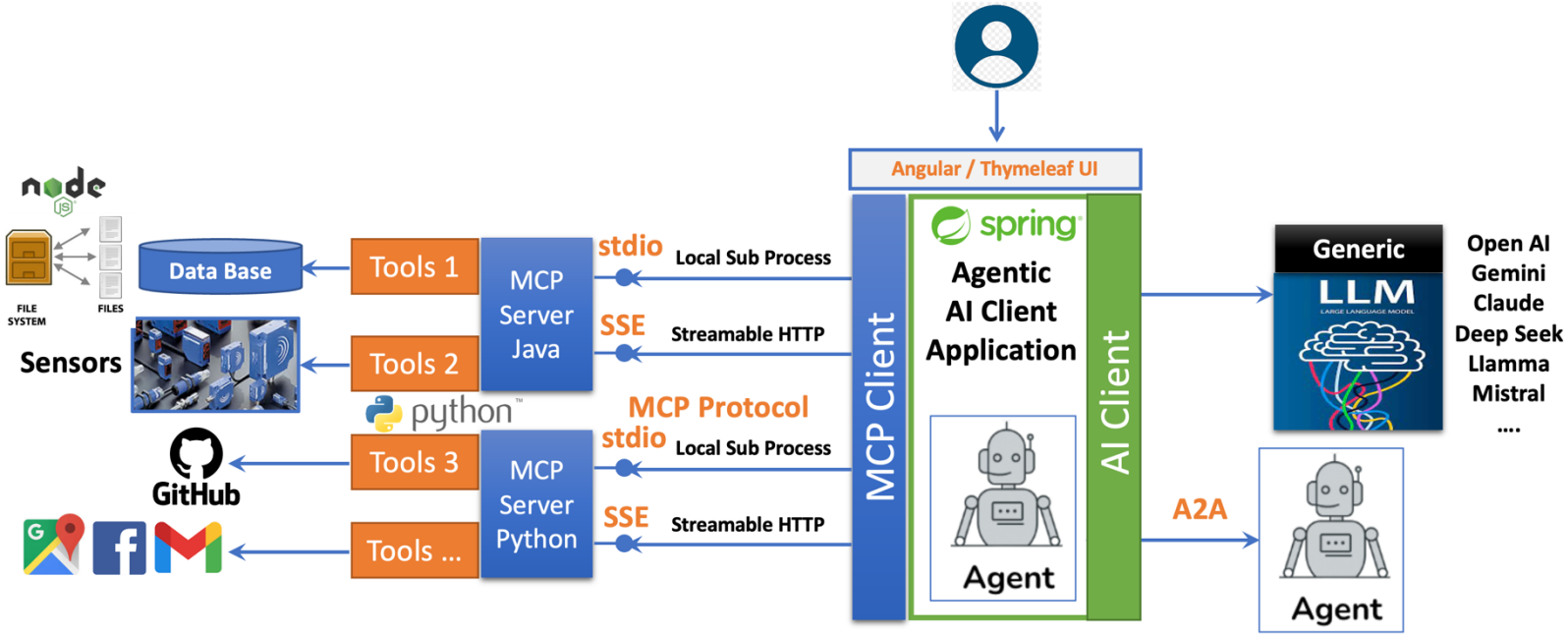
Open AI, GPT-4O

LLM

MCP

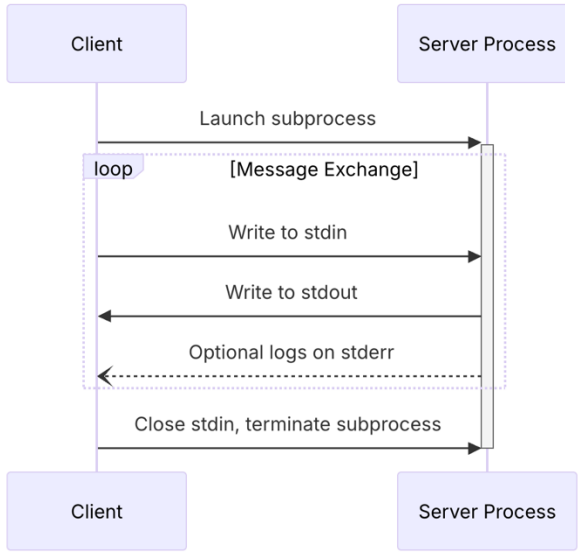
- Model Context Protocol (MCP) is an open protocol that enables seamless integration between LLM applications and external data sources and tools.
- MCP provides a standardized way for applications to:
  - Share contextual information with language models
  - Expose tools and capabilities to AI systems
  - Build composable integrations and workflows

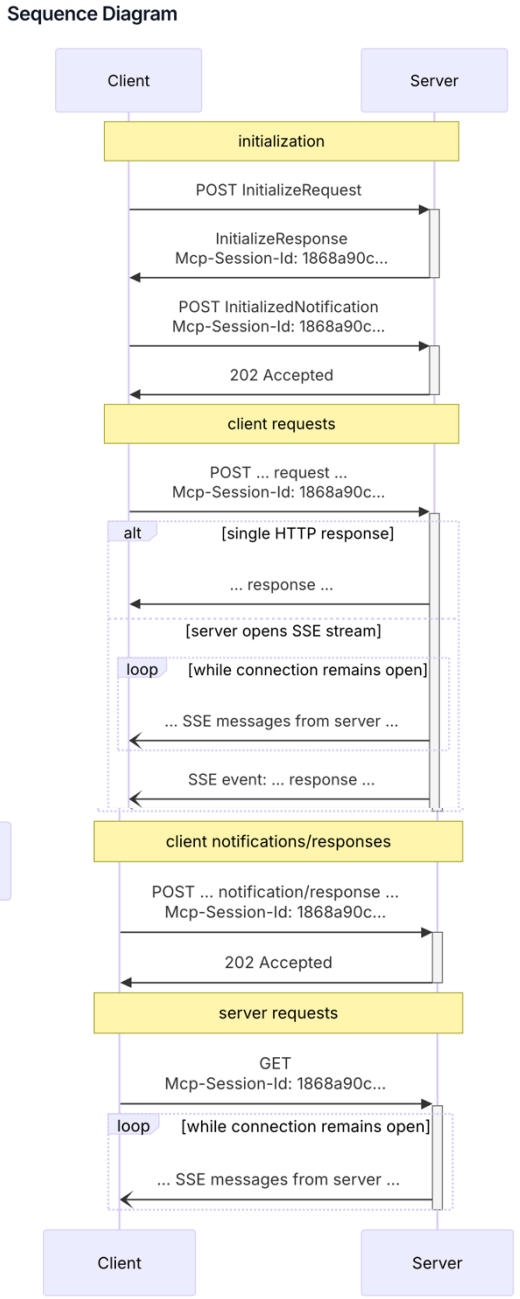
- The protocol uses **JSON-RPC 2.0 messages** to establish communication between:
  - **Hosts**: LLM applications that initiate connections
  - **Clients**: Connectors within the host application
  - **Servers**: Services that provide context, tools and capabilities
- MCP standardizes how to integrate **additional context and tools into the ecosystem of AI applications**.



MCP : Transport (stdio)

- The protocol currently defines two standard transport mechanisms for client-server communication:
  - **stdio**, communication over **standard in** and **standard out**
  - **Streamable HTTP (SSE)**
- **STDIO**
  - In the **stdio** transport:
    - The client launches the MCP server as a subprocess.
    - The server reads JSON-RPC messages from its standard input (stdin) and sends messages to its standard output (stdout).
- **Streamable HTTP (SSE)** :
  - In the **Streamable HTTP** transport, the server operates as an independent process that can handle multiple client connections.
  - This transport uses HTTP POST and GET requests.
  - Server can optionally make use of **Server-Sent Events (SSE)** to stream multiple server messages.
  - The server **MUST** provide a single HTTP endpoint path that supports both POST and GET methods. For example, this could be a URL like **https://example.com/mcp**.

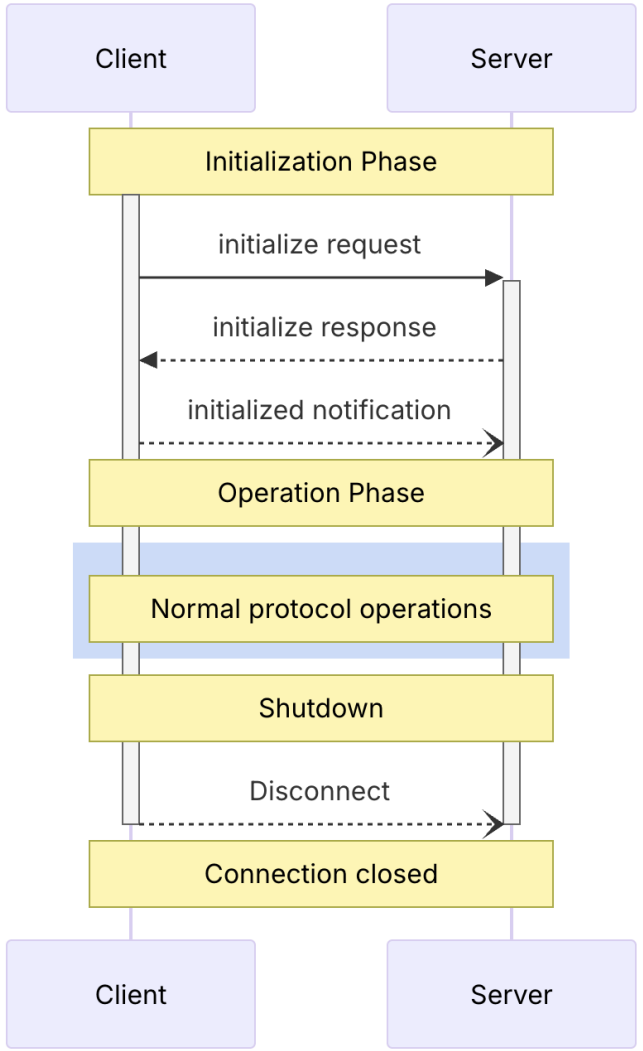




MCP Lifecycle

https://modelcontextprotocol.io/specification/2025-03-26/basic/lifecycle

- **Base Protocol** :
  - JSON-RPC message format
  - Stateful connections
  - Server and client capability negotiation
- **Features :**
  - Servers offer any of the following features to clients:
    - **Resources**: Context and data, for the user or the AI model to use
    - **Prompts**: Templated messages and workflows for users
    - **Tools**: Functions for the AI model to execute
- **MCP Lifecycle:**
  - **Initialization**: Capability negotiation and protocol version agreement
  - **Operation**: Normal protocol communication
  - **Shutdown**: Graceful termination of the connection



MCP Lifecycle : Connection

GET http://localhost:8089/sse Close

Params Authorization Headers (7) Body Scripts Tests Settings Cookies

Query Params

	Key	Value	Description
	Key	Value	Description

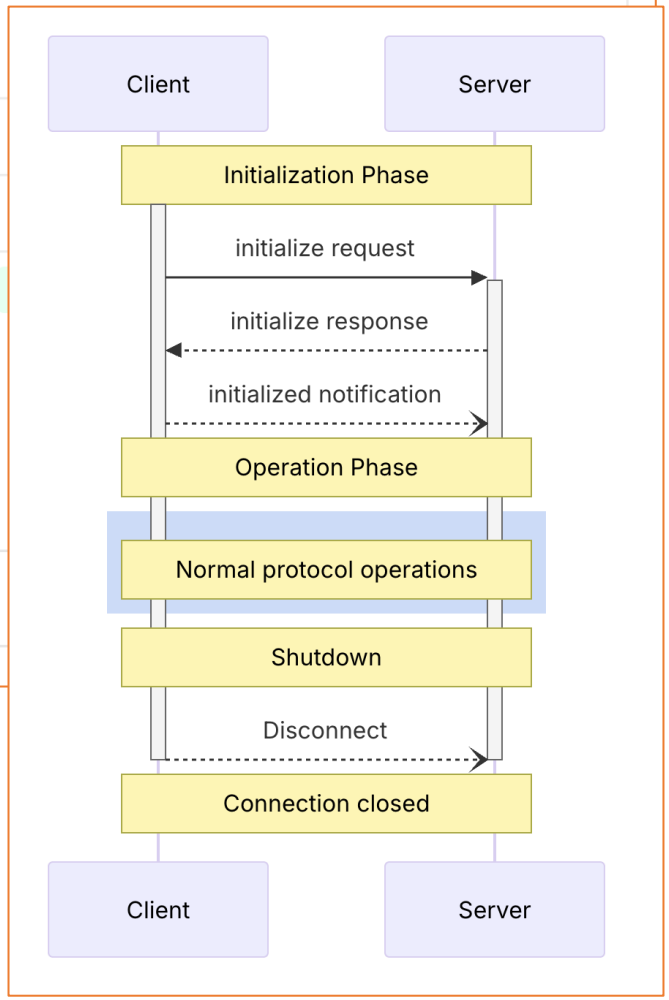
Body Cookies Headers (6) Test Results Receiving Response

Search

endpoint /mcp/message?sessionId=edfa8a68-a3fd-4b53-8cb3-eaa3abc1bd58

Connected to http://localhost:8089/sse

$ npx @modelcontextprotocol/inspector



MCP Lifecycle : Initialization request

POST | localhost:8089/mcp/message?sessionId=edfa8a68-a3fd-4b53-8cb3-eaa3abc1bd58 | Send

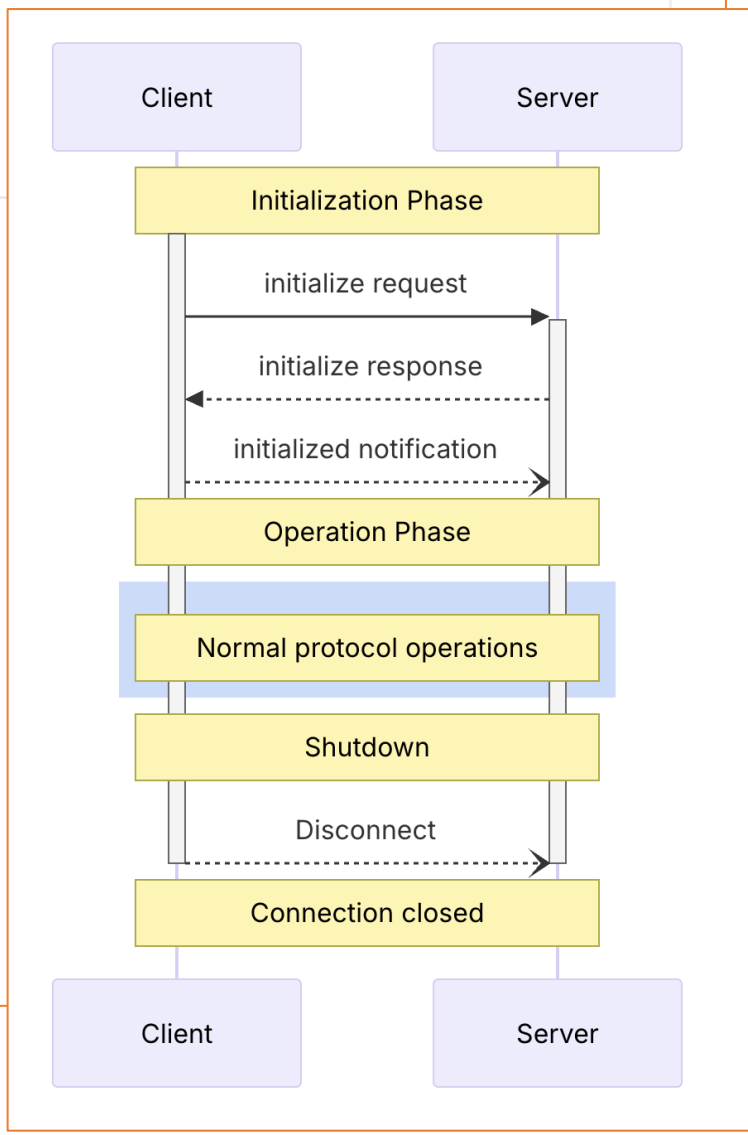
Params • | Authorization | Headers (9) | Body • | Scripts | Tests | Settings

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON

```json
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "initialize",
  "params": {
    "protocolVersion": "2024-11-05",
    "capabilities": {
      "roots": {
        "listChanged": true
      },
      "sampling": {}
    },
    "clientInfo": {
      "name": "ExampleClient",
      "version": "1.0.0"
    }
  }
}
```

Server Message Response

```json
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "protocolVersion": "2024-11-05",
    "capabilities": {
      "completions": {},
      "logging": {},
      "prompts": {
        "listChanged": true
      },
      "resources": {
        "subscribe": false,
        "listChanged": true
      },
      "tools": {
        "listChanged": true
      }
    },
    "serverInfo": {
      "name": "my-mcp-server",
      "version": "1.0.0"
    }
  }
}
```



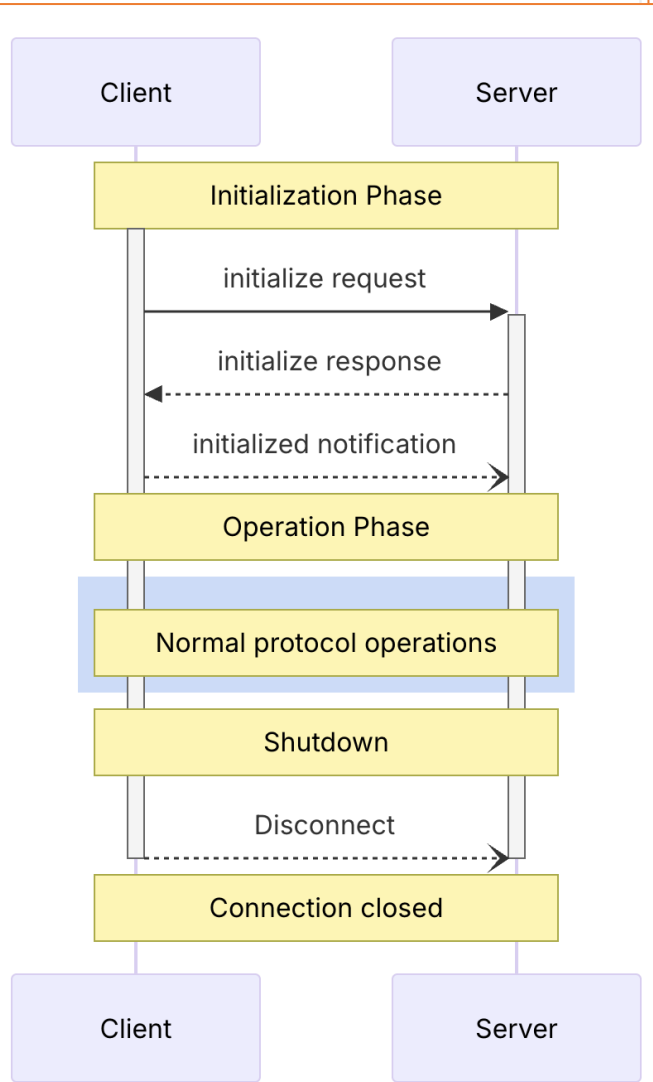
MCP Lifecycle : Initialization notification

POST | localhost:8089/mcp/message?sessionId=479a49ec-914f-4be8-9d61-93634e3c6a85 | Send

Params • | Authorization | Headers (9) | Body • | Scripts | Tests | Settings

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON

```
1  {
2    "jsonrpc": "2.0",
3    "method": "notifications/initialized"
4  }
```



MCP Lifecycle : Operation phase (tools/list)

POST localhost:8089/mcp/message?sessionId=479a49ec-914f-4be8-9d61-93634e3c6a85 Send

Params • Authorization Headers (9) Body • Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

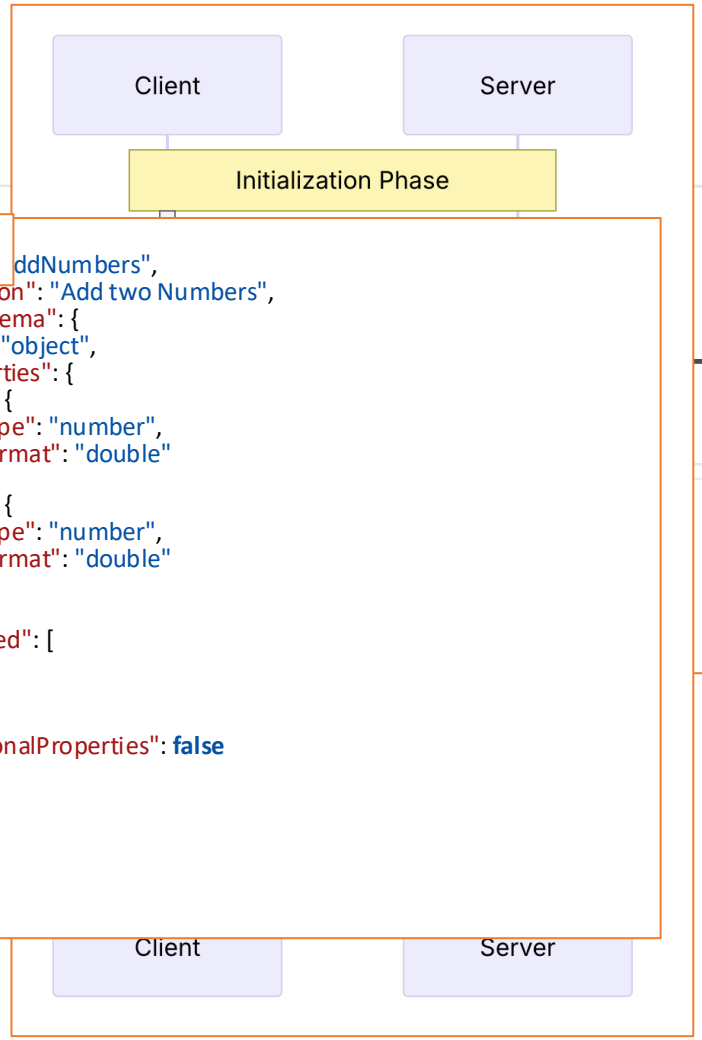
```
{
  "jsonrpc": "2.0",
  "method": "tools/list",
  "id":1
}
```

Body Cookies Headers (4) Test Results

Raw Preview Visualize

Server Message Response


```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "tools": [
      {
        "name": "multiplyNumbers",
        "description": "Multiply two Numbers",
        "inputSchema": {
          "type": "object",
          "properties": {
            "n1": {
              "type": "number",
              "format": "double"
            },
            "n2": {
              "type": "number",
              "format": "double"
            }
          },
          "required": [
            "n1",
            "n2"
          ],
          "additionalProperties": false
        }
      },
      {
        "name": "addNumbers",
        "description": "Add two Numbers",
        "inputSchema": {
          "type": "object",
          "properties": {
            "n1": {
              "type": "number",
              "format": "double"
            },
            "n2": {
              "type": "number",
              "format": "double"
            }
          },
          "required": [
            "n1",
            "n2"
          ],
          "additionalProperties": false
        }
      }
    ]
  }
}
```

MCP Lifecycle : Operation phase (tools/call)

POST localhost:8089/mcp/message?sessionId=3c907428-fb16-465e-8b27-a615f658d34d **Send**

Params • Authorization Headers (9) Body • Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

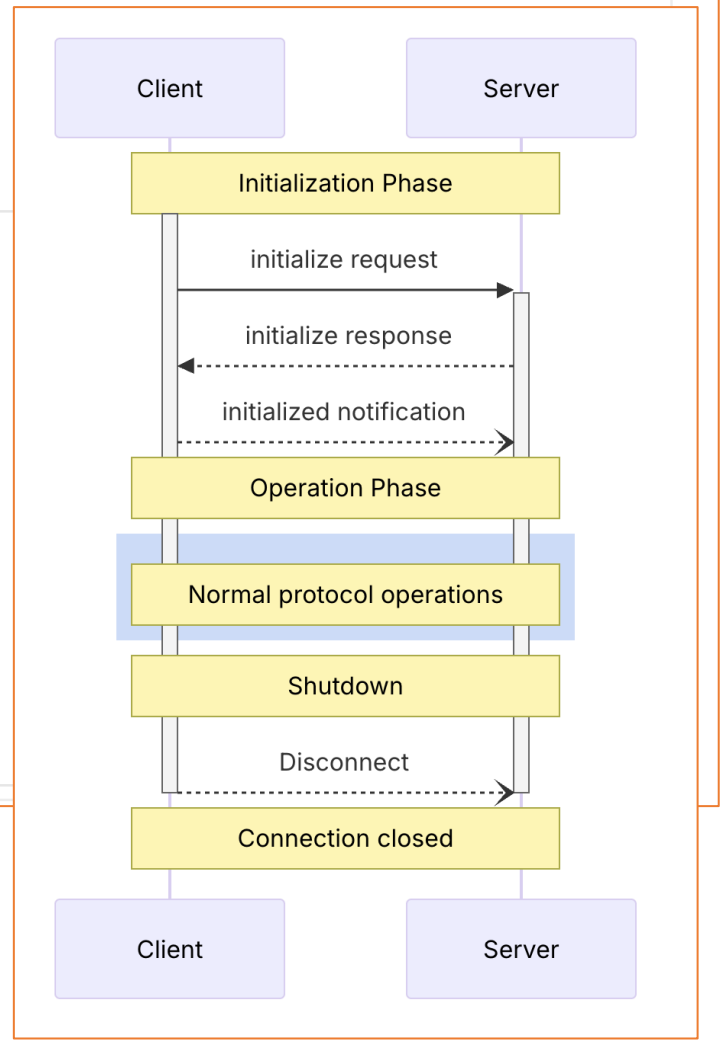
```
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "params":{
    "name":"addNumbers",
    "arguments":{
        "n1":34,
        "n2":11
    }
  },
  "id":1
}
```

Server Message Response

message {"jsonrpc":"2.0","id":1,"res

JSON

```
{
    "jsonrpc": "2.0",
    "id": 1,
    "result": {
        "content": [
            {
                "type": "text",
                "text": "45.0"
            }
        ],
        "isError": false
    }
}
```



MCP Lifecycle : Operation phase (tools/call)

Untitled MCP Request

Save | Share

HTTP | localhost:8089/sse | Run

Message | Authorization | Headers | Settings

Tools | Prompts | Resources

multiplyNumbers

Multiply two Numbers

n1* : number

78

n2* : number

12

2. addNumbers

Add two Numbers

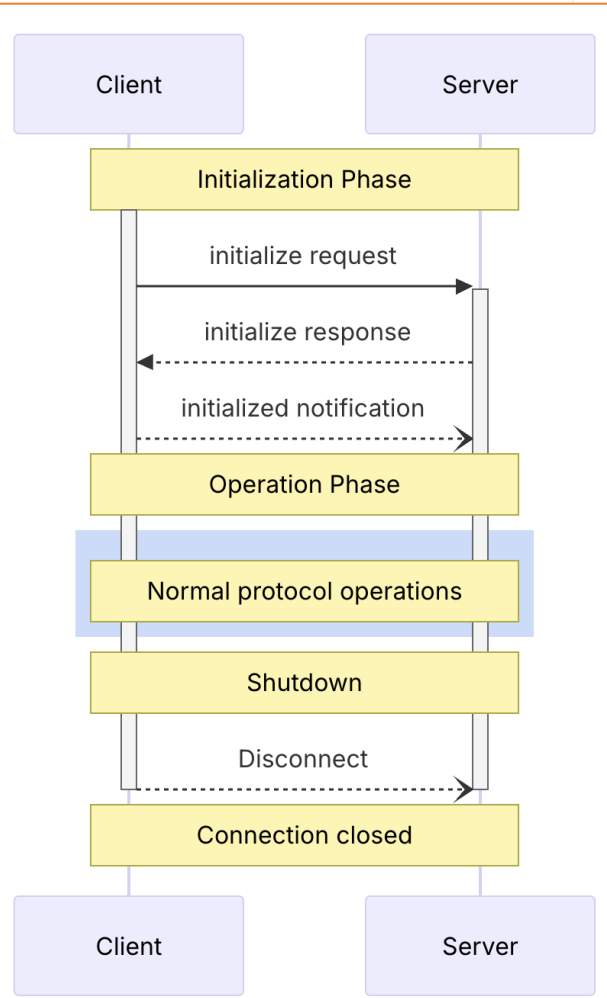
Response | Notifications

Search | All Messages

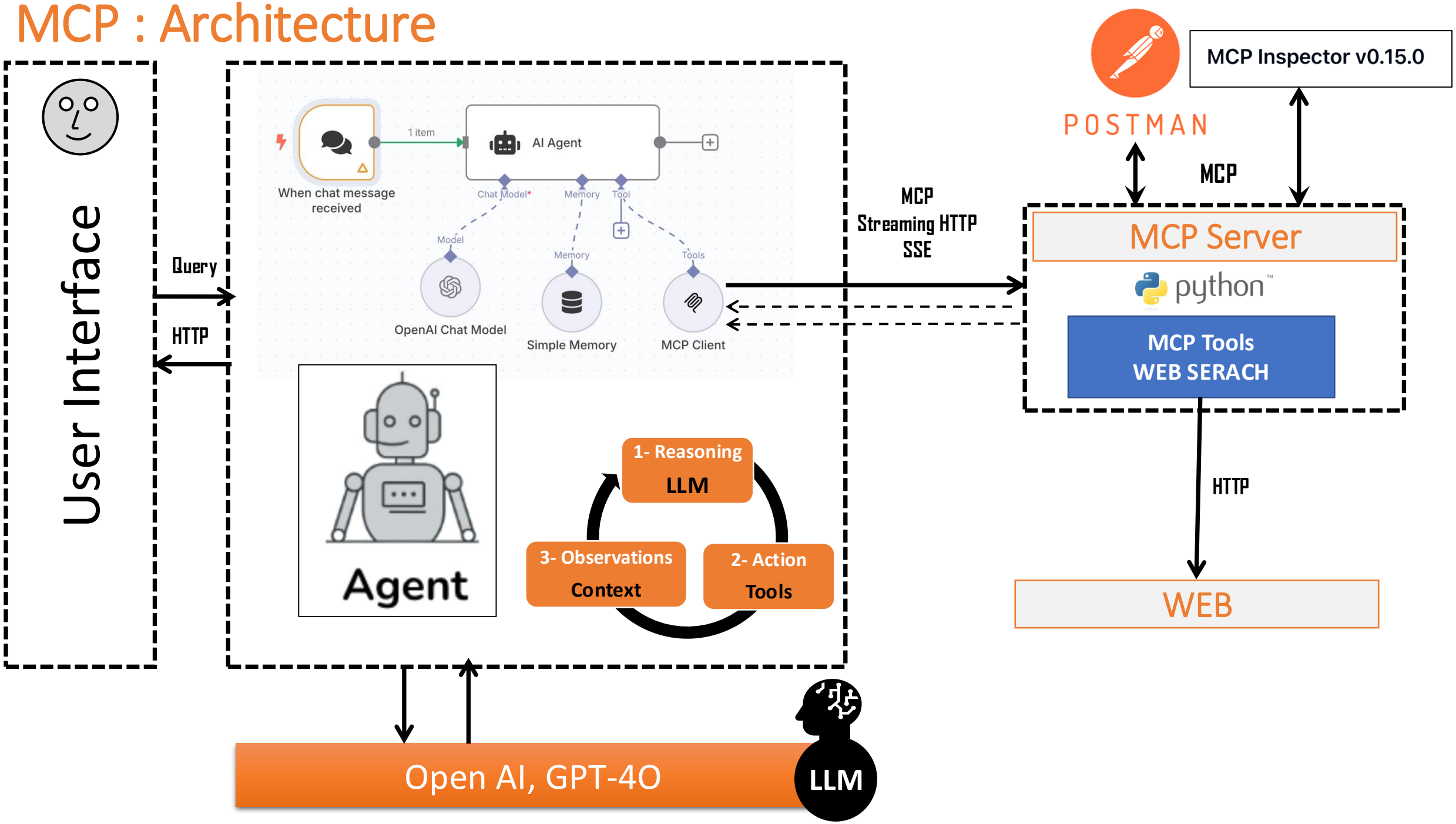
Connected 23:12:16.076

Server Message Response

```
{
  "method": "tools/call",
  "params": {
    "name": "multiplyNumbers",
    "arguments": {
      "n1": 78,
      "n2": 12
    }
  }
}
```



MCP : Architecture



Remote MCP Server – AI Agent – N8N

Mohamed Youssfi, Enseignant Chercheur, ENSET Mohammedia, Directeur Laboratoire Informatique, Intelligence Artificielle et Cyber Sécurité



MCP : Architecture

